

# Shooting World Learning Guide Document

---

## Learning Goals

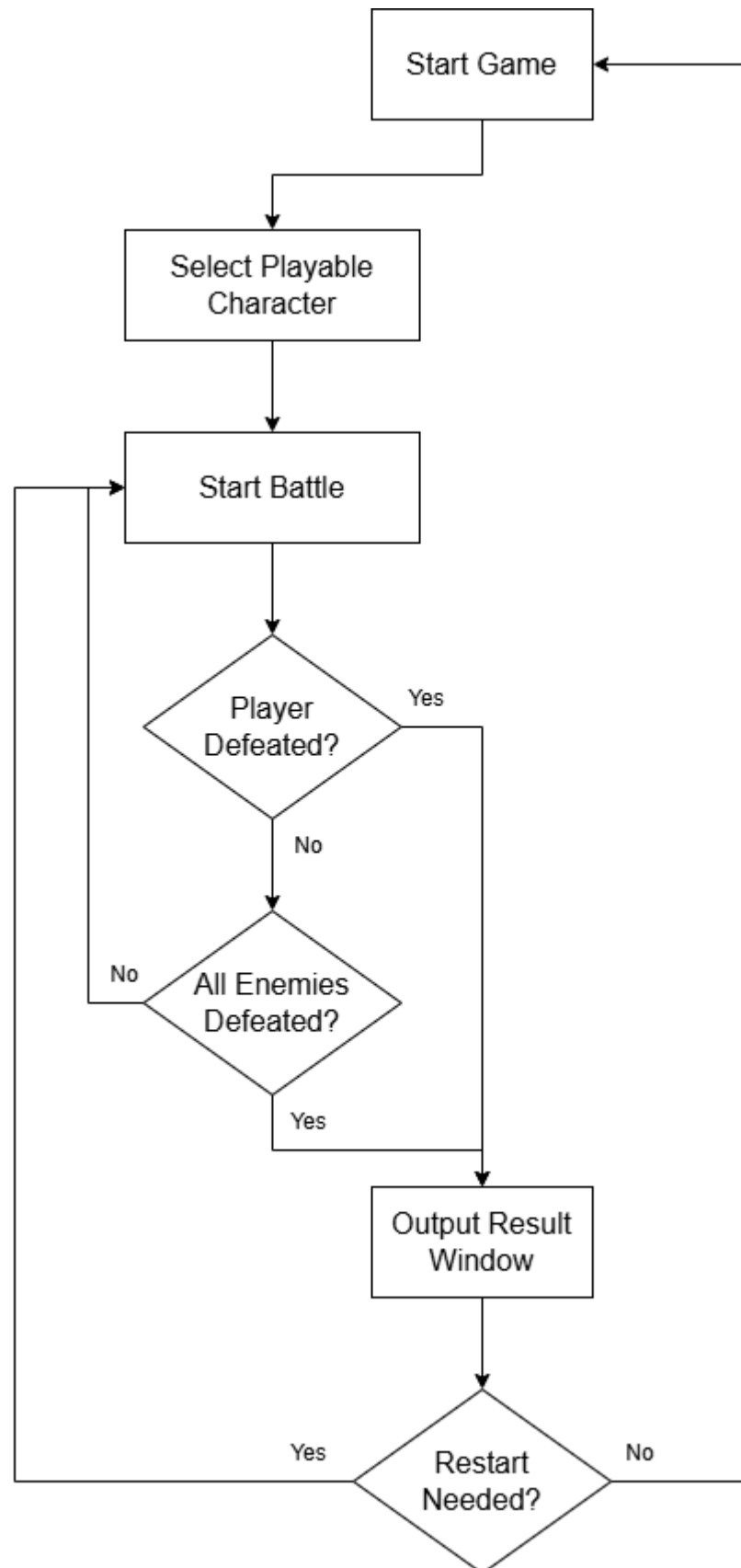
- Learn how to implement core shooting game functions using the guide document and the sample world.
  - Learn how to set the map up based on the game that is to be created in MapleStory Worlds.
  - Understand Logic and design ways to use it.
  - Modify and create a world in the form of your choosing.
- 

## Learning Core Functions

- **Managing DataSet:** Learn how to use the Manager that manages a DataSet with a single Logic by utilizing DataTableLogic.
- **Managing Collisions Between Entities:** Set and use collision conditions by utilizing the TriggerComponent's Collision Group and Matrix.
- **Object Pool:** Learn the object pool technique and create one to manage multiple projectiles.



## Core Gameplay Loop



**1. Start Game:** Start the game or learn how to play from the title screen.

**2. Select Play Character:** When the Start Game button is pressed on the title screen, the UI for selecting a character is displayed.

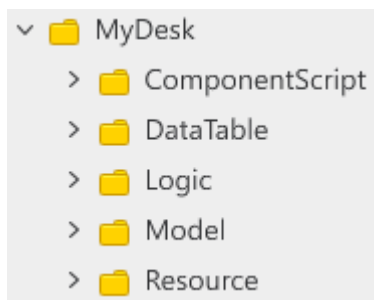
**3. Start Battle:** Proper start to the shooting game. Game interactions for shooting games like monster spawns and player attacks happen here.

**4. Check Whether All Enemies Are Dead:** Checks whether the requisite number of monsters to clear the current map have been defeated.

**5. Confirm Results:** Produces a results screen based on whether or not the player defeated all monsters.

**6. Check for Restarts:** Go to the corresponding screen based on whether the player requested a restart or chose to return to the title.

## Folder Composition



| Name            | Description                                                                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ComponentScript | The folder containing components added to entities to actualize the world.                                                                                                   |
| DataTable       | The folder containing the DataSet needed to compose the sample world.                                                                                                        |
| Logic           | The folder containing scripts created with the MapleStory Worlds Logic function.                                                                                             |
| Model           | The folder containing entities that went through modelization.                                                                                                               |
| Resource        | The folder containing Resources needed for game creation.<br>It currently only contains the TileSet, but you can add the resources needed to create the world in the future. |

## Implementation Order

-  **Basic Training:** The training material for the most basic functions for MapleStory Worlds or for the relevant genre.
-  **Advanced Training:** Category of training for learning and implementing functions visible in the sample world. Using these materials can expand the functions for the genre and understanding of the training materials.

### [ Basic Training ] 1. Setting a Map

- Learn about the types of maps provided by MapleStory Worlds.
- Learn about the relationship between the BodyComponent and map types.
- Learn about the KinematicBodyComponent.
- Learn how to compose maps by placing RectTiles.

## [💡 Basic Training] 2. Declaring Logic

- Learn about the DataSet components for composing players.
- Learn how to load DataSets through DataTableLogic and how to use them.
- Learn how to create a Component that will contain the saved data and how to apply it.
- Learn how to add and edit TriggerComponent.

## [💡 Basic Training] 3. Creating a Playercharacter

- Learn about the DataSet components used to configure players.
- Learn how to load and use DataSets through DataTableLogic.
- Explore how to create and apply Components for storing saved data.
- Learn how to assign and edit TriggerComponents.

## [💡 Basic Training] 4. Creating Monsters

- Learn about the DataSet components required to compose monsters.
- Learn how to load and use DataSets through DataTableLogic.
- Explore how to create and apply Components for storing saved data.
- Learn how to assign and edit TriggerComponents.
- Learn how to craft and use the AIChaseComponent, so the monsters chase after the player.
- Learn Modelization for reuse of created monsters.

## [💡 Basic Training] 5. Creating Basic Attacks

- Learn about the DataSet components required to compose projectiles.
- Learn how to load and use DataSets through DataTableLogic.
- Explore how to create and apply Components for storing saved data.
- Learn how to assign and edit TriggerComponents.
- Learn about Collision Groups and how to edit using the Matrix.
- Add a function that makes a projectile fly in the set direction.

## [💡 Basic Training] 6. Creating an Object Pool

- Understand the objective of having an object pool by learning the concepts of the object pool.
- Implement the object pool Component.
- See for yourself how the object pool functions by creating and returning one.
- Learn how to implement the player and monster projectile firing system.

## [📖 Advanced Training] 7. Implementing Skills

- Learn about the DataSet components required to compose skills.
- Learn how to load skill related DataSets through DataTableLogic and how to use them.
- Learn how to create and implement Components that will contain the saved skill data.
- Learn how to add and edit a TriggerComponent to a skill entity.
- Learn how to convert a created entity into a Model and use it.
- Add a function that makes a projectile type skill fly in the set direction.
- Learn about the TriggerEvent used to implement projectile-type skill functions.



## [📖 Advanced Training] 8. Implementing Items

- Learn about skill DataSet components required to compose items.
- Learn how to load an item DataSet through DataTableLogic and how to use it.
- Add and edit components to apply item data and item spawn data.
- Learn how to create a Model that composes items.
- Create an ItemHelper Logic that is in charge of an item's spawn chance and item spawns.
- Learn how to implement functions, so item effects can be processed normally.
- Learn how to spawn a Monster to apply an item spawn chance.

## [📖 Advanced Training] 9. Creating a MonsterSpawn

- Implement a function, so a Monster appears in InGame based on the value entered in the DataSet.
- Learn about the DataSet structure and how to use said structure.
- Add an item drop chance to each spawning monster to implement a function that drops items based on a set chance when monsters are defeated.

## [💡 Basic Training] 10. Composing UI

- Learn about UIGroup, its components, and how to edit it.
- Learn how to edit the UI by creating a title screen.
- Create a Component that controls the title screen, so the target UI's function can be processed.
- Learn how to create a SelectCharacterUI that allows the player to select a character.
- Learn how to compose a DefaultUI that controls the in-game information.
- Create a Component that controls DefaultUI to implement a function that updates the remaining HP and skill use count.

---

## [💡 Basic Training] Setting a Map

---

### Video Timeline

- You can follow along by using the timeline under the training video.
- Understanding Map Types: 00:01:29

### Learning Goals

- Learn about MapleStory Worlds's map methods and select what kind of map method you will need when making a world.
- Learn to identify which components can interact with platform tiles on the map based on each map's method.
- Learn about the KinematicBodyComponent for interactions with the RectTileMap.
- Learn how to place tiles on the RectTileMap and formulate a map.

## Practical Exercises to Do After Watching the Video

- Changing the Map format to a RectTileMap for the creation of a shooting world map.
- Create a TileSet.
- Use the created TileSet to create an interactable Map.

## Map Types

### Key Point

The game's POV changes based on map format!

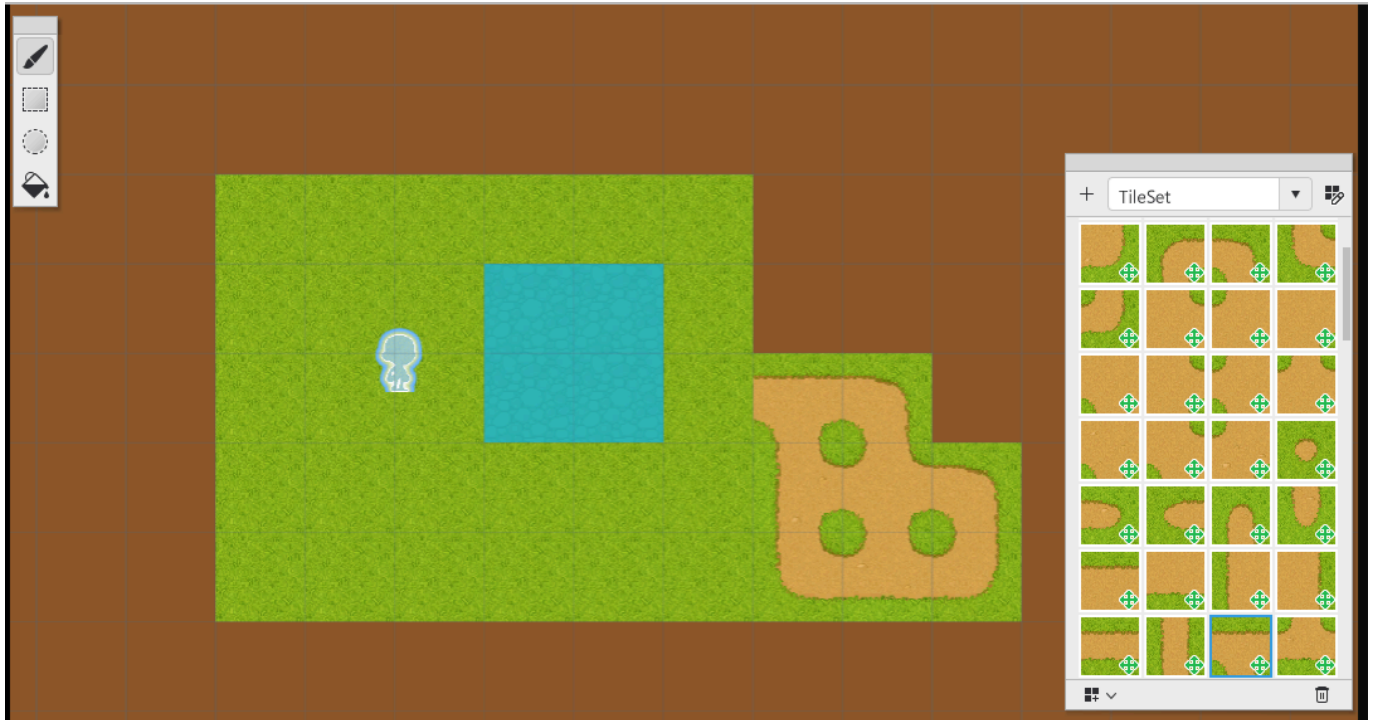
| Map Format          | Component Used          | POV Applied                                   |
|---------------------|-------------------------|-----------------------------------------------|
| TileMap             | Rigidbody Component     | Side-Scrolling Game Reminiscent of MapleStory |
| RectTileMap         | Kinematicbody Component | Top-Down View                                 |
| SideViewRectTileMap | Sideviewbody Component  | Side-Scrolling Game Using a Gridded Tile      |

## TileMap



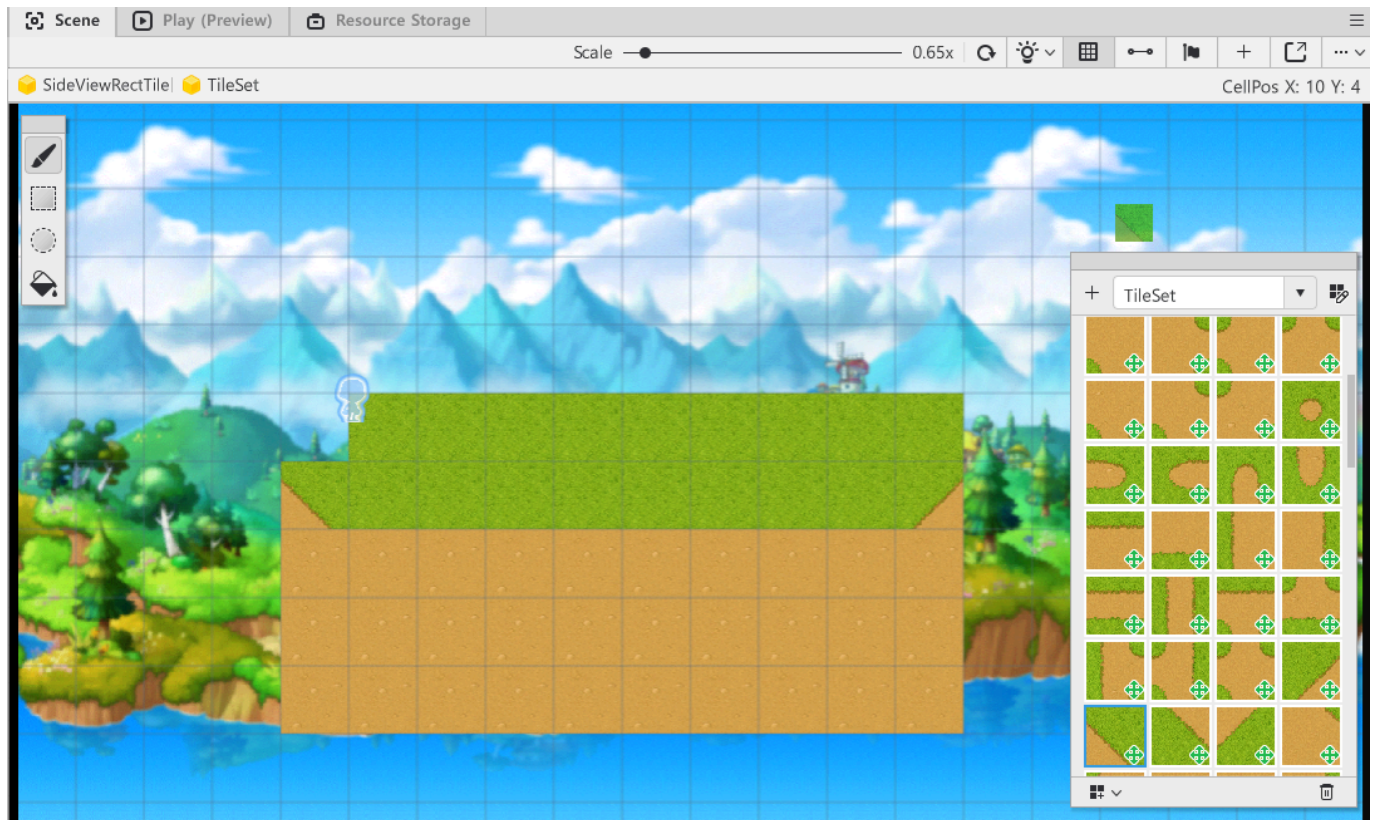
- A map format that is set up as the default when creating a world for the first time in MapleStory Worlds.
- A world that is best suited for the feel of MapleStory.
- Formulate a map by utilizing Tiles provided by MapleStory Worlds to create platforms and place entities.
- Create a ramp based on how the tiles are placed.
- A useful map format when you want to make a side scrolling-type game reminiscent of MapleStory.

## RectTileMap



- A map format that is effective in making top-down view type games.
- Create your own tile sets or use the tile set resources provided by MapleStory Worlds to formulate a map.
- You can formulate a map by placing tiles in a gridded format.
- You can create mobile and immobile areas through each tile.
- Create a game where you can move in 4 or 8 directions, unlike with the TileMap and SideViewRectTileMap methods.

## SideViewRectTileMap



- A map format that is effective in making side scrolling-type games.
- Create your own tile sets or use the tile set resources provided by MapleStory Worlds to formulate a map.
- You can formulate a map by placing tiles in a gridded format.
- The character moves **through** tiles if the tile set doesn't prevent it, unlike with RectTileMap.
- So if you formulate a map with SideViewRectTileMap, the tile set that prevents movement must be used to create **platforms**.

## BodyComponent

- The BodyComponent is required to interact with tiles that are platforms in MapleStory Worlds.
- Different BodyComponents are needed based on the map type.
- There are 3 main types of BodyComponents.

## RigidBodyComponent

▼

RigidbodyComponent

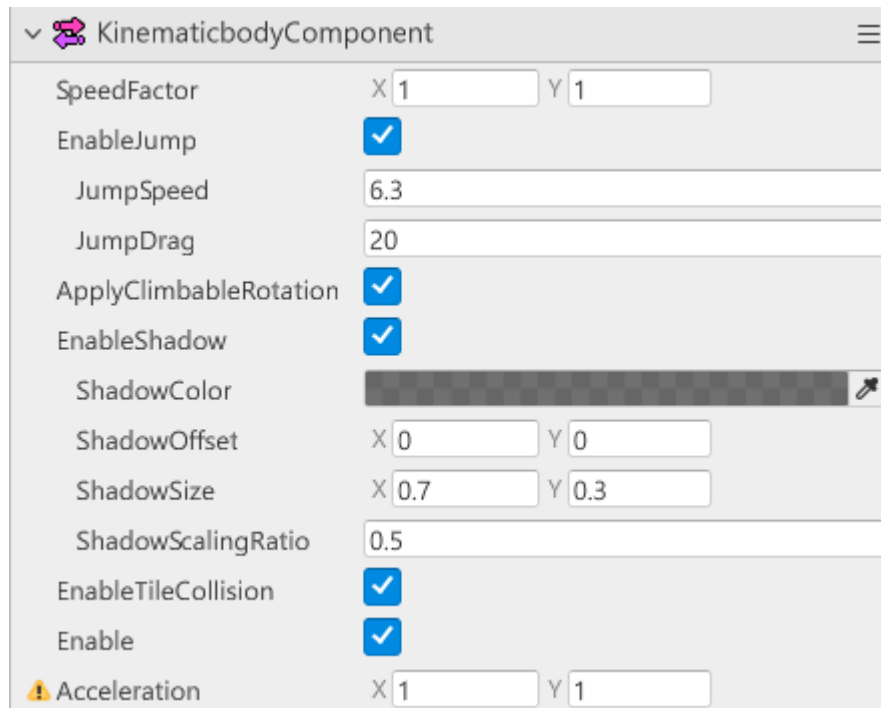
☰

|                        |                                     |
|------------------------|-------------------------------------|
| Mass                   | 100                                 |
| Gravity                | 1                                   |
| JumpBias               | 0.03                                |
| WalkJump               | 1.23                                |
| DownJumpSpeed          | 2                                   |
| WalkSpeed              | 1.4                                 |
| WalkAcceleration       | 1                                   |
| WalkDrag               | 1                                   |
| WalkSlant              | 0.9                                 |
| AirAccelerationX       | 1                                   |
| AirDecelerationX       | 1                                   |
| FallSpeedMaxX          | 1                                   |
| FallSpeedMaxY          | 1                                   |
| LayerSettingType       | All ▼                               |
| IsBlockVerticalLine    | <input type="checkbox"/>            |
| IgnoreMoveBoundary     | <input type="checkbox"/>            |
| ApplyClimbableRotation | <input checked="" type="checkbox"/> |
| IsolatedMove           | <input type="checkbox"/>            |
| KinematicMove          | <input type="checkbox"/>            |
| Enable                 | <input checked="" type="checkbox"/> |

- Controls the character's physics with MapleStory Worlds's TileMap method.
- If the map is a TileMap but doesn't have a RigidBodyComponent, the character and tiles will not interact.
- Provides various physics property values that control the jumps, gravity, and more for characters within RigidBodyComponent.
- The target values can be changed to change the physics methods.

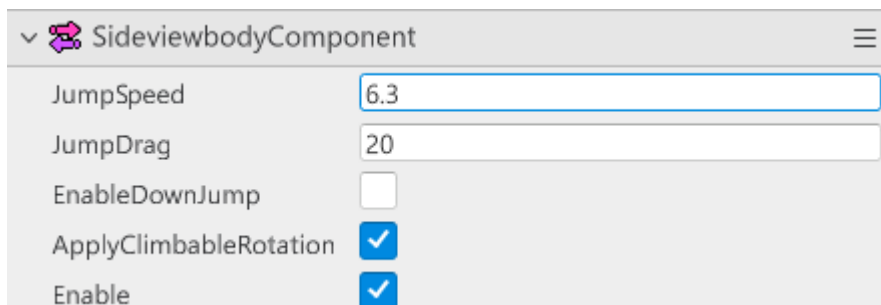
## KinematicBodyComponent





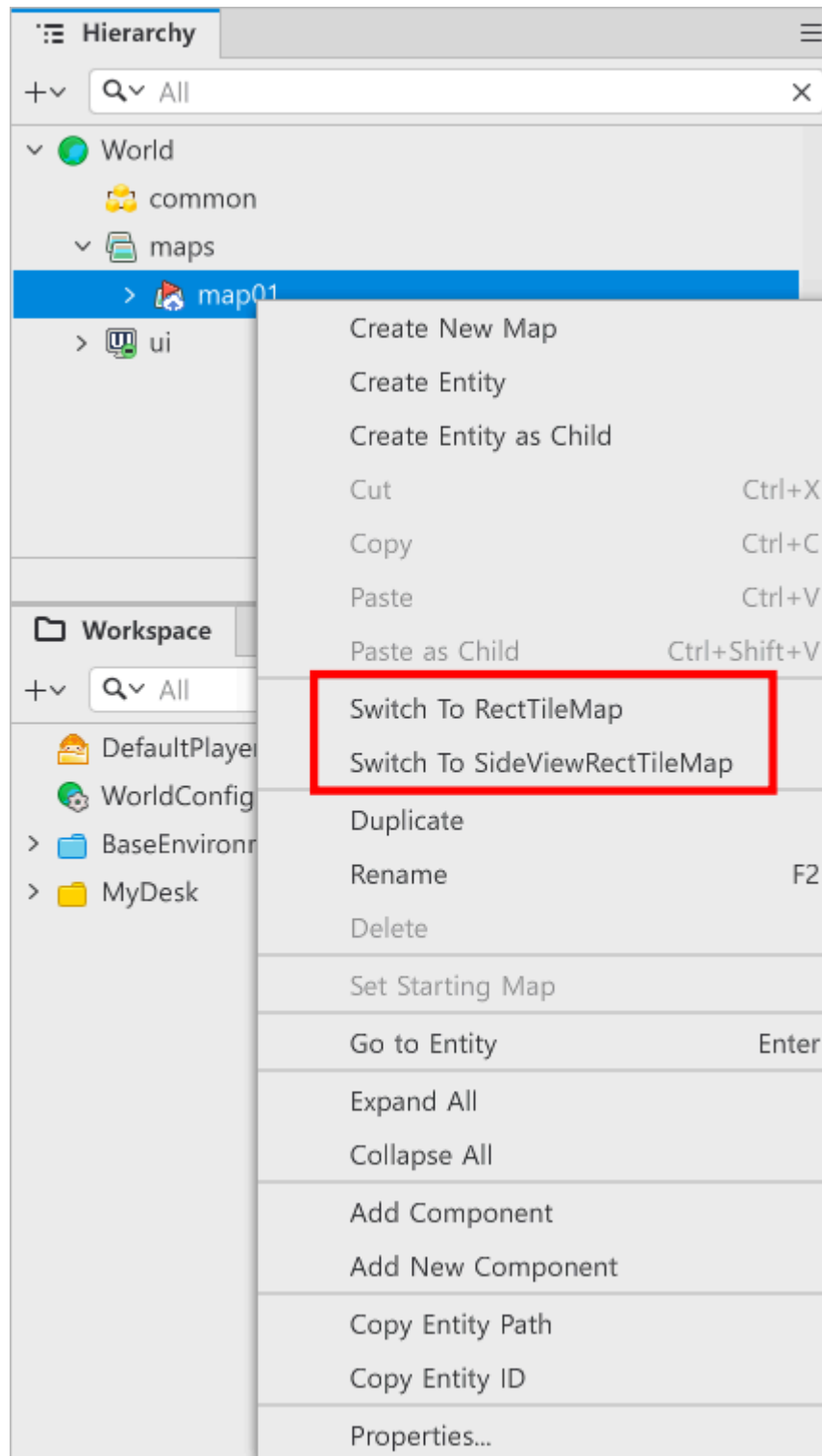
- Controls the character's physics for the MapleStory Worlds's RectTileMap type.
- If the map is RectTileMap but there is no KinematicBodyComponent, no interactions will happen with the RectTileMap tiles.
- You can adjust the speed, jump availability, shadow status, etc. for characters within KinematicBodyComponent.

## SideViewRectTileMap



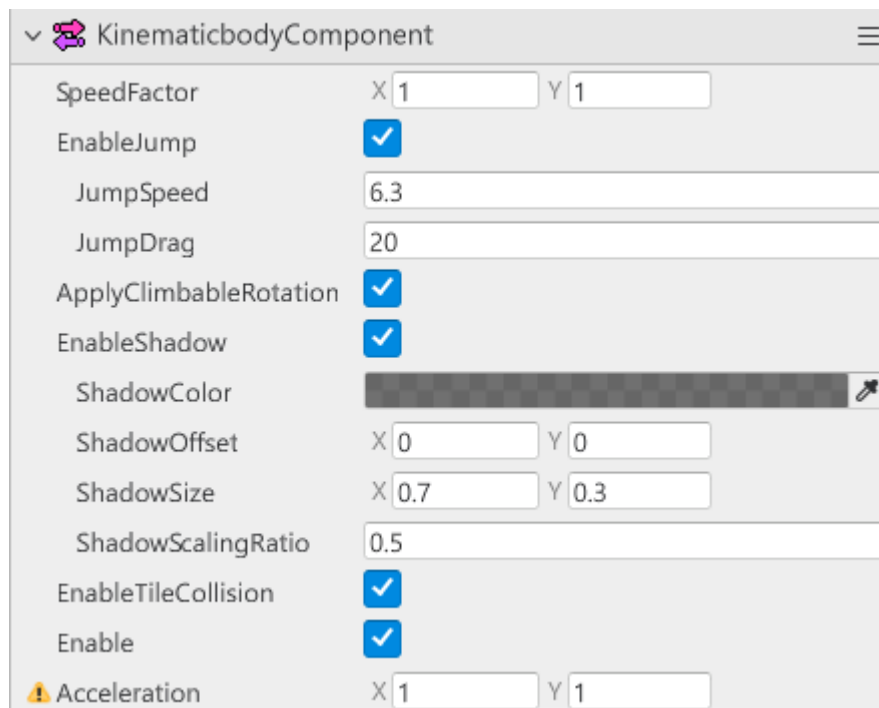
- Controls the character's physics if MapleStory Worlds is using the SideViewRectTileMap method.
- If this component doesn't exist, the character won't be able to interact with the map's tiles.
- Controls the character's jump speed and gravitational acceleration.

## Changing Map Formats



- Right-click the map you want to change from the Hierarchy window.
- Click Switch To RectTileMap or Switch To SideViewRectTileMap.
  - If the current map isn't a TileMap, Switch To TileMap will be visible and the conversion to the current map state will be hidden.
- Click on Switch To RectTileMap to convert the map to RectTileMap to create a shooting world.

## Exploring KinematicBodyComponent Elements



- **SpeedFactor:** The value that controls the character's speed. Control the horizontal or vertical speed based on the X and Y values.
- **EnableJump:** Set whether characters can jump. If this value is un-checked, the character won't be able to jump.
- **JumpSpeed:** Control the character's jump speed. The larger the value, the higher and faster the character can jump.
- **JumpDrag:** Control the character's gravitational acceleration. The smaller the value, the weaker gravity becomes, which makes the character return to the ground more slowly.
- **ApplyClimbableRotation:** A value that determines whether the character is spun when the character interacts with elements like ladders and ropes based on whether the ladder or rope is rotated.
- **EnableShadow:** Sets whether the character shadow is visible. When the value is removed, the character's shadow is removed.
- **ShadowColor:** Change the color of the shadow.
- **ShadowOffset:** Change the output position of the shadow.
- **ShadowSize:** Set the size of the shadow.
- **ShadowScalingRatio:** Change how much the shadow shrinks when the character jumps. The smaller the value, the smaller the shadow becomes.
- **EnableTileCollision:** The character will be able to **ignore** RectTileMap tiles that prevent movement.

## Placing Tiles on RectTileMap

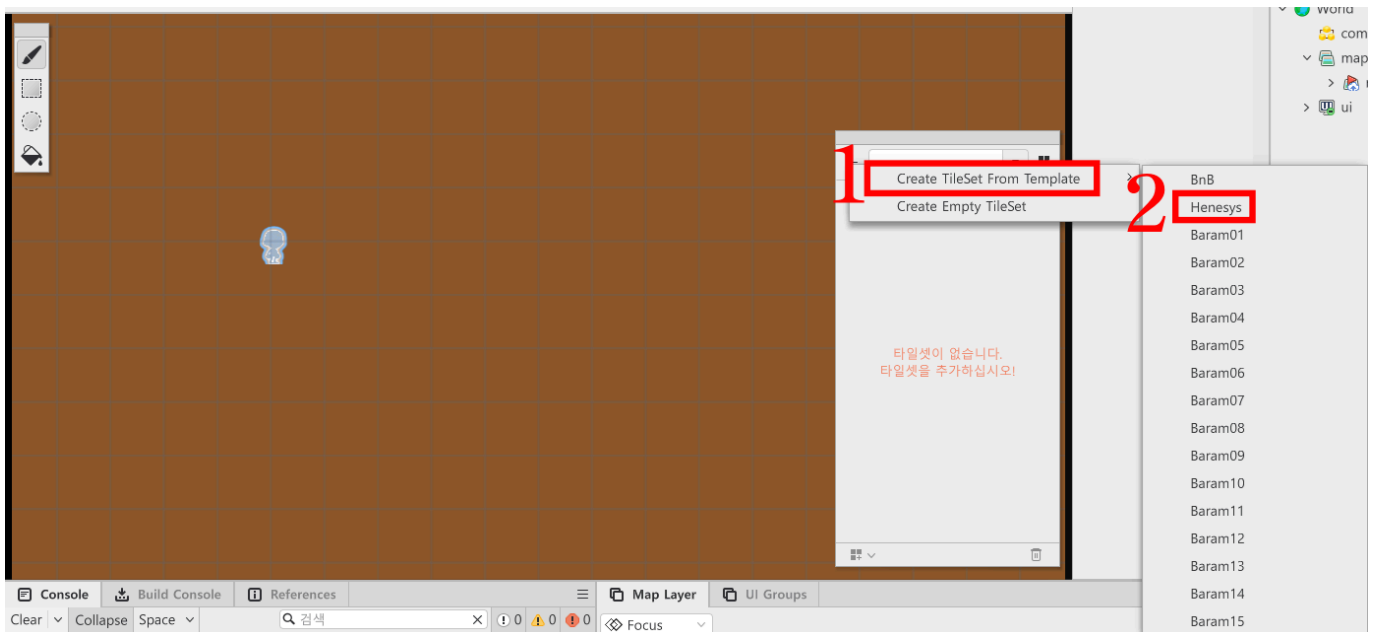
- A RectTileMap must be placed to set the movement area for a shooting world.
- If it is being changed to a RectTileMap for the first time, a tile set is required to place tiles.
- For the sample world, we will create a tile set by using resources provided by MapleStory Worlds.
- If you already have resources you've created or want to use, you can create a tile set yourself for use.

## Creating Tile Sets

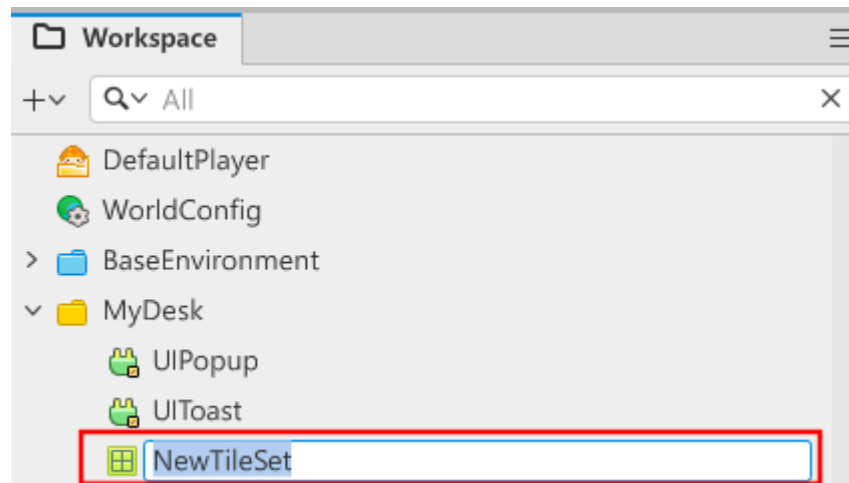




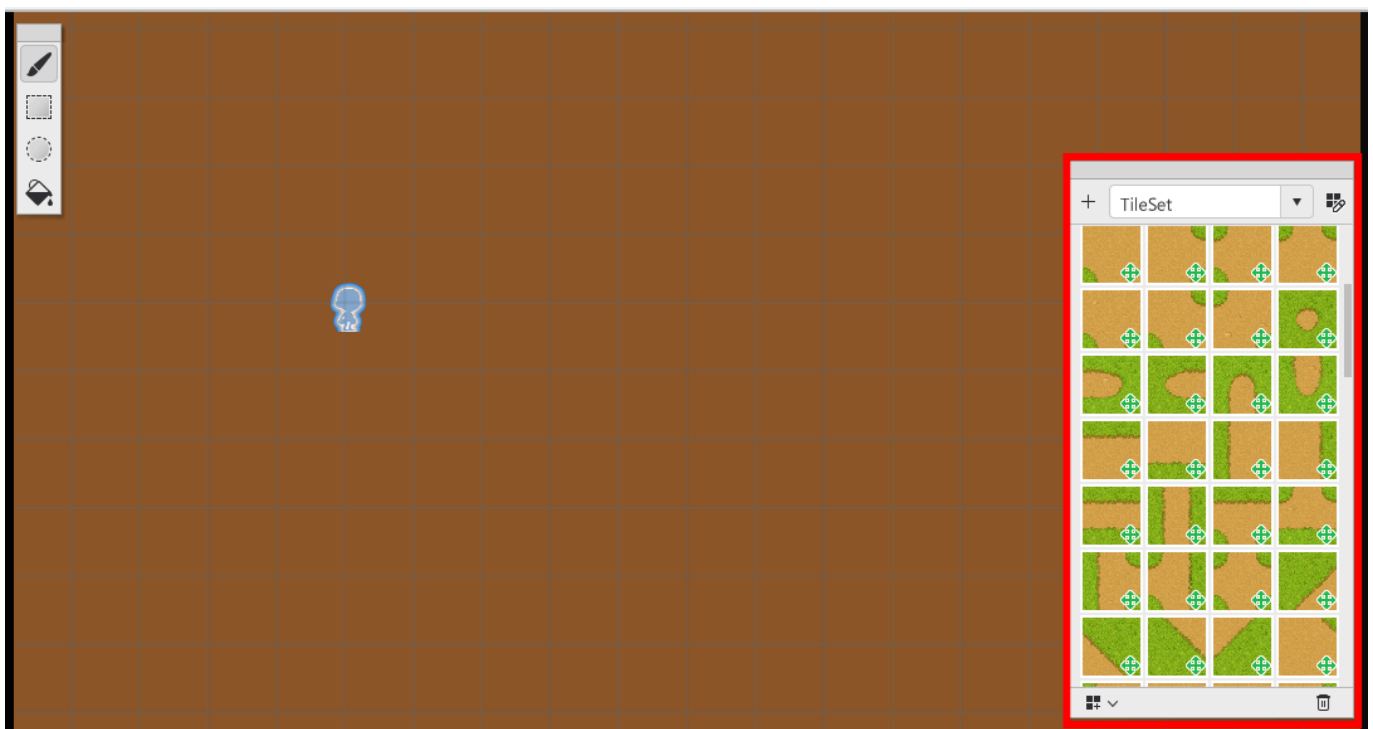
- Click the + button on the tile set editor screen in RectTileMap mode.



- Click Create TileSetFrom Template.
- Select the desired resource.
  - The sample world used Henesys.
- If there is a separate resource you want to use as a tile set, you can create a tile set yourself through Create Empty TileSet.

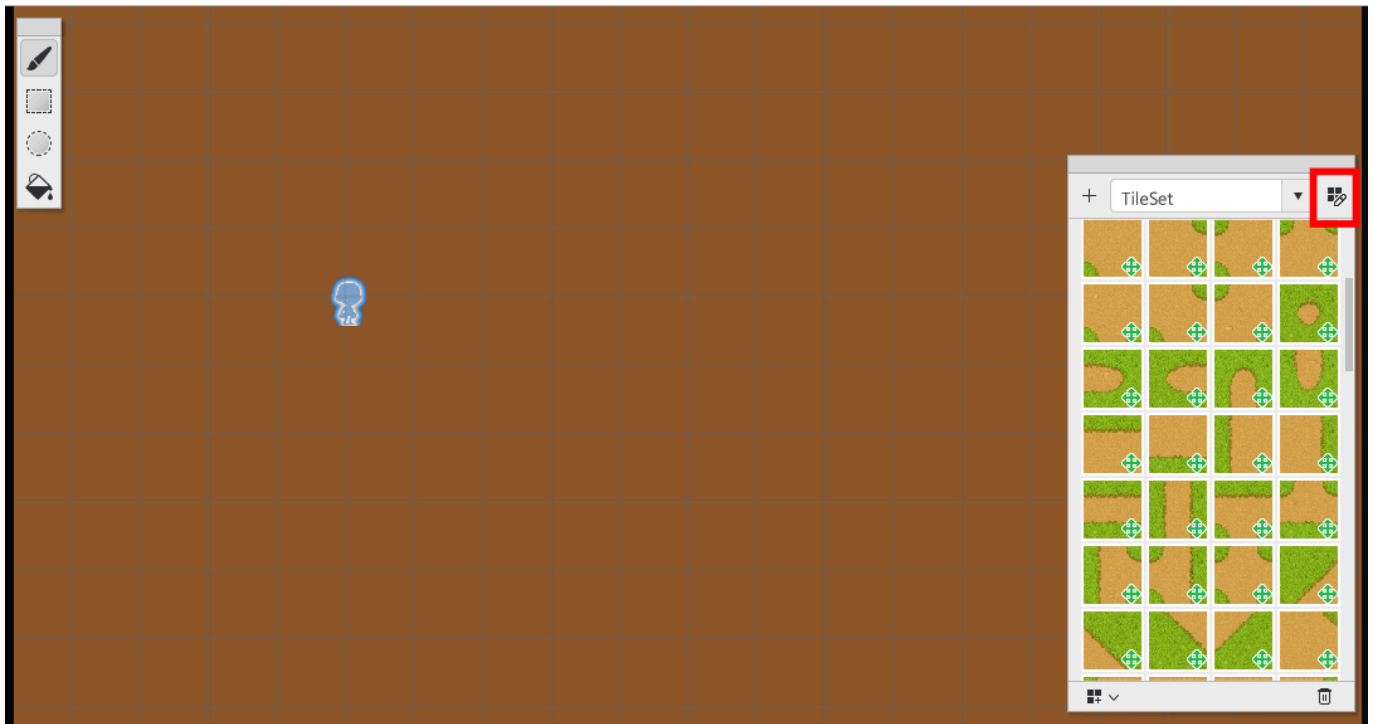


- Change the name of the NewTileSet that was added to Workspace to finish tile set creation.

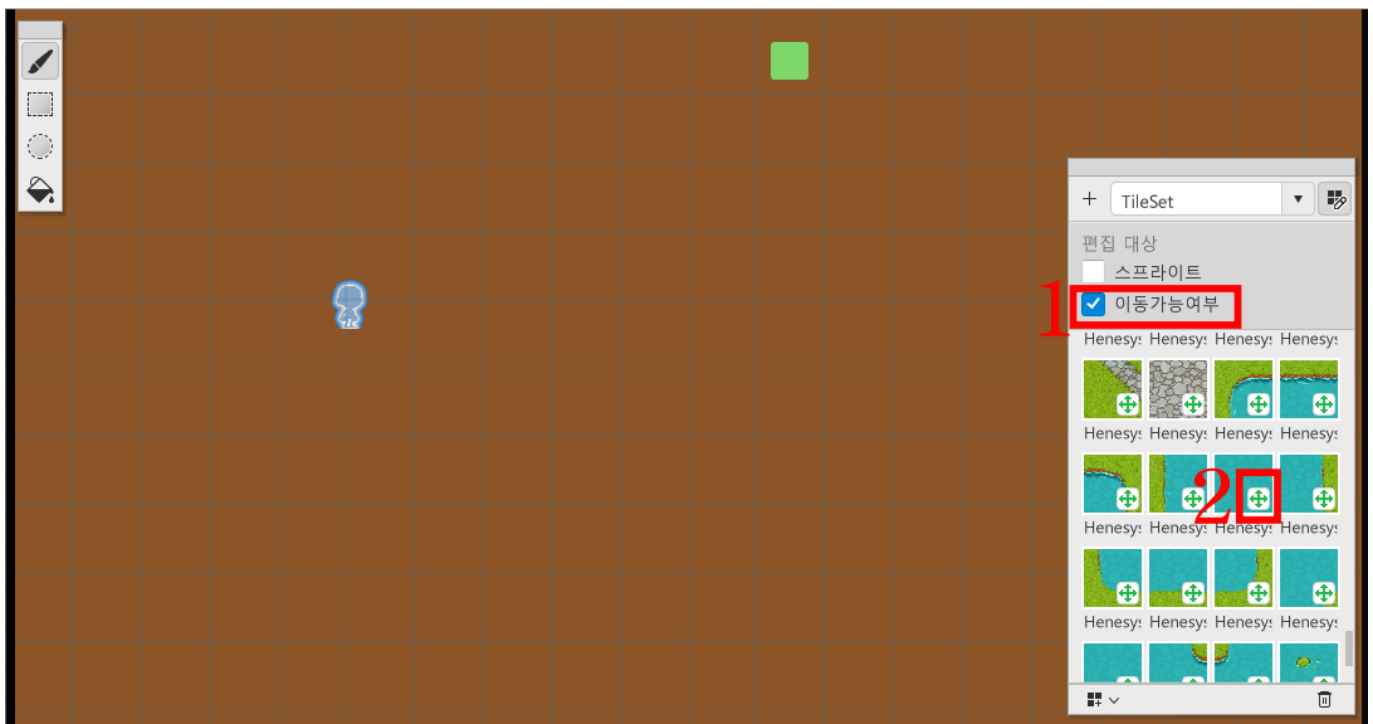


- If tile set creation was finished normally, placeable tiles will be visible in the tile set editor screen.

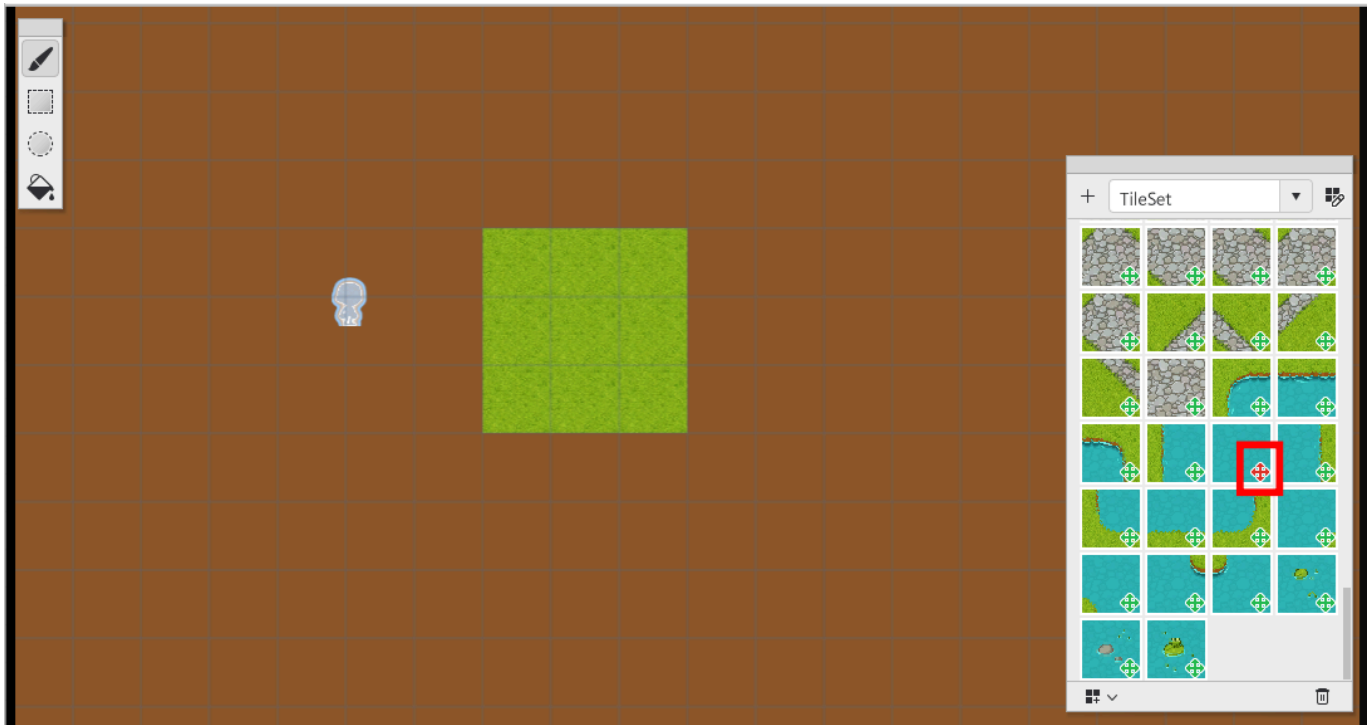
## Changing the Movability of Tiles



- Click the edit tile button on the right side of the editor screen.

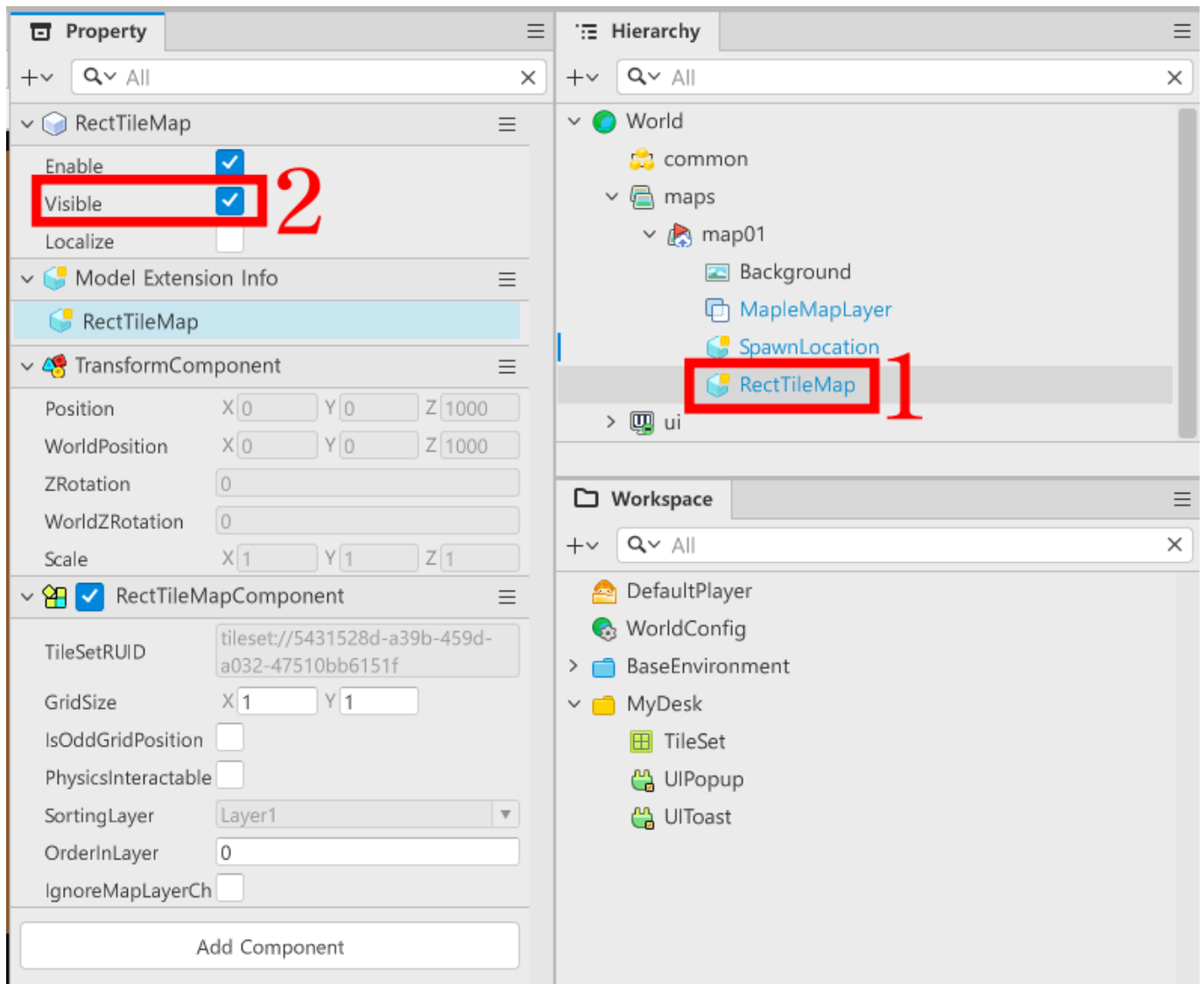


- Click on movability for the target you want to edit.
- Click on the green cross UI to change it to the red cross UI for the tile you want to make not movable.

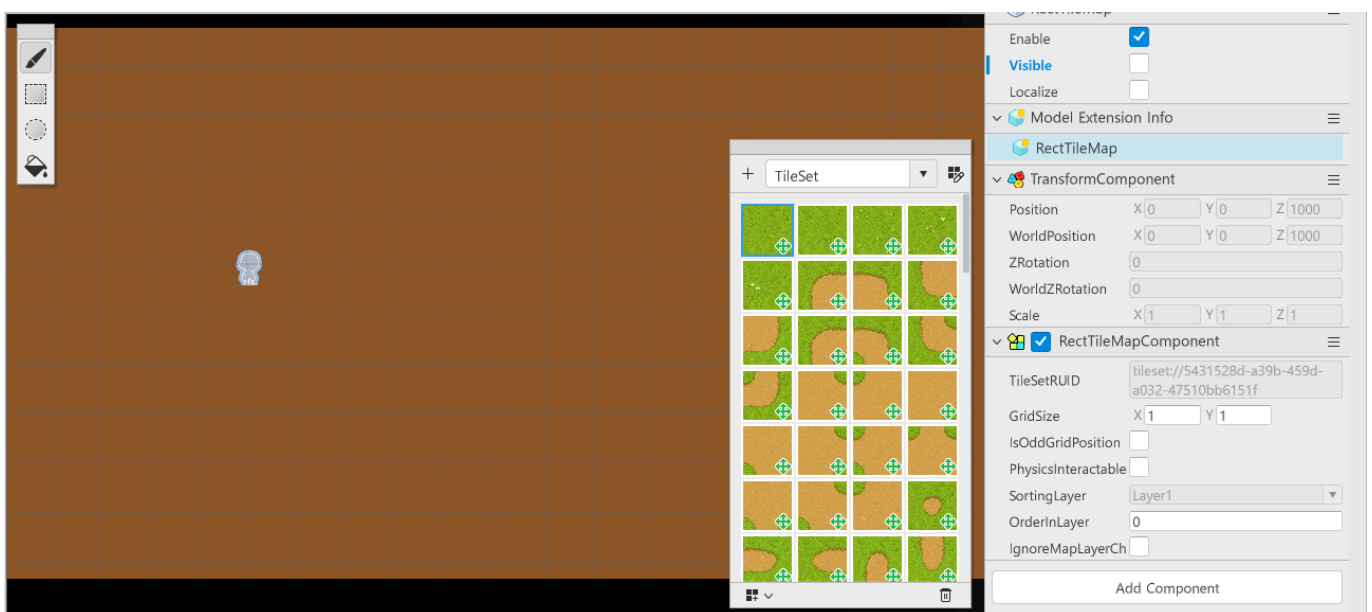


- If the change went through, the UI will change into a red cross while in tile placement mode.
- Place the tile on the map and then confirm that the character can't move normally through test play.

Hide Placed Tiles on the Map

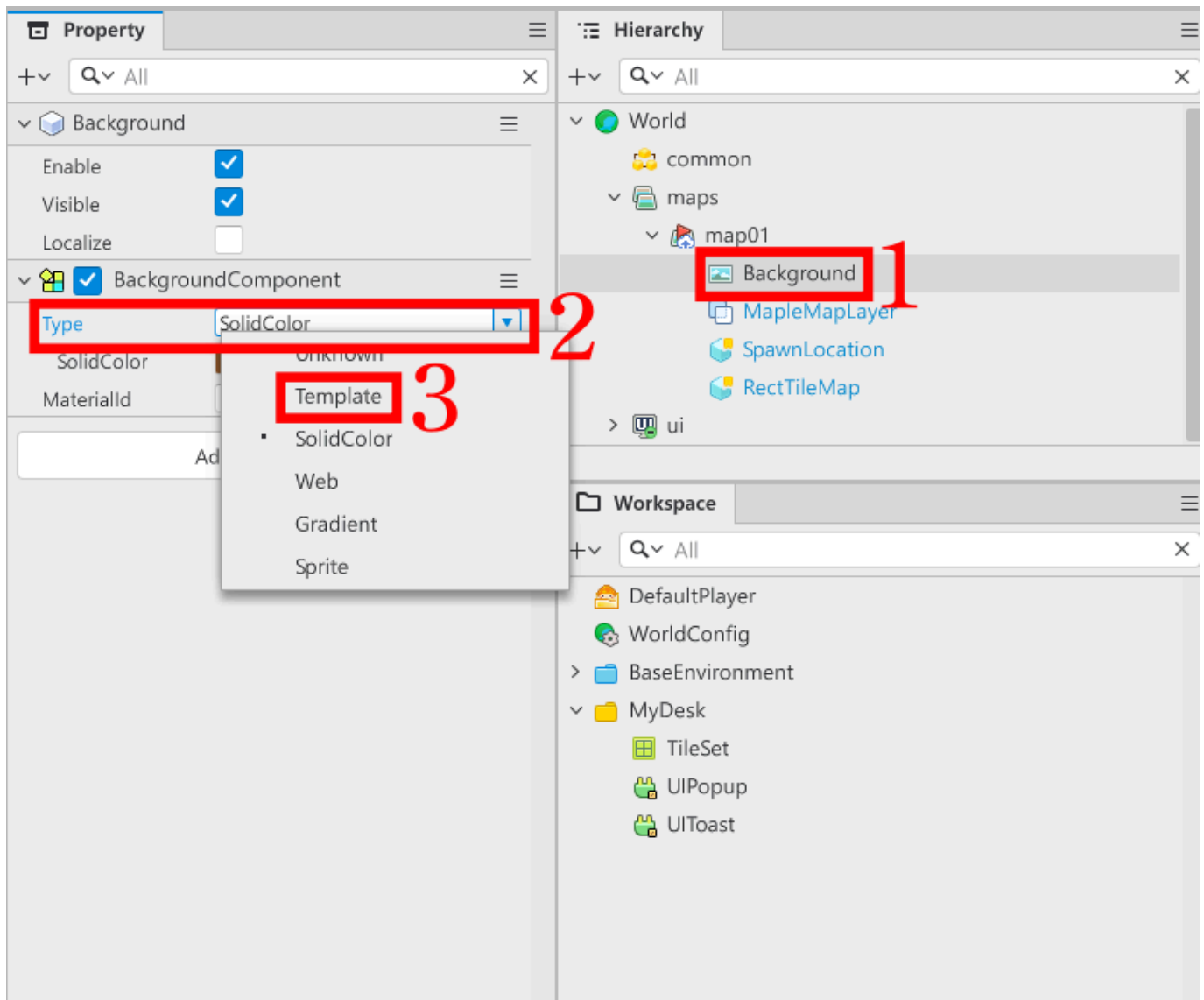


- Find RectTileMap in the Hierarchy window and click it.
- Click Visible in the Property window and **un-check** it.

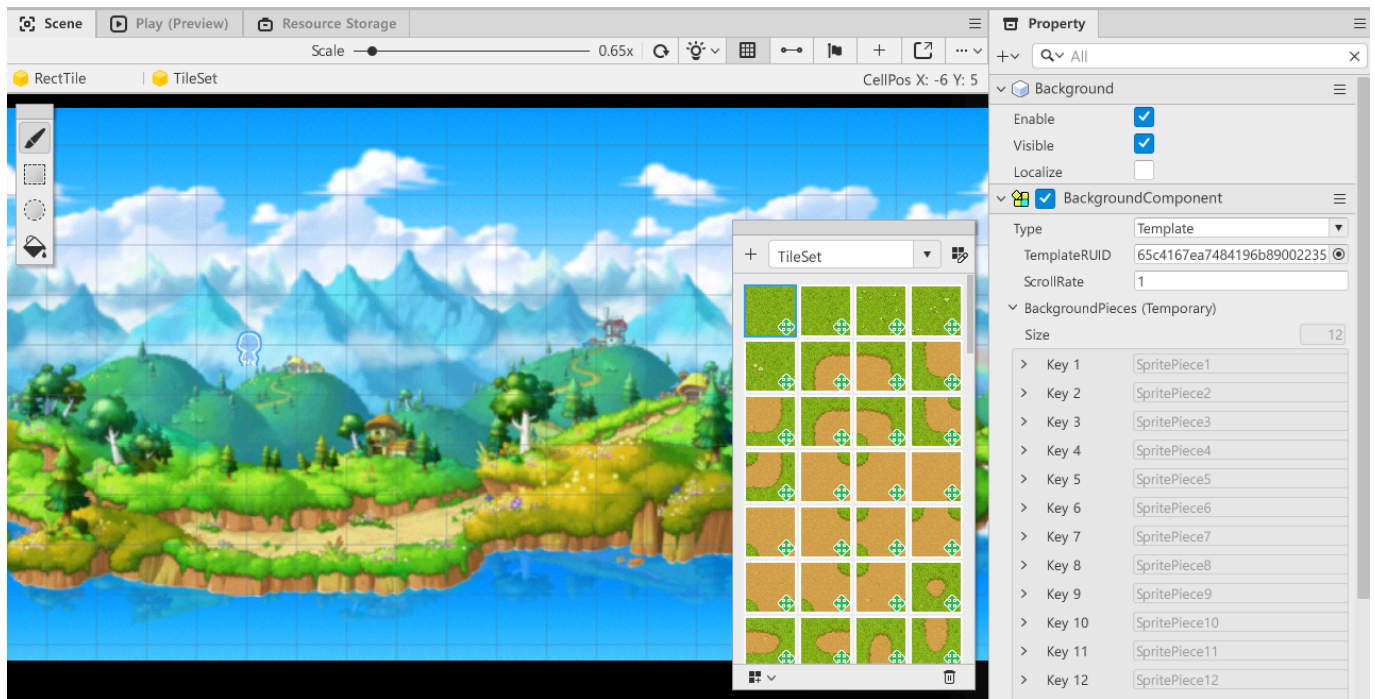


- When Visible is removed normally, then the Scene screen will show the screen where the tiles have been hidden.

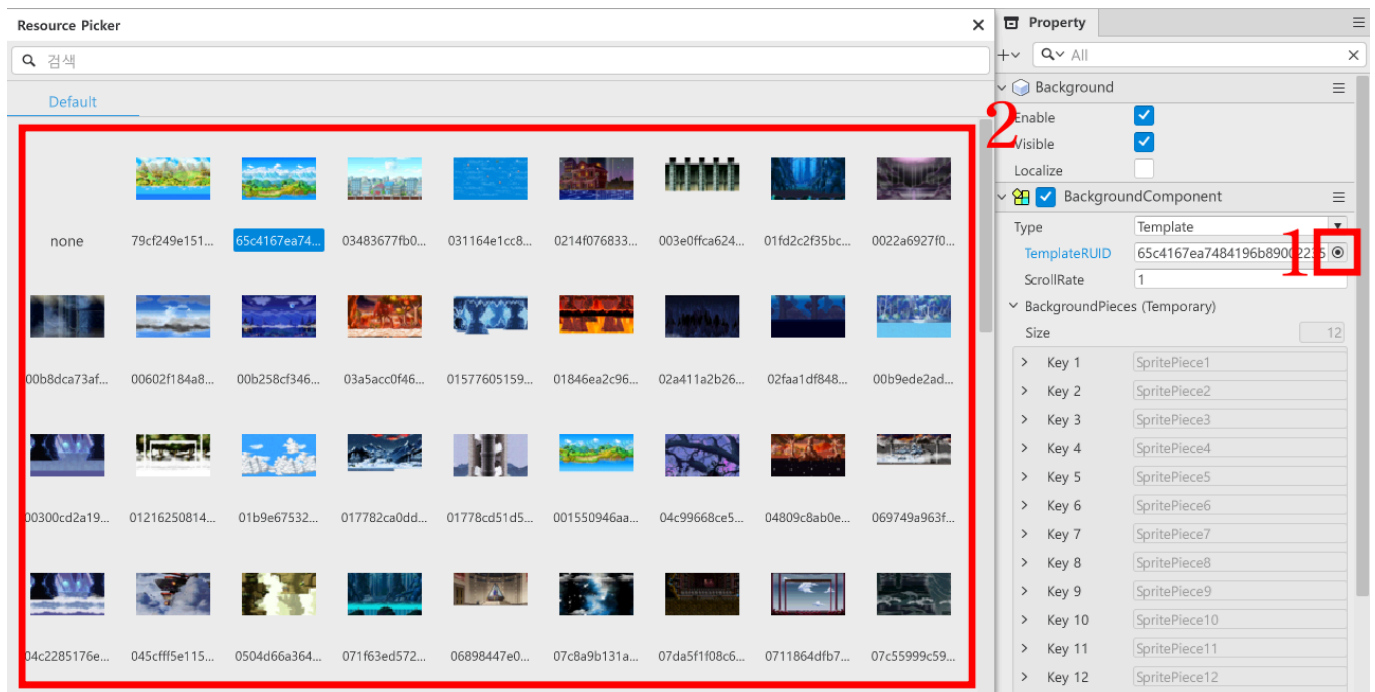
## Changing Backgrounds



- Find Background in the Hierarchy window and click it.
- Find the BackgroundComponent in the Property window.
- Click Type.
- Change from SolidColor to Template.

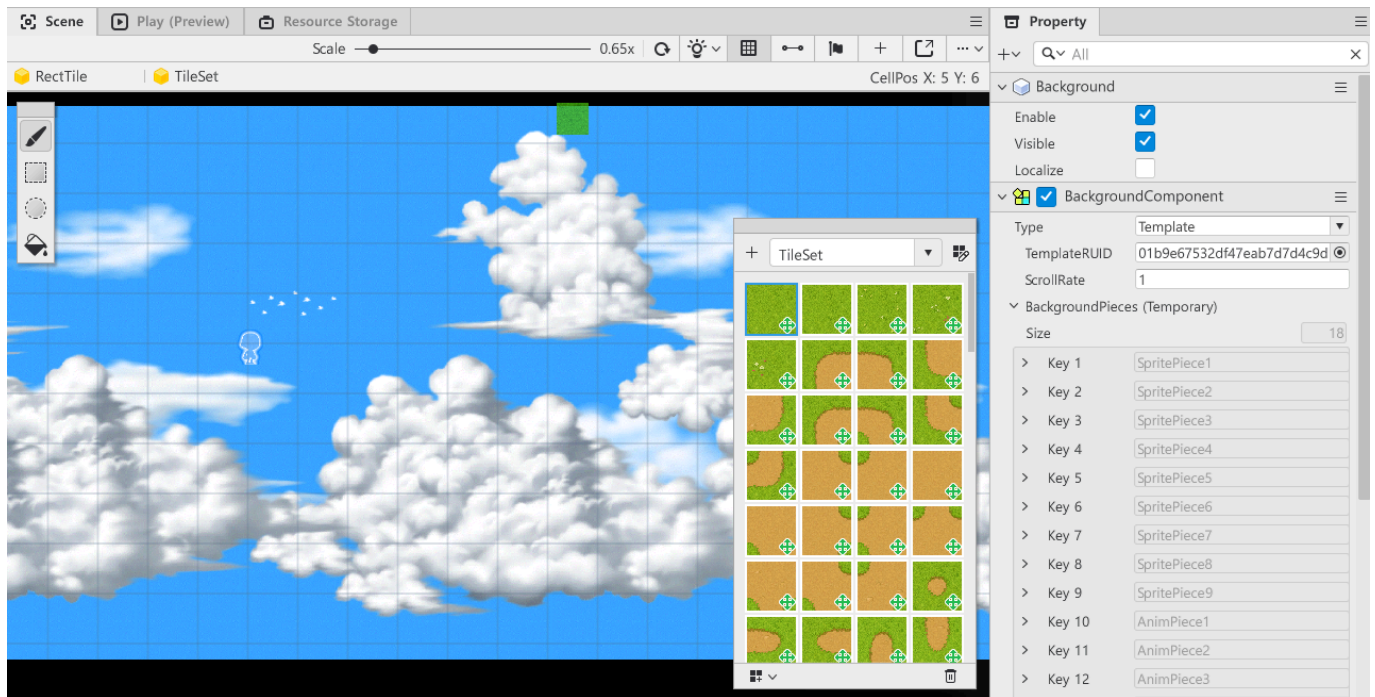


- If the change went through, the Scene screen will change from a single color background to one provided by MapleStory Worlds.



- If you want to use a different background, find BackgroundComponent in the Property window.
- Click on the Resource Picker button under TemplateRUID.
- Look for the background of choice and click on it to change the background.





- If the change went through, the Scene screen will show the newly-applied background.

## Minimum Implementation Standards

- Create a map for a shooting world for proper use.
  - You need to categorize areas with mobility and areas without mobility by using TileSet.

## Usage and Extension Direction

- When using resources provided by MapleStory Worlds, you will create your own resources, followed by a TileSet using those resources.
- Use the created TileSet to use a created map that can be used instead of a MapleStory Worlds resource.
- Learn about CameraComponent to find a way to fix the POV, so it doesn't follow the character.
  - MapleStory Worlds Official Document, Player Camera Control Document [Link](#)

---

## [💡 Basic Training] Declaring Logic

---

### Video Timeline

'- You can follow along by using the timeline under the training video. '- Understanding Logic: 00:23:57

### Learning Goals

- Learn about the Logic function that is provided by MapleStory Worlds.
- Understand the concept of a Manager and how to design Logic.
- Utilize Logic to create Logic that controls the game's system, Logic that manages data, etc.

- Understand the pros and cons of Logic, allowing for more thought out designs when creating additional Logic in the future.

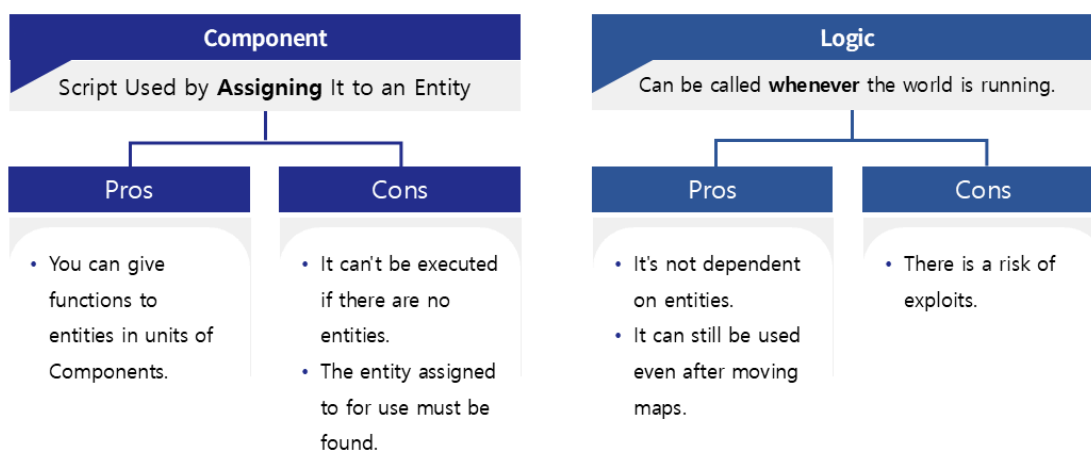
## Practical Exercises to Do After Watching the Video

- Add a Component to an empty entity and call the created Logic to review the traits of that Logic personally.
- Create a Logic with a Manager trait to think and design how that Manager should be managed.

## What is Logic?

### Key Point

Script Differences Based on **Whether the Entity is Dependent or Not**



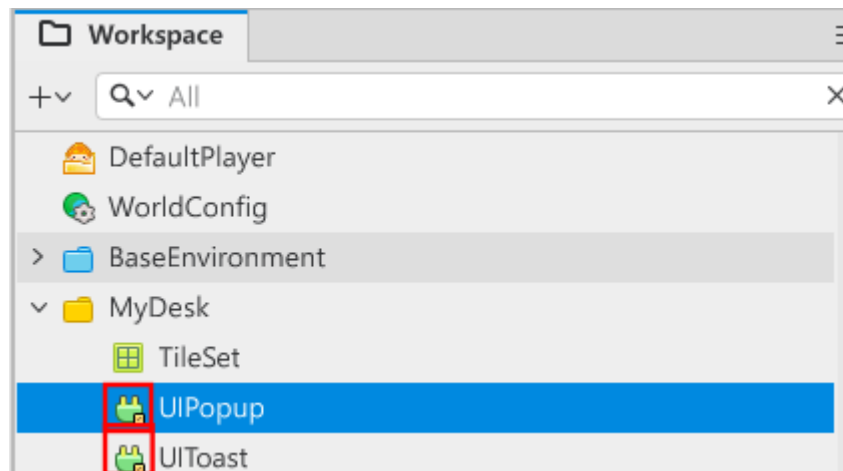
- This is one of the script types provided by MapleStory Worlds.
- When creating a Logic, it will be marked with a green plug icon within the Workspace.
- Also, unlike Components, it can access scripts even when not added to an entity.

## Manager Concepts

- Logic can be created as a manager that executes specific functions or roles.
- The manager of a single function or role is called a Manager.
- When there are issues with a specific function, it can be fixed quickly and conveniently by only changing the content of the Manager.
  - Case 1: When a character completes damage calculation and delivers it to a monster.
    - Each time the damage formula changes or each time a new element is added to damage calculation, the formula needs to be changed.
    - You will need to find all characters to edit and change the code entered.
  - Case 2: When making a request for damage calculation from a Logic-created Manager.
    - Change the damage formula inside the Logic.

- The Logic handles the damage calculation of all characters and monsters, so there are no more elements that require editing.
- Even if there are issues with the damage formula in future, editing the Manager can solve them.

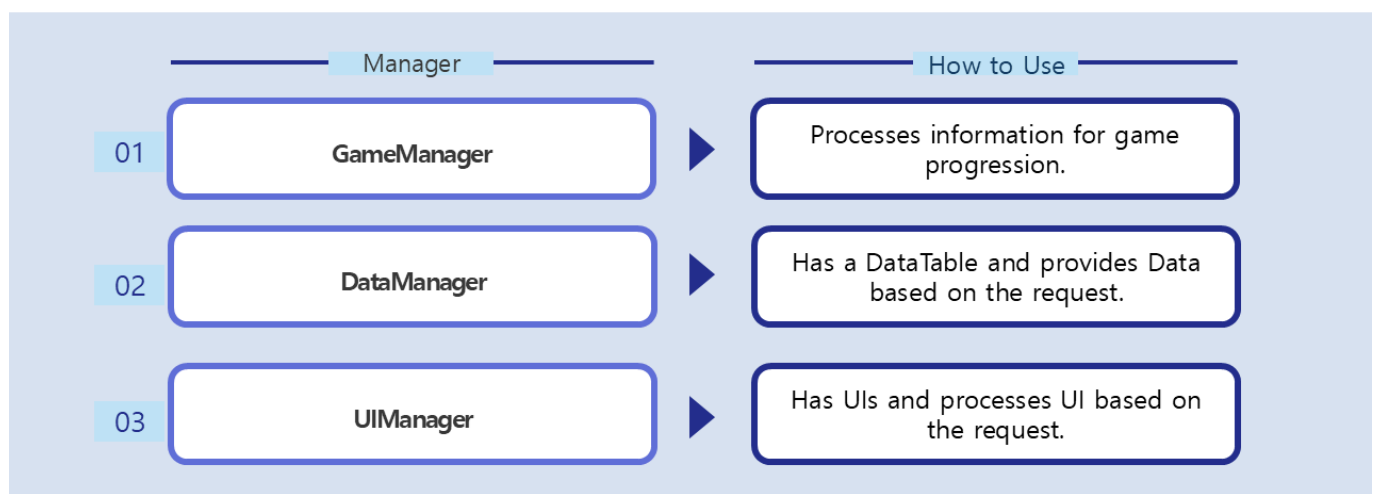
## How to Create Logic



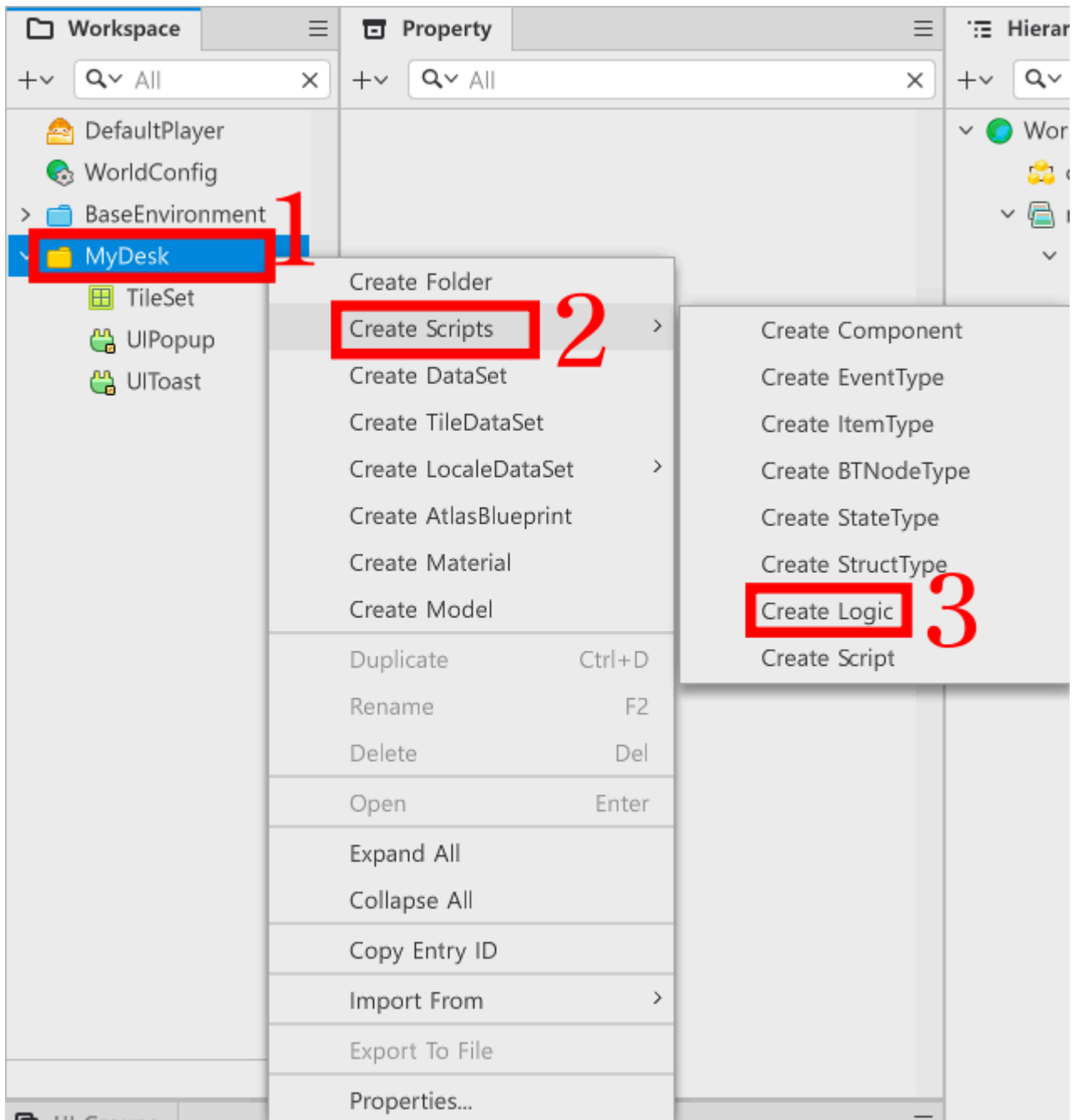
- Right-click MyDesk from Workspace.
- Click Create Scripts from the menu screen.
- Click Create Logic to create a new Logic.

### Key Point

Designed to **control specific functions** through the Manager.



- Change the created Logic's name to one of your choice.



- If the name is changed, you will see the Logic created within the Workspace.

## Manager Logic Usage Within the Sample World

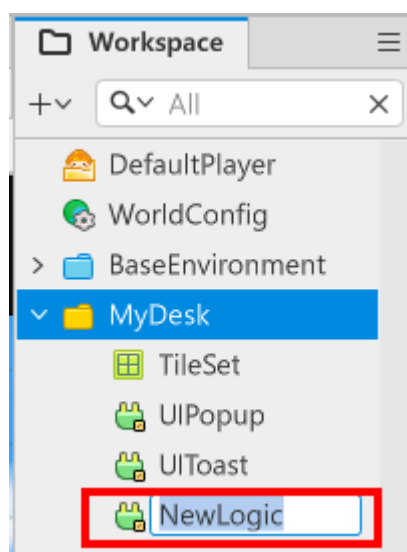
| Name           | Usage                                                                                                                                                                                                                                           |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GameLogic      | <p>Manager Logic created to manage functions that need to be handled as the game progresses.</p> <p>It is responsible for processing essential gameplay actions such as checking game clear status, calling UI, and teleporting characters.</p> |
| DataTableLogic | <p>Logic used to manage and utilize DataSets.</p> <p>Loading each required DataSet individually whenever data is needed is inefficient.</p>                                                                                                     |

| Name          | Usage                                                                                                                                                                                                                                                                        |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               | When the game starts, DataTableLogic caches all DataSets and returns the cached data whenever needed.                                                                                                                                                                        |
| UIManageLogic | <p>Manager Logic for managing UIGroups.</p> <p>When a component or GameLogic requires UI-related processing, it sends a request to this Manager Logic.</p> <p>Since UIManageLogic contains the components of all UIGroups, it handles all UI-related requests centrally.</p> |

- The sample world creates and uses Logic like in the above formats.
- When you need to create an additional function or extend one, you can declare your own Logic and create one for your use.

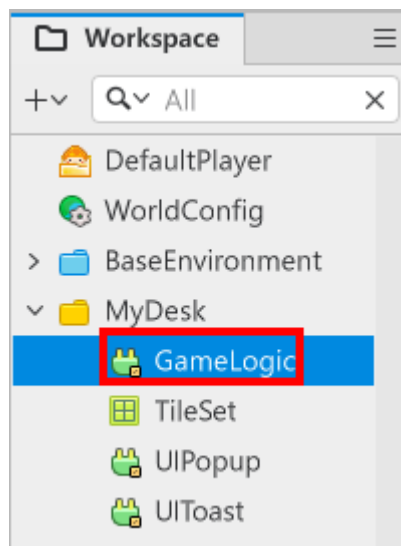
## Logic Usage Example Within the Sample World

### GameLogic Example



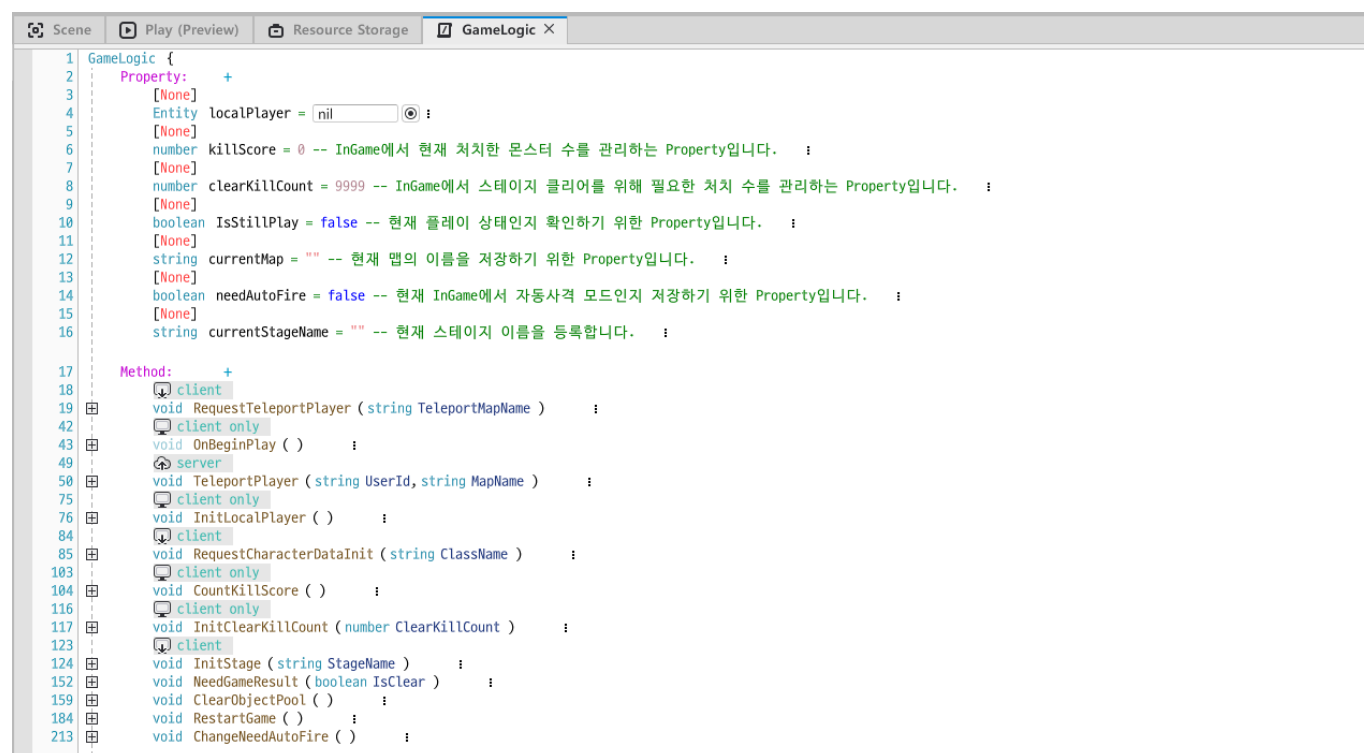
- It is processing a function that changes the map position of the user entity.
- When the game begins, it calculates how many more enemies need to be defeated in the stage and how many have been defeated already.
- Determines which UI effect will be called based on whether the game was cleared or the game is over.
- It can declare whether the player character requires an auto attack and change as needed.

### DataTableLogic Example



- Caches all DataSets by itself when the game begins.
- Returns the DataSet in the processed format after caching as requested.

## UIManageLogic Example



- Registers all Components that manage themselves in each UIGroup when the game begins.
- When there is any processing related to the UI while playing the game (showing UI, hiding UI, UI update, etc.), it will receive a request and orders the relevant UI's work.

## Writing GameLogic Code

```

@Logic
script GameLogic extends Logic

```

```
property Entity localPlayer = nil

property number killScore = 0 -- Property that manages the current number of
defeated monsters from InGame.

property number clearKillCount = 9999 -- Property that manages the number of
monsters that need to be defeated to clear the stage from InGame.

property boolean IsStillPlay = false -- Property that checks the current play
state.

property string currentMap = "" -- Property that saves the current map's name.

property boolean needAutoFire = false -- Property that saves if it is currently in
auto-fire mode from InGame.

property string currentStageName = "" -- Registers the name of the current stage.

@ExecSpace("Client")
method void RequestTeleportPlayer(string TeleportMapName)
log("GameManager RequestTeleportPlayer Start")

-- Searches for LocalPlayer if there is no Player currently.
if not invalid(self.localPlayer) then
    self:InitLocalPlayer()
    -- Halts code execution if there is no LocalPlayer.
    if not invalid(self.localPlayer) then return end
end

-- Makes it so the UI can change based on the current map.
if TeleportMapName == "Title" then
    _UIManageLogic:ShowTitle()
elseif TeleportMapName == "InGame" then
    _UIManageLogic:ShowInGame()
end

self.currentMap = TeleportMapName
-- The Name of the player entity in MapleStory Worlds is the same as the UserId.
self:TeleportPlayer(self.localPlayer.Name, TeleportMapName)
end

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- Searches for and registers local players when the game begins.
self:InitLocalPlayer()
end

@ExecSpace("Server")
method void TeleportPlayer(string UserId, string MapName)
-- 1. Runs another search for an entity in the server via UserId.
log("TeleportPlayerEntry Success")
local playerEntity = _UserService:GetUserEntityByUserId(UserId)
-- Generates an error log if no playerEntity can be found.
if not invalid(playerEntity) then
```

```

        log("Teleport Failed: User not found.")
        return
    end

    -- Sets the movement path.
    local targetPath = ""
    if MapName == "Title" then
        targetPath = "/maps/Title"
    elseif MapName == "InGame" then
        targetPath = "/maps/InGame"
    end

    -- Teleports the player character through the path below if the movement path
    is registered correctly.
    if targetPath ~= "" then
        _TeleportService:TeleportToEntityPath(playerEntity, targetPath)
        log("Teleport Execution Complete: " .. targetPath)
    end
end

@ExecSpace("ClientOnly")
method void InitLocalPlayer()
    -- Searches for and registers local players.
    if not isValid(self.localPlayer) then
        self.localPlayer = _UserService.LocalPlayer
    end
end

@ExecSpace("ClientOnly")
method void RequestCharacterDataInit(string ClassName)
    -- Resets PlayerState based on the ID value from the received
    CharacterDefaultData.
    if ClassName ~= "" and ClassName ~= nil then
        -- Searches for the data table that corresponds to ClassName from
        DataTableManager.
        local findPlayerData =
        _DataTableLogic:GetCharacterDefaultDataByCharacterID(ClassName)
        -- Resets PlayerState with any data that is found.
        if isValid(findPlayerData) and next(findPlayerData) ~= nil then
            log("CharacterDefaultData Init request complete.")
            self.localPlayer.PlayerState:InitPlayerData(findPlayerData)
        -- If none is found, it leaves a log.
        else
            log("Abnormal CharacterDefaultData Init failed!")
        end
    end
end

@ExecSpace("ClientOnly")
method void CountKillScore()
    -- Increases killScore by 1 when a monster dies.
    self.killScore = self.killScore + 1
    -- Processes as a stage clear if the killScore is higher than clearKillCount.
    if self.killScore >= self.clearKillCount then
        self.IsStillPlay = false
    end
end

```



```

        self:NeedGameResult(true)
        log("Game Clear")
    end
end

@ExecSpace("ClientOnly")
method void InitClearKillCount(number ClearKillCount)
    -- Resets the stage clear condition to the calculated number of monsters.
    self.clearKillCount = ClearKillCount
end

@ExecSpace("ClientOnly")
method void InitStage(string StageName)
    -- Sets the path of the MonsterSpawnManager for the current map based on the
    StageName value.
    local spawnManagerPath = "/maps/"..StageName.."/MonsterSpawnManager"
    -- Searches for the MonsterSpawnManager entity via the pre-set path.
    local stageSpawnManager = _EntityService:GetEntityByPath(spawnManagerPath)
    -- Leaves a log and returns if it fails to find a MonsterSpawnManager.
    if not isValid(stageSpawnManager) then
        log("Failed to find a MonsterSpawnManager object in "StageName.."!")
        return
    end
    -- Calls a method so ItemHelper can register the ItemSpawnManager of the current
    map.
    _ItemHelper:InitData()
    -- Launches the game and changes the IsStillPlay value to true if the game isn't
    currently running.
    if not self.IsStillPlay then
        stageSpawnManager.MonsterSpawnManager:StartGame()
        self.IsStillPlay = true
    end
    -- Resets the current monster kill count to 0.
    self.killScore = 0
    -- All processes are complete, so the received stage name is registered.
    self.currentStageName = StageName
    -- Temporarily halts all BGM.
    _SoundService:StopBGM(true)
    -- Generates the BGM that needs to be played in InGame.
    _SoundService:PlayBGM("d155c5573a3f46a7b2ffe38629441132", 1.0)
end

@ExecSpace("ClientOnly")
method void NeedGameResult(boolean IsClear)
    -- Plays the UI effect based on the delivered game clear result.
    self.IsStillPlay = false
    _UIManageLogic:ShowResultUI(IsClear)
end

@ExecSpace("ClientOnly")
method void ClearObjectPool()
    -- Ignores the code below if the target method is called in another map other than
    InGame.
    if not self.currentMap == "InGame" then

```

```

        return
    end
    -- Sets the path for Managers that manage each ObjectPool in the InGame map.
    local missilePoolPath = "/maps/"..self.currentMap.."/ProjectileManager"
    local monsterPoolPath = "/maps/"..self.currentMap.."/MonsterSpawnManager"

    -- Searches for the Managers based on the path set above.
    local missilePoolEntity = _EntityService:GetEntityByPath(missilePoolPath)
    local monsterPoolEntity = _EntityService:GetEntityByPath(monsterPoolPath)
    -- Deactivates entities in each object pool if the search is successful.
    if invalid(missilePoolEntity) then
        missilePoolEntity.ProjectileManager:ResetPool()
    end
    if invalid(monsterPoolEntity) then
        monsterPoolEntity.MonsterSpawnManager:ClearAllChild()
    end

    -- Prevents errors from being triggered when the helper class is called from a
    place other than the InGame map.
    _ItemHelper:ReleaseData()
end

@ExecSpace("ClientOnly")
method void RestartGame()
    -- Searches for local players.
    local findLocalPlayer = _UserService.LocalPlayer
    -- Leaves a log and returns if no local players can be found.
    if not invalid(findLocalPlayer) then
        log("Couldn't find a LocalPlayer!")
        return
    end
    -- Generates a log and returns if there is no PlayerState component in the local
    player entity.
    if not invalid(findLocalPlayer.PlayerState) then
        log("Couldn't find a PlayerState component from LocalPlayer!")
        return
    end

    -- Searches for the current character's ID value after receiving the PlayerState.
    local currentPlayerCharacter = findLocalPlayer.PlayerState.characterID
    -- Resets the ObjectPool for restarts.
    self:ClearObjectPool()
    -- Resets the data for the stage that will be restarted.
    self:InitStage(self.currentStageName)
    -- Resets the data again to return the character information to its original
    state.
    self:RequestCharacterDataInit(currentPlayerCharacter)
    -- Converts the UI state to the in-game start state.
    _UIManageLogic:ShowInGame()
    -- Converts the IsStillPlay state to true, since the game has restarted.
    self.IsStillPlay = true
end

@ExecSpace("ClientOnly")

```

```

method void ChangeNeedAutoFire()
-- Changes the needAutoFire value to the opposite value if the target function is
called.
self.needAutoFire = not self.needAutoFire
end

end

```

## Writing DataTableLogic Code

```

@Logic
script DataTableLogic extends Logic

property table missileData = {} -- Property for managing the MissileData dataset.

property table characterDefaultData = {} -- Property for managing the
CharacterDefaultData dataset.

property table characterSelectUIData = {} -- Property for managing the
CharacterSelectUIData dataset.

property table monsterDefaultData = {} -- Property for managing the
MonsterDefaultData dataset.

property table monsterSpawnData = {} -- Property for managing the MonsterSpawnData
dataset.

property table skillData = {} -- Property for managing the SkillData dataset.

property table itemData = {} -- Property for managing the ItemData dataset.

property table itemDropData = {} -- Property for managing the ItemDropData
dataset.

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- Resets the MissileData dataset.
self:InitMissileData()
-- Resets the CharacterSelectUIData dataset.
self:InitCharacterSelectUIData()
-- Resets the MonsterSpawnData dataset.
self:InitMonsterSpawnData()
-- Resets the MonsterDefaultData dataset.
self:InitMonsterDefaultData()
-- Resets the CharacterDefaultData dataset.
self:InitCharacterDefaultData()
-- Resets the SkillData dataset.
self:InitSkillData()
-- Resets the ItemData dataset.

```

```

self:InitItemData()
-- Resets the ItemDropData dataset.
self:InitItemDropData()
end

@ExecSpace("ClientOnly")
method void InitMissileData()
-- Determines whether the missile data table is parsed.
if isvalid(self.missileData) == false or next(self.missileData) == nil then
    -- Searches the MissileData data table when missile data parsing is required.
    local missileDefaultData = _DataService:GetTable("MissileData")
    if isvalid(missileDefaultData) == nil then
        -- Generates and Returns the log, since the MetaData table couldn't be
        found.
        log("The MissileData data table couldn't be found!")
        return
    end
    -- Calculates the total number of Rows.
    local rowCount = missileDefaultData:GetRowCount()

    -- Plugs in the MissileID that will be the Key, starting at Row 1 in the data
    table.
    for i = 1, rowCount do
        local missileID = missileDefaultData:GetCell(i, "MissileID")

        -- Determines whether the ID is valid.
        if missileID ~= nil and missileID ~= "" then

            -- Bundles all of the information from the target row into a single
            table.
            local rowData =
            {
                MissileID = missileID,
                AtkValue = tonumber(missileDefaultData:GetCell(i, "AtkValue")),
                MoveSpeed = tonumber(missileDefaultData:GetCell(i, "MoveSpeed")),
                LifeTime = tonumber(missileDefaultData:GetCell(i, "LifeTime")),
                HitEffectRUID = tostring(missileDefaultData:GetCell(i,
"HitEffectRUID")),
                ModelRUID = tostring(missileDefaultData:GetCell(i, "ModelRUID"))
            }

            -- Registers to missileData Property. (Key: MissileID, Value: Data)
            self.missileData[missileID] = rowData
        end
    end
    -- Generates the Log that indicates that the data table caching is complete.
    log("MissileData data table caching complete!")
    return
end
-- Generates the Log that indicates that the data table caching was not completed.
log("Either the MissileData data table has already been cached or the MissileData
data table couldn't be found.")
end

```

```

@ExecSpace("ClientOnly")
method void InitCharacterDefaultData()
-- Item that determines whether player character parsing is complete.
if isValid(self.characterDefaultData) == false or next(self.characterDefaultData)
== nil then
    local getTableData = _DataService:GetTable("CharacterDefaultData")
    -- Displays warning text if it failed to load the data table.
    if getTableData == nil then
        log("Failed to fetch the CharacterDefaultData data table!")
        return
    end
    --Start Parsing
    local rowCount = getTableData:GetRowCount() -- Calculates the total number of
Rows.

    for i = 1, rowCount do
        -- Plugs in a single CharacterID that will be the Key, starting at the
data table Row.
        local characterID = tostring(getTableData:GetCell(i, "CharacterID"))

        -- Determines whether the ID is valid.
        if characterID ~= nil and characterID ~= "" then

            -- Bundles all of the information from a single row into a table.
            local rowData =
            {
                CharacterID = characterID,
                StartHP = tonumber(getTableData:GetCell(i, "StartHP")),
                StartSkillCount = tonumber(getTableData:GetCell(i,
"StartSkillCount")),
                StartMoveSpeed = tonumber(getTableData:GetCell(i,
"StartMoveSpeed")),
                DefaultMissile = tostring(getTableData:GetCell(i,
"DefaultMissile")),
                DefaultSkill = tostring(getTableData:GetCell(i, "DefaultSkill")),
                AttackInterval = tonumber(getTableData:GetCell(i,
"AttackInterval"))
            }

            -- Save to Cache Table (Key: MissileID, Value: Data)
            self.characterDefaultData[characterID] = rowData
        end
    end
    -- Generates the Log that indicates that the data table caching is complete.
    log("CharacterDefaultData data table caching complete!")
    return
end
-- Generates the Log that indicates that the data table caching was not completed.
log("Either the CharacterDefaultData data table has already been cached or the
CharacterDefaultData data table couldn't be found.")
end

@ExecSpace("ClientOnly")
method void InitCharacterSelectUIData()

```

```

-- Checks whether the character selection window information table is cached.
if isValid(self.characterSelectUIData) == false or
next(self.characterSelectUIData) == nil then
    -- The character selection window's information table isn't registered, so the
    data table is searched to cache that information.
    local characterSelectData = _DataService:GetTable("CharacterSelectUIData")
    -- Displays the error as a log if it was unable to find the character
    selection window's information table.
    if characterSelectData == nil then
        log("The CharacterSelectUIData data table couldn't be found!")
        return
    end
    -- Calculates the RowCount of a data table if one is successfully found.
    local rowCount = characterSelectData:GetRowCount()
    -- Plugs in the CharacterID that will be the Key, starting at Row 1 in the
    data table.
    for i = 1, rowCount do
        local characterID = characterSelectData:GetCell(i, "CharacterID")

        -- Determines whether the ID is valid.
        if characterID ~= nil and characterID ~= "" then

            -- Bundles all of the information from the target row into a single
            table.
            local rowData =
            {
                CharacterID = characterID,
                LocalizeNameID = tostring(characterSelectData:GetCell(i,
"LocalizeNameID"))),
                SpriteRUID = tostring(characterSelectData:GetCell(i,
"SpriteRUID"))
            }

            -- Save to Cache Table (Key: MissileID, Value: Data)
            self.characterSelectUIData[characterID] = rowData
        end
    end
    -- Generates the Log that indicates that the data table caching is complete.
    log("CharacterSelectUIData data table caching complete!")
    return
end
-- Generates the Log that indicates that the data table caching was not completed.
log("Either the CharacterSelectUIData data table has already been cached or the
MissileData data table couldn't be found.")
end

@ExecSpace("ClientOnly")
method void InitMonsterDefaultData()
    -- Determines whether monster data parsing is complete.
    if isValid(self.monsterDefaultData) == false or next(self.monsterDefaultData) ==
nil then
        local getTableData = _DataService:GetTable("MonsterDefaultData")
        -- Displays warning text if it failed to load the data table.
        if getTableData == nil then

```

```

        log("Failed to load the MonsterDefaultData data table!")
        return
    end
    -- Calculates the total number of Rows.
    local rowCount = getTableData:GetRowCount()

    for i = 1, rowCount do
        -- Plugs in a single MissileID that will be the Key, starting with the
data table Row.
        local monsterID = getTableData:GetCell(i, "MonsterID")

        -- Determines whether the ID is valid.
        if monsterID ~= nil and monsterID ~= "" then

            -- Bundles all of the information from a single row into a table.
            local rowData =
            {
                monsterID = monsterID,
                StartHP = tonumber(getTableData:GetCell(i, "StartHP")),
                StartMoveSpeed = tonumber(getTableData:GetCell(i,
"StartMoveSpeed")),
                DefaultMissile = getTableData:GetCell(i, "DefaultMissile"),
                AttackInterval = tonumber(getTableData:GetCell(i,
"AttackInterval")),
                ModelRUID = tostring(getTableData:GetCell(i, "ModelRUID"))
            }

            -- Save to Cache Table (Key: MissileID, Value: Data)
            self.monsterDefaultData[monsterID] = rowData
        end
    end
    -- Generates the Log that indicates that the data table caching is complete.
    log("MonsterDefaultData data table caching complete!")
    return
end
-- Generates the Log that indicates that the data table caching was not completed.
log("Either the MonsterDefaultData data table has already been cached or the
MonsterDefaultData data table couldn't be found.")
end

@ExecSpace("ClientOnly")
method void InitMonsterSpawnData()
    -- Determines whether monster spawn data parsing is complete.
    if isValid(self.monsterSpawnData) == false or next(self.monsterSpawnData) == nil
then
        local getTableData = _DataService:GetTable("MonsterSpawnData")
        -- Displays warning text if it failed to load the data table.
        if getTableData == nil then
            log("Failed to load the MonsterSpawnData data table!")
            return
        end
        -- Calculates the total number of Rows.
        local rowCount = getTableData:GetRowCount()

```

```

    for i = 1, rowCount do
        -- Plugs in a single MissileID that will be the Key, starting with the
data table Row.
        local stageID = getTableData:GetCell(i, "StageID")

        -- Determines whether the ID is valid.
        if stageID ~= nil and stageID ~= "" then

            -- Bundles all of the information from a single row into a table.
            local rowData =
            {
                StageID = getTableData:GetCell(i, "StageID"),
                MonsterID = tostring(getTableData:GetCell(i, "MonsterID")),
                SpawnPosX = tonumber(getTableData:GetCell(i, "SpawnPosX")),
                SpawnPosY = tonumber(getTableData:GetCell(i, "SpawnPosY")),
                EntranceTime = tonumber(getTableData:GetCell(i, "EntranceTime")),
                ItemDropRate = tonumber(getTableData:GetCell(i, "ItemDropRate")),
                MustDropItem = tostring(getTableData:GetCell(i, "MustDropItem"))
            }

            -- Converts the StageID that will be registered as a key value into
number format.
            stageID = tonumber(stageID)
            -- Saves to cache table. (Key: MissileID, Value: Data)
            if self.monsterSpawnData[stageID] == nil then
                self.monsterSpawnData[stageID] = {}
            end
            table.insert(self.monsterSpawnData[stageID], rowData)
        end
    end

    -- Generates the Log that indicates that the data table caching is complete.
    log("MonsterSpawnData data table caching complete!")
    return
end

-- Generates the Log that indicates that the data table caching was not completed.
log("Either the MonsterSpawnData data table has already been cached or the
MonsterSpawnData data table couldn't be found.")
end

@ExecSpace("ClientOnly")
method void InitSkillData()
    -- If there is no cached skillData, it attempts to cache the data table.
    if not isValid(self.skillData) or not next(self.skillData) then
        local findData = _DataService:GetTable("SkillData")
        -- Generates a log and a code if no SkillData is found.
        if not isValid(findData) then
            log("Failed to load SkillData!")
            return
        end
        -- Calculates the total number of Rows.
        local rowCount = findData:GetRowCount()

        for i = 1, rowCount do
            -- Plugs in a single MissileID that will be the Key, starting with the
data table Row.

```



```

        local skillID = findData:GetCell(i, "SkillID")

        -- Determines whether the ID is valid.
        if skillID ~= nil and skillID ~= "" then

            -- Bundles all of the information from a single row into a table.
            local rowData =
            {
                SkillID = findData:GetCell(i, "SkillID"),
                SkillType = findData:GetCell(i, "SkillType"),
                IsPierce = findData:GetCell(i, "IsPierce"),
                AtkValue = tonumber(findData:GetCell(i, "AtkValue")),
                MoveSpeed = tonumber(findData:GetCell(i, "MoveSpeed")),
                LifeTime = tonumber(findData:GetCell(i, "LifeTime")),
                MaxHitCount = tonumber(findData:GetCell(i, "MaxHitCount")),
                HitInterval = tonumber(findData:GetCell(i, "HitInterval")),
                ModelRUID = findData:GetCell(i, "ModelRUID"),
                HitEffectRUID = findData:GetCell(i, "HitEffectRUID")
            }

            -- Saves to cache table. (Key: MissileID, Value: Data)
            self.skillData[skillID] = rowData
        end
    end

    -- Generates the Log that indicates that the data table caching is complete.
    log("SkillData data table caching complete!")
    return
end

-- Generates the Log that indicates that the data table caching was not completed.
log("Either the SkillData data table has already been cached or the SkillData data
table couldn't be found.")
end

@ExecSpace("ClientOnly")
method void InitItemData()
    -- Checks whether the ItemData data table is parsed.
    if not invalid(self.itemData) or not next(self.itemData) then
        -- Searches for the ItemData data table if ItemData data table parsing is
        required.
        local itemDefaultData = _DataService:GetTable("ItemData")
        if not invalid(itemDefaultData) then
            -- Generates and Returns the log, since the ItemData data table couldn't
            be found.
            log("The ItemData data table couldn't be found!")
            return
        end
        -- Calculates the total number of Rows.
        local rowCount = itemDefaultData:GetRowCount()

        -- Plugs in the ItemID that will be the Key, starting at Row 1 in the data
        table.
        for i = 1, rowCount do
            local itemID = itemDefaultData:GetCell(i, "ItemID")

```

```

        -- Determines whether the ID is valid.
        if itemID ~= nil and itemID ~= "" then

            -- Bundles all of the information from the target row into a single
table.
            local rowData =
            {
                ItemID = itemID,
                ValueType = tonumber(itemDefaultData:GetCell(i, "ValueType")),
                TargetPlayerStateName = tostring(itemDefaultData:GetCell(i,
"TargetPlayerStateName")),
                NumberValue = tonumber(itemDefaultData:GetCell(i, "NumberValue")),
                StringValue = tostring(itemDefaultData:GetCell(i, "StringValue")),
                SpriteRUID = tostring(itemDefaultData:GetCell(i, "SpriteRUID")),
                GainEffect = tostring(itemDefaultData:GetCell(i, "GainEffect"))
            }

            -- Saves to cache table. (Key: ItemID, Value: Data)
            self.itemData[itemID] = rowData
        end
    end
    -- Generates the Log that indicates that the data table caching is complete.
    log("ItemData data table caching complete!")
    return
end
-- Generates the Log that indicates that the data table caching was not completed.
log("Either the ItemData data table has already been cached or the ItemData data
table couldn't be found.")
end

@ExecSpace("ClientOnly")
method void InitItemDropData()
    -- Determines whether the ItemDropData data table is parsed.
    if not isValid(self.itemDropData) or not next(self.itemDropData) then
        -- Searches the ItemDropData data table, since ItemDropData data table parsing
is required.
        local itemDropDefaultData = _DataService:GetTable("ItemDropData")
        if not isValid(itemDropDefaultData) then
            -- Generates and Returns the log, since the ItemDropData data table
couldn't be found.
            log("The ItemDropData data table couldn't be found!")
            return
        end
        -- Calculates the total number of Rows.
        local rowCount = itemDropDefaultData:GetRowCount()

        -- Plugs in the ItemID that will be the Key, starting at Row 1 in the data
table.
        for i = 1, rowCount do
            local itemID = itemDropDefaultData:GetCell(i, "ItemID")

            -- Determines whether the ID is valid.
            if itemID ~= nil and itemID ~= "" then

```

```

        -- Bundles all of the information from the target row into a single
table.
        -- Note: Numbers loaded to GetCell may be strings, so using tonumber()
is recommended for stat values.
        local rowData =
        {
            ItemID = itemID,
            DropRate = tonumber(itemDropDefaultData:GetCell(i, "DropRate")),
        }

        -- Saves to cache table. (Key: ItemID, Value: Data)
        self.itemDropData[itemID] = rowData
    end
end
-- Generates the Log that indicates that the data table caching is complete.
log("ItemDropData data table caching complete!")
return
end
-- Generates the Log that indicates that the data table caching was not completed.
log("Either the ItemDropData data table has already been cached or the
ItemDropData data table couldn't be found.")
end

@ExecSpace("ClientOnly")
method table GetMissileDataByMissileID(string MissileID)
-- Determines whether missileData has been properly cached.
if invalid(self.missileData) or next(self.missileData) ~= nil then
    -- If there is a Value that uses MissileID as a Key, that Value is returned.
    if self.missileData[MissileID] then
        return self.missileData[MissileID]
    end
-- Generates an error log if the missileData isn't cached.
else
    log("There is no missile data parsing!")
    return nil
end
end

@ExecSpace("ClientOnly")
method table GetCharacterDefaultDataByCharacterID(string CharacterID)
-- Attempts caching if the characterDefaultData hasn't been cached normally.
if self.characterDefaultData == nil then
    self:InitCharacterDefaultData()
end
-- Determines whether CharacterDefaultData is currently registered as intended.
if invalid(self.characterDefaultData) and next(self.characterDefaultData) ~= nil
then
    -- Returns any data that has the CharacterID value.
    if self.characterDefaultData[CharacterID] then
        return self.characterDefaultData[CharacterID]
    end
-- Generates an error log if the relevant Key's data is missing.
else
    log("Either the "..CharacterID.." value doesn't have PlayerDefaultData data or

```

```

it doesn't exist!")
    return nil
end
end

@ExecSpace("ClientOnly")
method table GetCharacterSelectUIDataTable()
-- Returns the character selection UI data table in its entirety.
if invalid(self.characterSelectUIData) and next(self.characterSelectUIData) ~= nil
then
    return self.characterSelectUIData
else
    -- Generates an error log if there is no value to return.
    log("Either the CharacterSelectUIData data table doesn't exist or it hasn't
been cached properly!")
    return nil
end
end

@ExecSpace("ClientOnly")
method table GetMonsterSpawnDataTableByStageID(number StageID)
-- If the StageID received isn't a normal value that is 1 or higher, it generates
a log and ends loading.
if StageID <= 0 then
    log("The value delivered to call SpawnData from DataTableManager isn't 1 or
higher!")
    return
end
-- Returns the log and data together if the monster spawn data can be delivered
normally.
if self.monsterSpawnData[StageID] ~= nil then
    log("Delivered Spawn Data Value: "..self.monsterSpawnData[StageID]
[1].MonsterID)
    return self.monsterSpawnData[StageID]
end
-- Generates an error log if the spawn data can't be returned.
log("Couldn't find the StageID value data that is "StageID.." in the
MonsterSpawnData data table!")
return nil
end

@ExecSpace("ClientOnly")
method table GetMonsterDefaultDataByMonsterID(string MonsterID)
-- Determines whether monsterDefaultData has been properly cached.
if invalid(self.monsterDefaultData) or next(self.monsterDefaultData) ~= nil then
    -- Returns any data that has a MonsterID value that can be returned.
    if self.monsterDefaultData[MonsterID] then
        return self.monsterDefaultData[MonsterID]
    end
-- Generates an error log if no data can be returned.
else
    log("There is no monster data parsing!")
    return nil
end
end

```

```
end

@ExecSpace("ClientOnly")
method table GetSkillDataBySkillID(string SkillID)
-- Attempts caching if the skillData hasn't been cached.
if not invalid(self.skillData) or not next(self.skillData) then
    self:InitSkillData()
end
-- If there is a Value that can use skillData and has SkillID as a Key, that Value
is returned.
if invalid(self.skillData[SkillID]) and next(self.skillData[SkillID]) then
    return self.skillData[SkillID]
end
-- Generates an error log if no data can be returned.
log("Couldn't find SkillData with "SkillID.." as the name!")
return nil
end

@ExecSpace("ClientOnly")
method table GetItemDataByItemID(string ItemID)
-- Attempts caching if the itemData hasn't been cached.
if not invalid(self.itemData) or not next(self.itemData) then
    self:InitItemData()
end
-- If there is a Value that uses an ItemID from itemData as a Key, that Value is
returned.
if invalid(self.itemData[ItemID]) and next(self.itemData[ItemID]) then
    return self.itemData[ItemID]
end
-- Generates an error log if no ItemID value can be returned.
log("Couldn't find ItemData with "ItemID.." as the name!")
return nil
end

@ExecSpace("ClientOnly")
method table GetItemDropData()
-- Attempts caching if the itemDropData hasn't been cached.
if not invalid(self.itemDropData) or not next(self.itemDropData) then
    self:InitItemDropData()
end
-- Returns all itemDropData if itemDropData can be returned.
if invalid(self.itemDropData) and next(self.itemDropData) then
    return self.itemDropData
end
-- Returns nil with an error log if returning is not possible.
log("ItemDropData value isn't registered to DataTableLogic!")
return nil
end

end
```

## Writing UIManageLogic Code

```

@Logic
script UIManageLogic extends Logic

property TitleUIController titleUIController = nil

property SelectCharacterUIController selectCharacterUIController = nil

property ResultUIController resultUIController = nil

property DefaultUIController defaultUIController = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- Searches for and registers TitleUIController when the game starts.
self:InitTitleUIController()
-- Searches for and registers SelectCharacterUIController when the game starts.
self:InitSelectCharacterUIController()
-- Searches for and registers ResultUIController when the game starts.
self:InitResultUIController()
-- Searches for and registers DefaultUIController when the game starts.
self:InitDefaultUIController()
end

@ExecSpace("ClientOnly")
method void InitTitleUIController()
-- If there is no registered TitleUIController, registers the target component.
if not isvalid(self.titleUIController) then
    local titleUIEntity = _EntityService:GetEntityByPath("/ui/TitleUI")
    -- If titleUIGroup is successfully found, registers the target entity's
    component.
    if isvalid(titleUIEntity) then
        local component = titleUIEntity.TitleUIController
        -- Determines whether the component has been registered normally and then
        assigns it.
        if isvalid(component) then
            self.titleUIController = component
        end
    end
end
end

@ExecSpace("ClientOnly")
method void InitSelectCharacterUIController()
-- If there is no registered SelectCharacterUIController, registers the target
component.
if not isvalid(self.selectCharacterUIController) then
    local selectCharacterUIEntity =
_EntityService:GetEntityByPath("/ui/SelectCharacterUI")
    -- If SelectCharacterUIController is successfully found, registers the target
entity's component.

```

```

        if invalid(selectCharacterUIEntity) then
            local component = selectCharacterUIEntity.SelectCharacterUIController
            -- Determines whether the component has been registered normally and then
assigns it.
            if invalid(component) then
                self.selectCharacterUIController = component
            end
        end
    end
end
end

@ExecSpace("ClientOnly")
method void ShowTitle()
    -- Shows the title screen.
    self.titleUIController:Show()
    -- Hides all other UI.
    self.selectCharacterUIController:Hide()
    self.resultUIController:Hide()
    self.defaultUIController:Hide()
end

@ExecSpace("ClientOnly")
method void CallSelectCharacterUI()
    -- Shows the character selection screen.
    self.selectCharacterUIController:Show()
    -- Attempts to reset the character list in the character selection screen.
    self.selectCharacterUIController:InitCharacterSelectFrame()
end

@ExecSpace("ClientOnly")
method void ShowInGame()
    -- Resets the defaultUI of the game screen UI.
    self.defaultUIController:Show()
    -- Requests a reset of the auto-attack UI.
    self.defaultUIController:UpdateAutoUI()
    -- Hides other UI.
    self.titleUIController:Hide()
    self.selectCharacterUIController:Hide()
    self.resultUIController:Hide()
end

@ExecSpace("ClientOnly")
method void InitResultUIController()
    -- If there is no registered ResultUIController, registers the target component.
    if not invalid(self.resultUIController) then
        local resultUIEntity = _EntityService:GetEntityByPath("/ui/ResultUI")
        -- If ResultUIController is successfully found, registers the target entity's
component.
        if invalid(resultUIEntity) then
            local component = resultUIEntity.ResultUIController
            -- Determines whether the component has been registered normally and then
assigns it.
            if invalid(component) then
                self.resultUIController = component
            end
        end
    end
end

```

```

        end
    end
end
end

@ExecSpace("ClientOnly")
method void ShowResultUI(boolean IsClear)
-- Displays the results UI.
self.resultUIController:Show(IsClear)
if IsClear then
    -- Play Congratulations Sound Effect
    _SoundService:PlaySound("5d57b617cfc242cca347925e209c8e05", 1.0)
else
    -- Play Failure Sound Effect
    _SoundService:PlaySound("52dccfb8889b4181b9181780dc7e9040", 1.0)
end
end

@ExecSpace("ClientOnly")
method void InitDefaultUIController()
--If there is no registered defaultUIController, registers the target component.
if not invalid(self.defaultUIController) then
    local titleUIEntity = _EntityService:GetEntityByPath("/ui/DefaultUI")
    --If DefaultUIGroup is successfully found, registers the target entity's
    component.
    if invalid(titleUIEntity) then
        local component = titleUIEntity.DefaultUIController
        -- Determines whether the component has been registered normally and then
        assigns it.
        if invalid(component) then
            self.defaultUIController = component
        end
    end
end
end

@ExecSpace("ClientOnly")
method void UpdateLifeCountUI(number PlayerLife)
-- Updates the DefaultUI's HP UI in accordance with the PlayerLife number.
self.defaultUIController:UpdateLifeUI(PlayerLife)
end

@ExecSpace("ClientOnly")
method void UpdateSkillCountUI(number LeftSkillCount)
-- Updates the DefaultUI's skill UI in accordance to the LeftSkillCount number.
self.defaultUIController:UpdateSkillUI(LeftSkillCount)
end

end

```



## Pros of Logic

- It can be accessed conveniently from anywhere.
- There is no need to look for specific entities like components or to check if the found entity has a component.
- Making Logic will make that Logic reliable.
- Since Logic manages functions, when issues of responsibility arise, you can solve the issue by only fixing the Logic that is responsible.

## Notes on Logic Usage

- There is a risk of a single super Logic that controls all functions.
  - Example: A Logic in charge of damage calculation also managing damage value processing, sound output, hit effect output, and more.
- In the case of a super logic, the Logic will require more information that it necessarily needs.
- This goes outside of the theory of object-oriented design, which becomes the cause of the great increase to lines of code.
- The increased lines of code reduce readability, which then makes finding the issue quickly very difficult during debugging.

## Minimum Implementation Standards

- Create a Logic that will manage and make use of future DataSets.
  - Create a sample world-based DataTableLogic to manage DataSets.
- Create a Logic that can handle processing based on game progression.
  - Create a sample world-based GameLogic to implement processing based on game state.
- Create a Logic that can manage all UIs.
  - Create a sample world-based UIManageLogic to implement a function that can manage multiple UIs.

## Usage and Extension Direction

- Create a CalcDamageLogic that calculates the Final Damage according to the formula after receiving the damage value to manage a damage calculation function.
- Use Logic to create a ConfigLogic that manages fixed values that **aren't scheduled to be changed**.
  - Example: For games that have a max of 5 lives, create a Logic that can register and use MaxHP to keep the max HP a fixed value.

---

## [💡 Basic Training] Creating a PlayerCharacter

---

## Video Timeline

- You can follow along by using the timeline under the training video.
- Creating a PlayerCharacter: 00:49:40

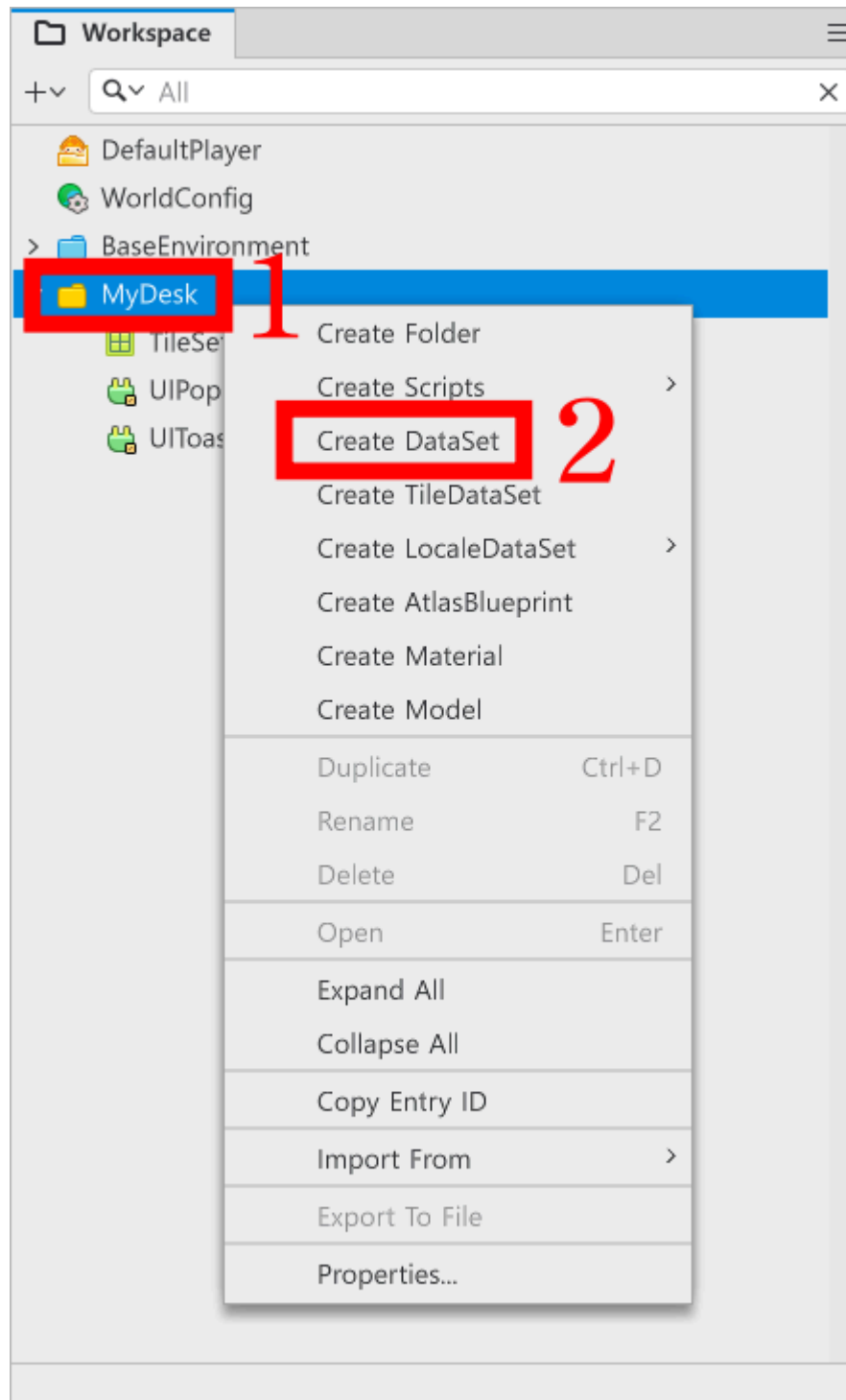
## Learning Goals

- Learn about the DataSet and how to edit it to compose a player character.
- Utilize DataTableLogic to implement a Manager for use.
- Create and use Components yourself.
- Use DataSets to reset values in Components.

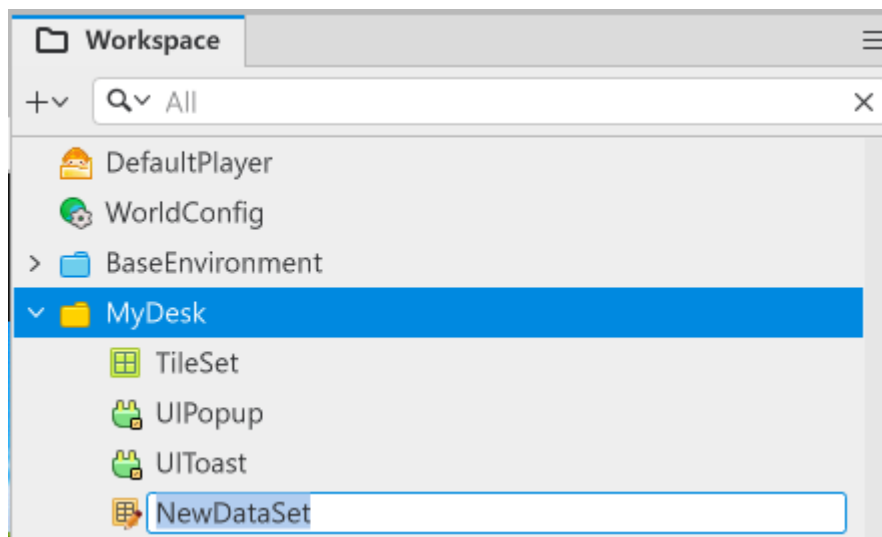
## Practical Exercises to Do After Watching the Video

- Implement a DataTableLogic to create a Logic that manages a DataSet.
- Create a user-defined PlayerState component and add it to a player character.
- Use TriggerComponent to set the collision area to a player.

## Creating a DataSet



- Right-click MyDesk from Workspace.
- Create a new DataSet by pressing Create DataSet.



- Change the name of the created DataSet.
- `CharacterDefaultData` is being used in the sample world.
- When you create a DataSet, double-click that DataSet to move to the edit window.

| Scene   Play (Preview)   Resource Storage   CharacterDefaultData X |             |         |                 |                |                |                 |                |
|--------------------------------------------------------------------|-------------|---------|-----------------|----------------|----------------|-----------------|----------------|
| 텍스트 입력                                                             |             |         |                 |                |                |                 |                |
|                                                                    | 1           | 2       | 3               | 4              | 5              | 6               | 7              |
| ID                                                                 | CharacterID | StartHP | StartSkillCount | StartMoveSpeed | DefaultMissile | DefaultSkill    | AttackInterval |
| 1                                                                  | Pirate      | 3       | 2               | 5.0            | Missile_Bullet | Skill_CannonBal | 0.5            |
|                                                                    |             |         |                 |                |                |                 |                |

- If you need to add rows and columns, press the + button to add them.
- In row 1, enter the name of the data you want to use.
- Starting from row 2, enter the value that fulfills the data's name.

## Sample World DataSet Composition

| Name            | Description                                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------------------------|
| CharacterID     | The ID value needed to classify characters. You can load the information that composes characters based on this value. |
| StartHP         | The value needed to set the character's starting HP when the game begins.                                              |
| StartSkillCount | The value needed to set the number of ultimates a character can use when the game begins.                              |
| StartMoveSpeed  | The value needed to set the character's speed when the game begins.                                                    |
| DefaultMissile  | The value needed to set the target character's default projectile.                                                     |
| DefaultSkill    | The value needed to set the character's ultimate.                                                                      |

| Name           | Description                                                                                             |
|----------------|---------------------------------------------------------------------------------------------------------|
| AttackInterval | The value needed to set the character's attack delay. The smaller the value, the faster the fire speed. |

## Registering DataSet to DataTableLogic

- Write the following code to DataTableLogic.
- The example has the code written based on PlainText.

```

1 DataTableLogic {
2     Property: +
3     [None]
4     table missileData = {} -- MissileData 데이터셋을 관리하기 위한 Property입니다. :
5     [None]
6     table characterDefaultData = {} -- CharacterDefaultData 데이터셋을 관리하기 위한 Property입니다. :
7     [None]
8     table characterSelectUIData = {} -- CharacterSelectUIData 데이터셋을 관리하기 위한 Property입니다. :
9     [None]
10    table monsterDefaultData = {} -- MonsterDefaultData 데이터셋을 관리하기 위한 Property입니다. :
11    [None]
12    table monsterSpawnData = {} -- MonsterSpawnData 데이터셋을 관리하기 위한 Property입니다. :
13    [None]
14    table skillData = {} -- SkillData 데이터셋을 관리하기 위한 Property입니다. :
15    [None]
16    table itemData = {} -- ItemData 데이터셋을 관리하기 위한 Property입니다. :
17    [None]
18    table itemDropData = {} -- ItemDropData 데이터셋을 관리하기 위한 Property입니다. :
19
20    Method: +
21    [client only]
22    void OnBeginPlay ( ) :
41    void InitMissileData ( ) :
85    void InitCharacterDefaultData ( ) :
129    void InitCharacterSelectUIData ( ) :
169    void InitMonsterDefaultData ( ) :
212    void InitMonsterSpawnData ( ) :

```

- The PlainText can be changed through the UI button on the bottom right of the code editor window.

```

property table characterDefaultData = {} -- Property for managing the
CharacterDefaultData dataset.

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- Resets the CharacterDefaultData dataset.
self:InitCharacterDefaultData()
end

method void InitCharacterDefaultData()
-- Item that determines whether player character parsing is complete.
if isValid(self.characterDefaultData) == false or next(self.characterDefaultData)
== nil then
    local getTableData = _DataService:GetTable("CharacterDefaultData")
    -- Displays warning text if it failed to load the data table.
    if getTableData == nil then

```

```

        log("Failed to fetch the CharacterDefaultData data table!")
        return
    end
    --Start Parsing
    local rowCount = getTableData:GetRowCount() -- Calculates the total number of
Rows.

    for i = 1, rowCount do
        -- Plugs in a single CharacterID that will be the Key, starting at the
data table Row.
        local characterID = tostring(getTableData:GetCell(i, "CharacterID"))

        -- Determines whether the ID is valid.
        if characterID ~= nil and characterID ~= "" then

            -- Bundles all of the information from a single row into a table.
            local rowData =
            {
                CharacterID = characterID,
                StartHP = tonumber(getTableData:GetCell(i, "StartHP")),
                StartSkillCount = tonumber(getTableData:GetCell(i,
"StartSkillCount")),
                StartMoveSpeed = tonumber(getTableData:GetCell(i,
"StartMoveSpeed")),
                DefaultMissile = tostring(getTableData:GetCell(i,
"DefaultMissile")),
                DefaultSkill = tostring(getTableData:GetCell(i, "DefaultSkill")),
                AttackInterval = tonumber(getTableData:GetCell(i,
"AttackInterval"))
            }

            -- Save to Cache Table (Key: MissileID, Value: Data)
            self.characterDefaultData[characterID] = rowData
        end
    end
    -- Generates the Log that indicates that the data table caching is complete.
    log("CharacterDefaultData data table caching complete!")
    return
end
-- Generates the Log that indicates that the data table caching was not completed.
log("Either the CharacterDefaultData data table has already been cached or the
CharacterDefaultData data table couldn't be found.")
end

method table GetCharacterDefaultDataByCharacterID(string CharacterID)
-- Attempts caching if the characterDefaultData hasn't been cached normally.
if self.characterDefaultData == nil then
    self:InitCharacterDefaultData()
end
-- Determines whether CharacterDefaultData is currently registered as intended.
if isValid(self.characterDefaultData) and next(self.characterDefaultData) ~= nil
then
    -- Returns any data that has the CharacterID value.
    if self.characterDefaultData[CharacterID] then

```

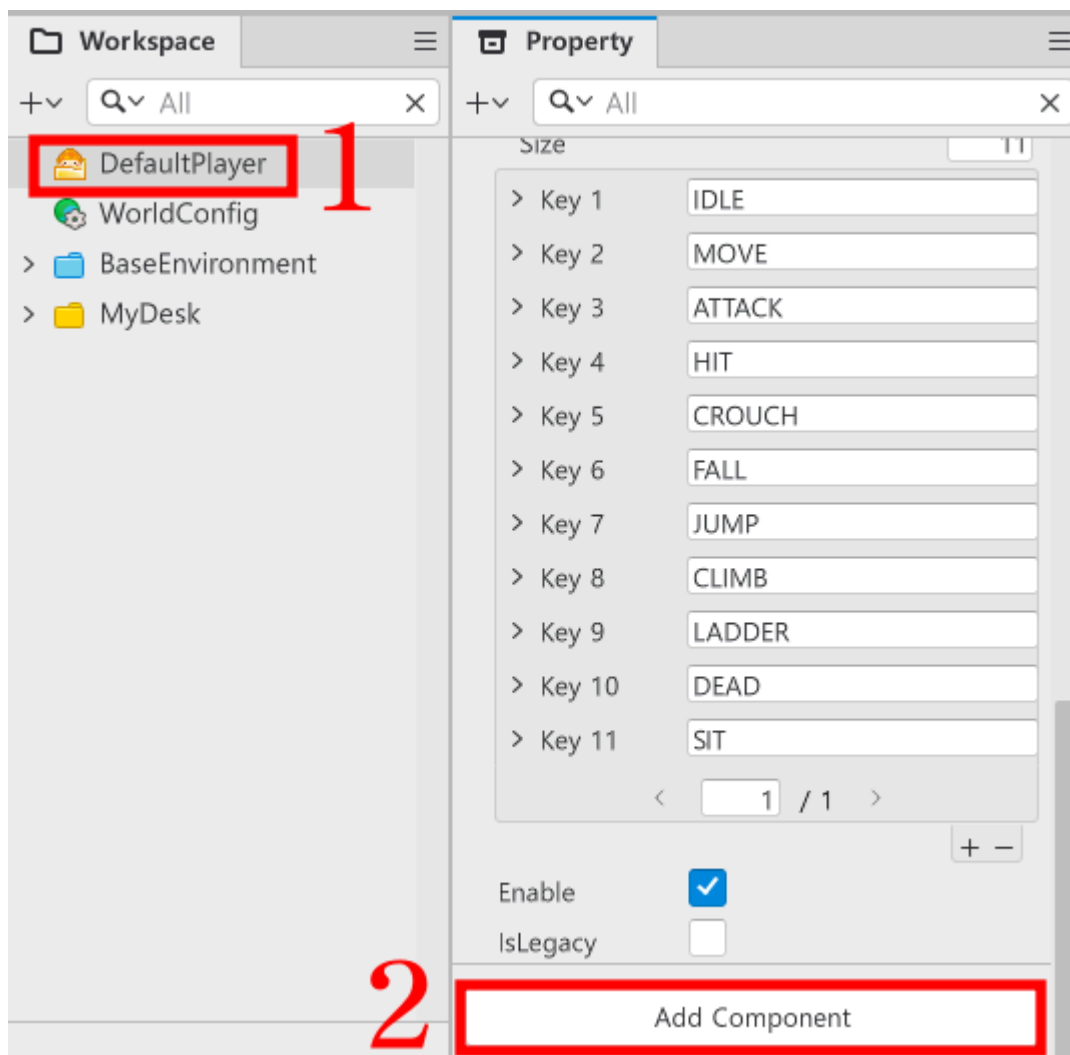
```

    return self.characterDefaultData[CharacterID]
end
-- Generates an error log if the relevant Key's data is missing.
else
    log("Either the "..CharacterID.." value doesn't have PlayerDefaultData data or
it doesn't exist!")
    return nil
end
end
end

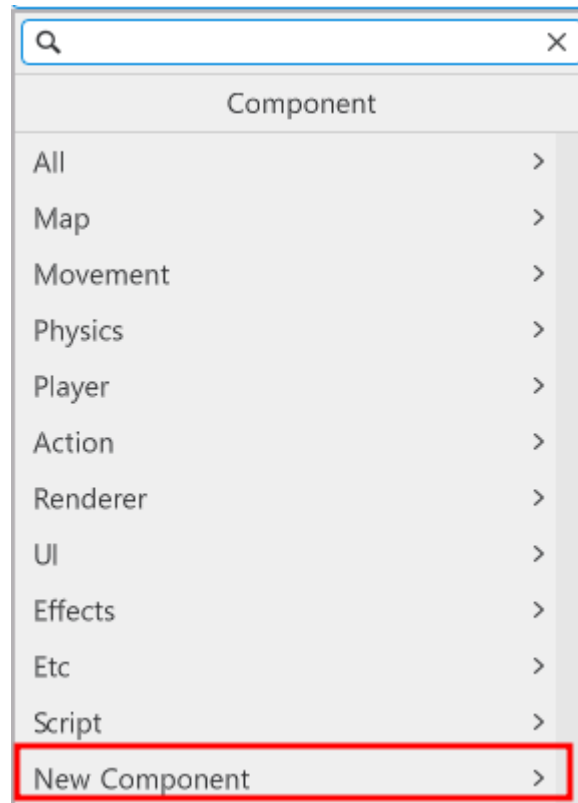
```

## Creating a PlayerState Component

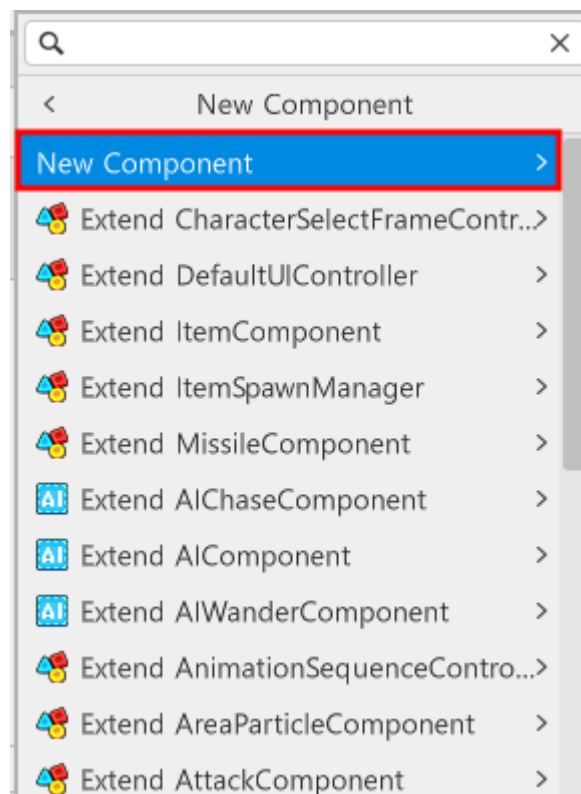
- If you saved player information to a DataSet, the saved information must be used.



- Click DefaultPlayer from Workspace.
- Drag the Property window to the bottom and click Add Component.

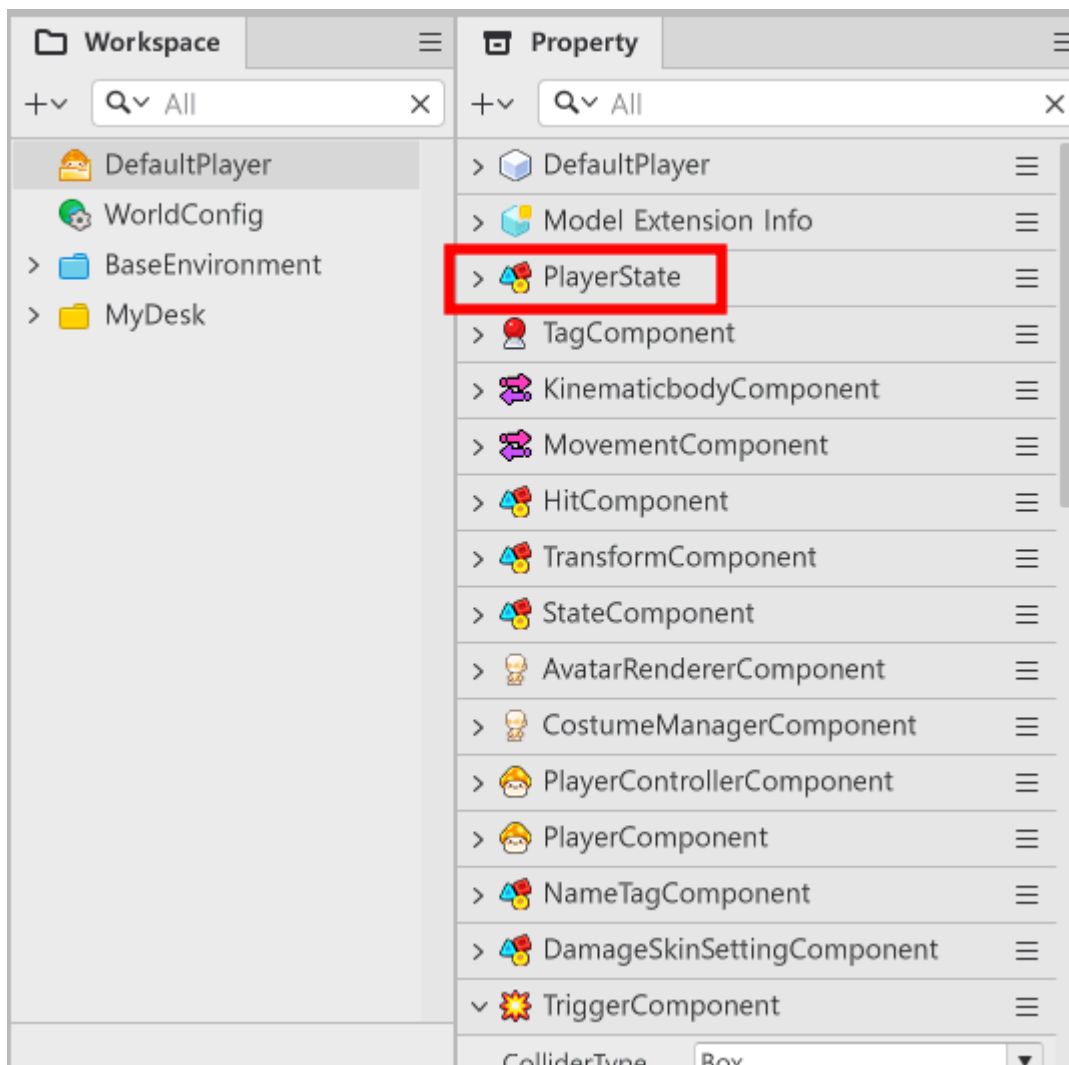


- Click NewComponent from the added menu.



- Click New Component, enter the name of the Component that will be created, and click Create and Add.
- The Component's name in the sample world is **PlayerState**.





- When registered normally, when clicking DefaultPlayer, the Component that was entered will be registered in the Property window.

## Resetting Values to PlayerState

- Write the following code to PlayerState.
- This code is written based on Plain Text.

```
@Component
script PlayerState extends Component

property number currentHP = 0 -- Property that manages the player's current HP.

property string currentMissile = "" -- Property that saves the player's current
projectile's ID.

property string characterID = "" -- Property that saves the ID of the character
that is currently being played.

property number attackInterval = 0.5 -- Property that manages the player's attack
delay.
```

```
property number attackTimer = 0 -- Property that calculates the player's attack
availability.

property any projectilePoolManager = nil

property Vector3 firePos = Vector3(0.25,0.25,0) -- Property that adjusts the
player's attack launch position.

property boolean isInvincibility = false -- Property that determines whether the
player is invincible.

property number invincibilityDelta = 0 -- Property that calculates invincibility
duration.

property number invincibilityTime = 1.0 -- Base Property for invincibility
duration. The duration can be changed by changing this value.

property boolean isAlive = false -- Property that determines the player's
survival.

property string skillID = "" -- Property that saves the player's used skill ID.

property number canUseSkillCount = 0 -- Property that manages the usable skill
count of the current player.

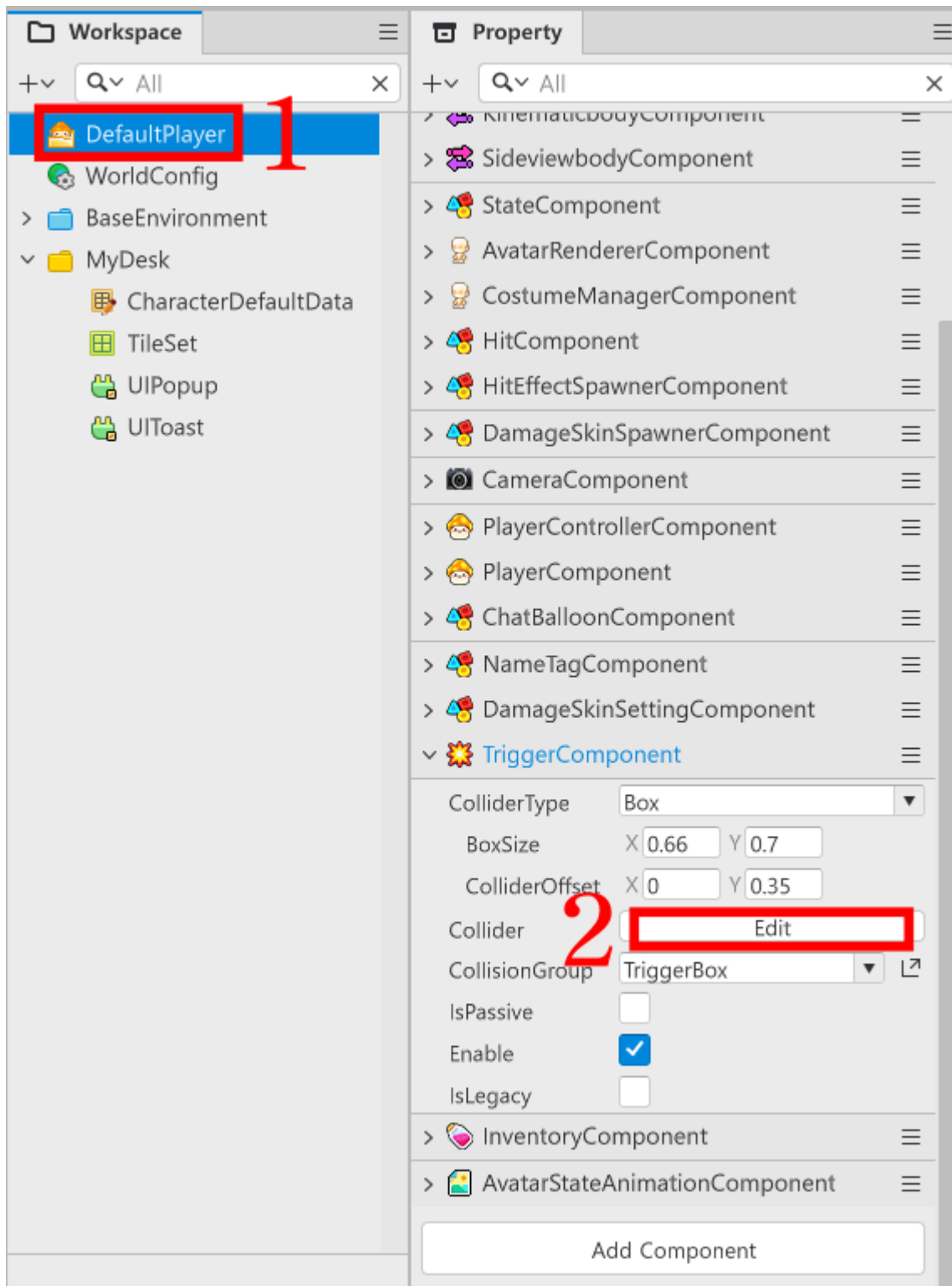
property number moveSpeed = 0 -- Property that registers the player's speed.

@ExecSpace("ClientOnly")
method void InitPlayerData(table PlayerDefaultData)
-- Generates an error log and halts the code if the received PlayerData is
abnormal data.
if PlayerDefaultData == nil or next(PlayerDefaultData) == nil then
    log("PlayerDefaultData is not normal!")
    return
end
-- Resets the player's current HP.
self.currentHP = PlayerDefaultData.StartHP
-- Resets the player's usable skill ID.
self.canUseSkillCount = PlayerDefaultData.StartSkillCount
-- Resets the player's speed.
self.moveSpeed = PlayerDefaultData.StartMoveSpeed
-- Delivers to MovementComponent so that the reset player speed can be implemented
as intended.
self.Entity.MovementComponent.InputSpeed = self.moveSpeed
-- Resets the player's default projectile ID.
self.currentMissile = PlayerDefaultData.DefaultMissile
-- Resets the player's character ID value.
self.characterID = PlayerDefaultData.CharacterID
-- Resets the player's attack delay.
self.attackInterval = PlayerDefaultData.AttackInterval
-- Resets the player's skill ID.
self.skillID = PlayerDefaultData.DefaultSkill
-- Resets the player's starting position.
```

```
self.Entity.TransformComponent.Position = Vector3.zero
-- Changes the player Entity's usage to true.
self.Entity.Enable = true
-- Changes the player character's Visible state to true.
self.Entity.Visible = true
end

@ExecSpace("ClientOnly")
method void OnBeginPlay()
    local getData = _DataTableLogic:GetCharacterDefaultDataByCharacterID("Pirate")
    self:InitPlayerData(getData)
    self.isAlive = true
end
```

## Editing TriggerComponents



- Click DefaultPlayer from Workspace.
- Find TriggerComponent in the Property window and click Edit.



- Drag and drop the green area to adjust the size to fit the character's image.

## What is a TriggerComponent?

- A TriggerComponent is a component used to detect collisions between entities.
- The green area is the collision trigger area.
- When the areas overlap, a collision is triggered.
- So, when there is a detection that is larger or smaller than the image resource, the player may think that the collision is unfair.
- However, depending on the game, the detection area can be intentionally increased to create generous detection.
- Depending on the type of game you're making, you will need to plan whether the collision will be precise or have some room.

## Minimum Implementation Standards

- You can create a DataSet.
- You can load the desired value from the DataSet through the created DataTableLogic.
- You can deliver the loaded DataSet value to the Component and plug it in.
- You can set the collision area by editing the TriggerComponent.

## Usage and Extension Direction

- You can divide the player relation functions that will be created into units of Components.
  - Example: For attacks, you can create a PlayerAttackComponent and implement the attack-related function to the target Component.
- Add needed values for game creation to the DataSet and edit the DataTableLogic, so the added values can be loaded.
  - By learning how to do this and based on need, you will be able to expand or add a DataSet.

---

## [💡 Basic Training] Creating Monsters

---

### Video Timeline

- You can follow along by using the timeline under the training video.
- Creating Monsters: 01:21:22

### Learning Goals

- Create enemies that are threats to the player to create a sense of tension.
- Create an entity and add Components to compose the enemy.
- Utilize Components provided by MapleStory Worlds and implement a function that makes enemies chase the player.

### Practical Exercises to Do After Watching the Video

- Create an empty entity and add the needed Components to create a Model.
- Create Monsters of various appearances based on what you need.
- Manage the Monster values from a DataSet and make it so they have different stats.

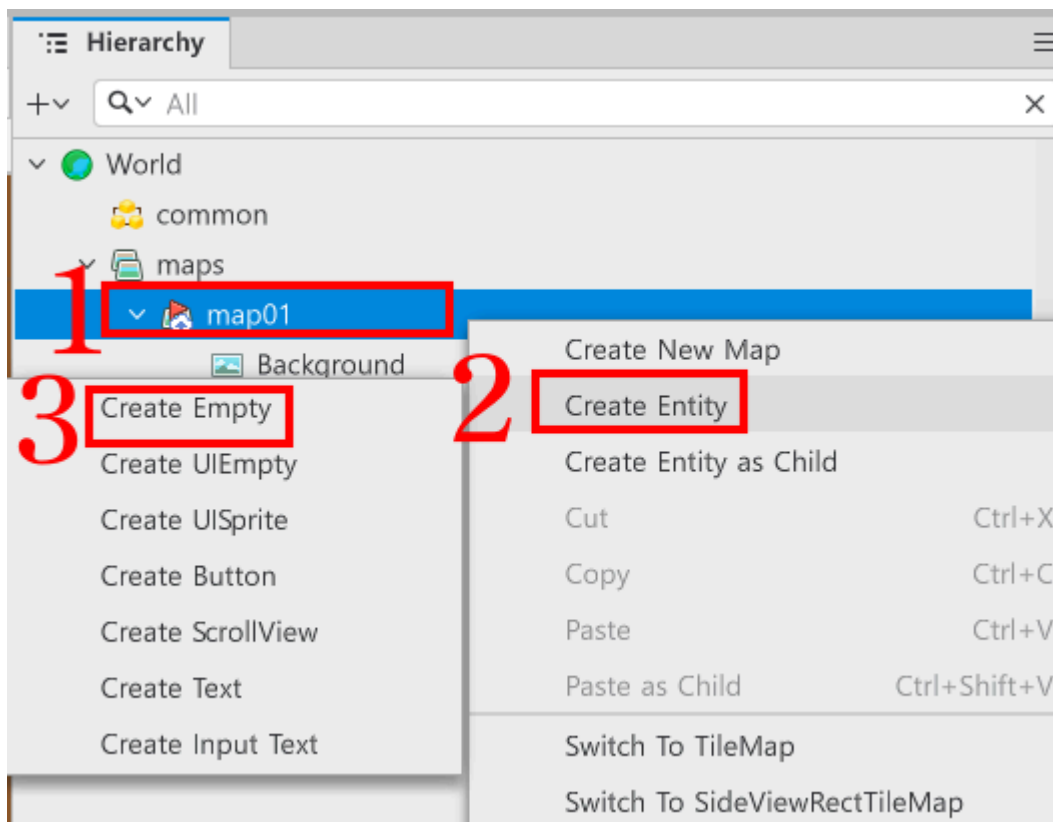
## Composing a MonsterDataSet

- Create a DataSet used to save information related to Monsters.
- The name of the DataSet in the sample world is `MonsterDefaultData`.
- If you don't know how to make a DataSet, make one by referencing the PlayerCharacter from the relevant document.

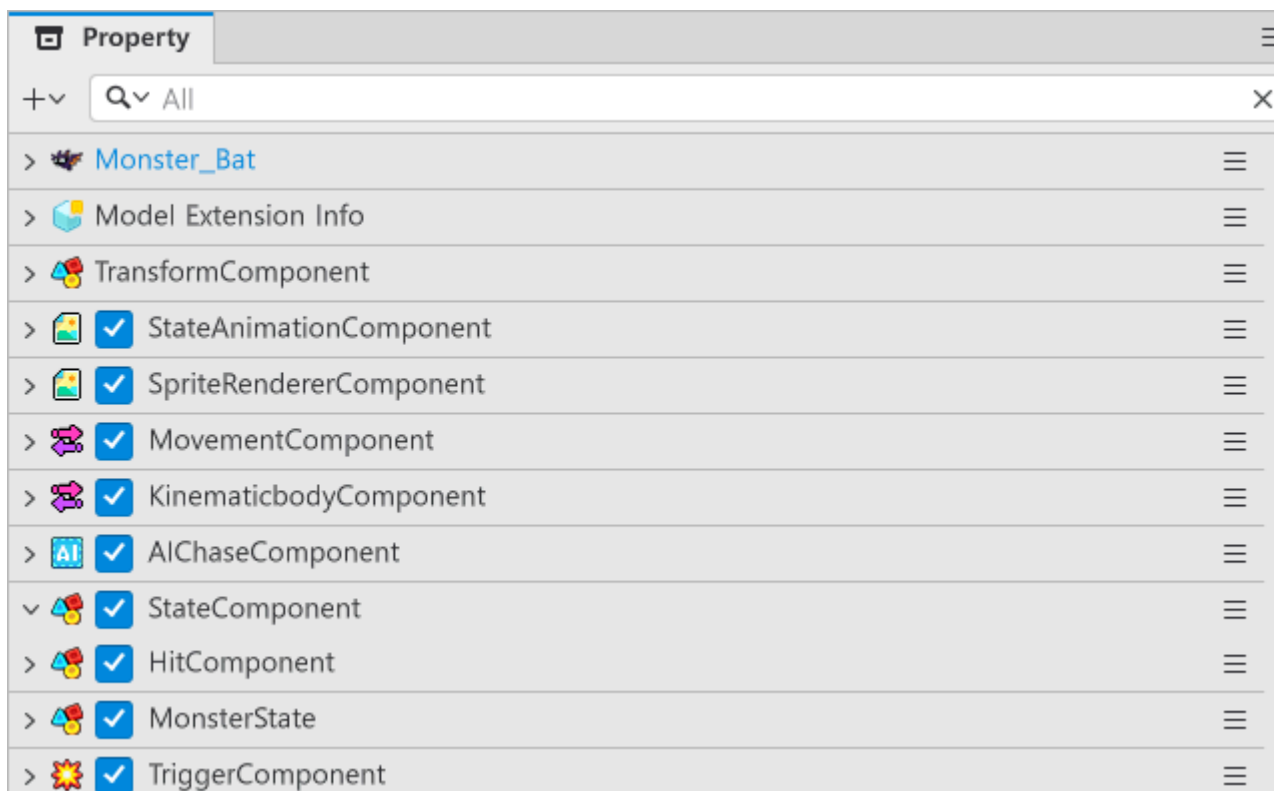
### Composing Elements of MonsterDefaultData

| Name           | Description                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| MonsterID      | The ID value needed to classify Monsters. You can classify monsters and load data based on this value.                                    |
| StartHP        | The value needed to set the Monster's HP when it is spawned in the game.                                                                  |
| StartMoveSpeed | The value needed to set the Monster's default speed when it is spawned in the game.                                                       |
| DefaultMissile | The value needed to set the Monster's default projectile.                                                                                 |
| AttackInterval | The value needed to set the Monster's attack delay. The smaller the value, the faster the fire speed.                                     |
| ModelRUID      | The value needed to set the created Monster Model's RUID. The Model can be copied based on this value from MonsterSpawnManager in future. |

## Composing Monsters Entities



- Right-click map01 from Hierarchy.
- Click Create Entity.
- Click Create Empty to create an empty entity as the child entity of map01.
- You can change the name of the created entity.
- The sample world named it **Monster\_Bat** to use as an example.

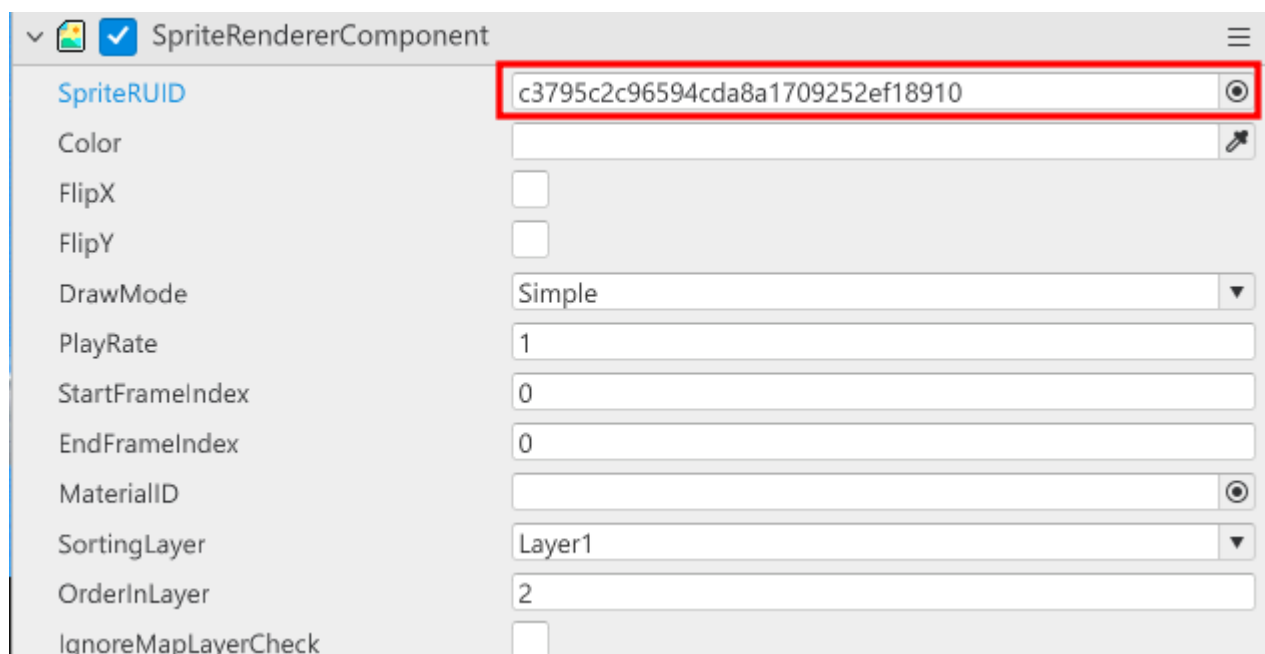


- Add the following Components to the created entities.
- **SpriteRendererComponent**: A Component used to output the Monster's image.

- **MovementComponent:** A Component needed for the Monster to move. The AIChaseComponent uses this Component to chase the player.
- **KinematicBodyComponent:** Makes it so that the monster can move in a RectTileMap. Change the Monster's EnableTileCollision value to false, so there aren't any areas that limit its movement.
- **AIChaseComponent:** A Component provided by MapleStory Worlds. Entities with this Component can chase the player.
- **HitComponent:** A Component needed for damage calculations when hit by a projectile.
- **MonsterState:** A Component that needs to be created directly. This Component contains Monster data or manages functions like attacks or death similar to PlayerState.
- **TriggerComponent:** A Component that sets a Monster's collision area. Set the Monster's collision area with this Component and check the collision status of projectiles or skills.

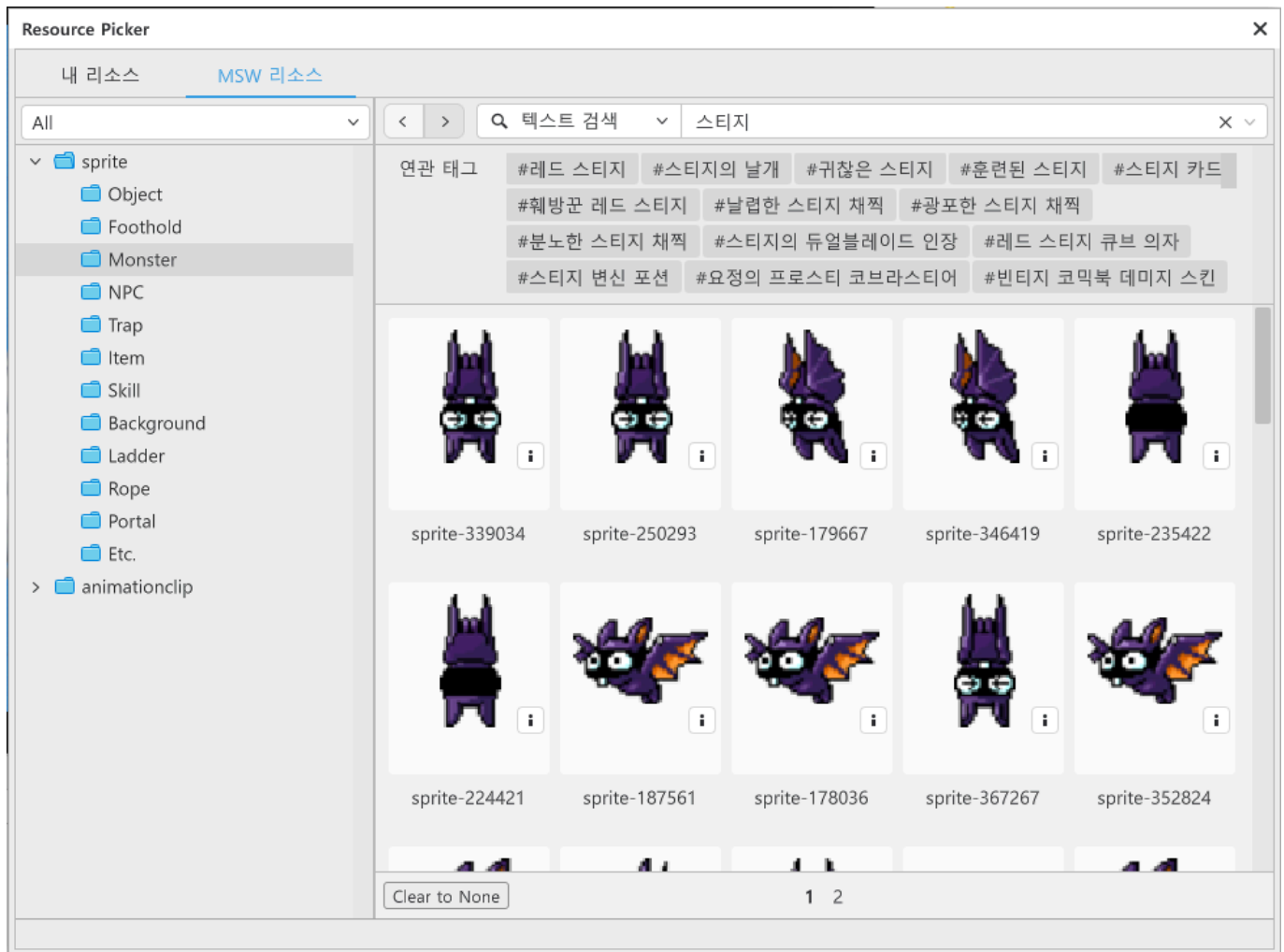
## Changing Monster Images

- Change and apply the SpriteRUID under SpriteRendererComponent to change a monster's image.

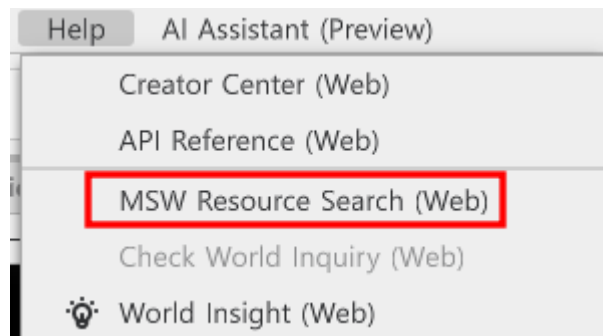


- Change the SpriteRUID value to change a monster's image.
- Use Resource Picker on the right to search and use other images.





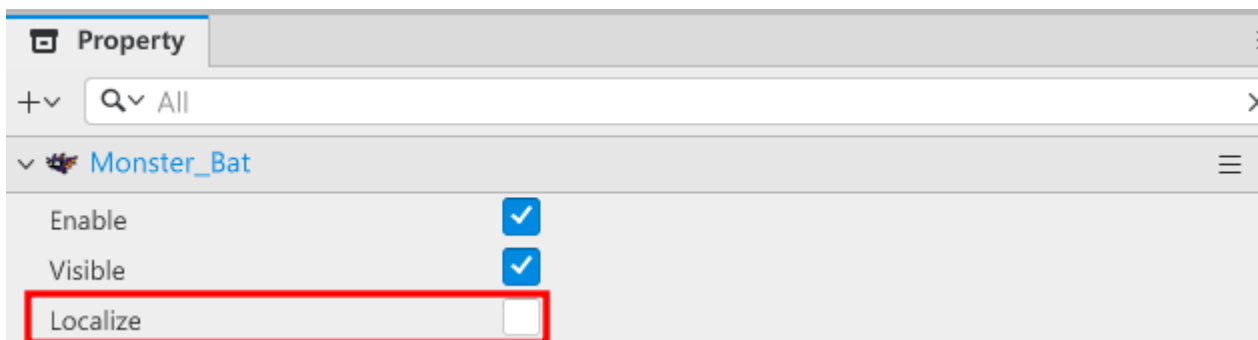
- Use resources provided by MapleStory Worlds by searching resources in the Resource Picker.



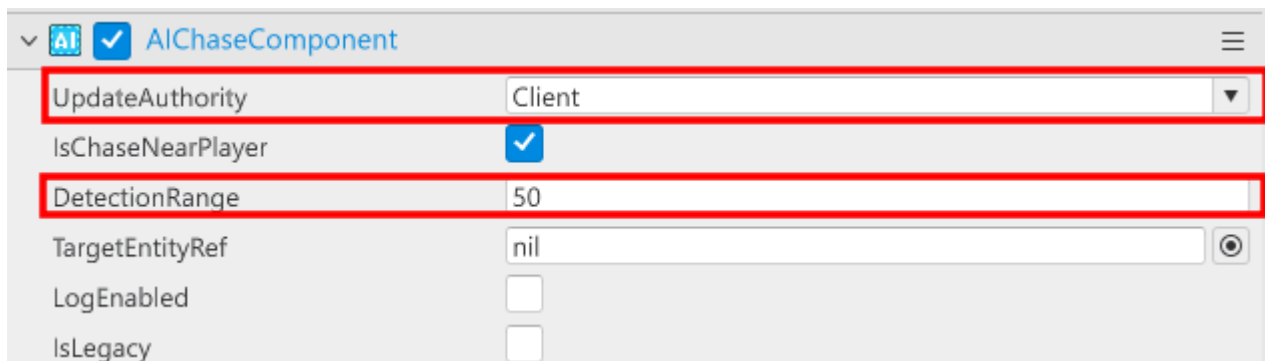
- If using the Resource Picker is too difficult, search for resources through Help on the top of the editor.

## Composing a Monster that Chases the Player

- The following elements must be applied to make the monster chase the player.



- Localize must be activated from Property. - The sample world is built based on the fact that it is client-driven. So Localize must be checked to ensure the Entity can function in local.



- Change the UpdateAuthority under AIChaseComponent to Client.
  - If this value is Server, the monster may not chase the player as intended in the sample world that is client-driven.
- Increase the DetectionRange value as needed to increase the enemy's player detection range.

## Composing a MonsterState Component

- Write the code of the MonsterState Component that was added to the Model.
- The Monster can now search for MonsterDefaultData and receive the value through this code.
- Write the following code.
- This code is written based on Plain Text.

```
@Component
script MonsterState extends Component

property number currentHP = 0 -- Property that manages the monster's current HP.

property number attackInterval = 0 -- Property that saves the monster's attack
delay.

property boolean isAlive = true -- Property that checks the monster's survival.

property string defaultMissile = "" -- Property that saves the ID of the
projectile fired by a monster.

property ProjectileManager projectileManager = nil

property Vector2 firePos = Vector2(0,0) -- Property that sets the position where
```

the projectile is being fired from.

property number deltaTime = 0 -- Property that calculates the monster's missile fire delay.

property number itemDropRate = 0 --Property that calculates the item drop rate when a monster is defeated.

property string mustDropItemValue = "" -- Property that determines which item is dropped if the monster must drop an item.

```
@ExecSpace("Client")
method void InitData(table MonsterData, number ItemDropRate, string MustDropItem)
-- Determines whether the received MonsterData is nil.
if MonsterData == nil or next(MonsterData) == nil then
    log("The received table MonsterData is nil!")
    return
end

-- Determines whether the required elements from the received MonsterData have
normal values.
if MonsterData.StartHP == nil or MonsterData.DefaultMissile == nil or
MonsterData.DefaultMissile == "" then
    log("The monster's starting HP and missile type aren't set!")
end
self:FindProjectileManager()

-- Starts reset if the required values are included.
-- Resets the monster's current HP to the max HP upon starting.
self.currentHP = MonsterData.StartHP
-- Resets the monster's starting projectile ID.
self.defaultMissile = MonsterData.DefaultMissile
-- Sets the monster's attack delay. If the loaded value is nil, then resets it to
1.0 second.
self.attackInterval = MonsterData.AttackInterval or 1.0
-- Resets the monster's survival state to true.
self.isAlive = true
-- Resets the item's drop rate.
self.itemDropRate = ItemDropRate
-- Resets the target value if there is an item that must be dropped.
self.mustDropItemValue = MustDropItem
-- Resets the monster's speed.
self.Entity.MovementComponent.InputSpeed = MonsterData.StartMoveSpeed
end

method void ShowMonster()
-- Changes the monster's Enable state to true.
self.Entity.Enable = true
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- Immediately exits the code if the game's state isn't set to Play.
```

```

if not _GameLogic.IsStillPlay then
    self.Entity.MovementComponent.Enable = false
    return
end
-- Starts processing if the monster is currently alive and the game's state is set
to Play.
if self.isAlive and _GameLogic.IsStillPlay then
    -- Activates a monster's MovementComponent if it was previously inactive.
    if self.Entity.MovementComponent.Enable == false then
        self.Entity.MovementComponent.Enable = true
    end
    -- Adds the flow of time equal to delta to the deltaTime.
    self.deltaTime = self.deltaTime + delta
    -- Fires the projectile and resets the deltaTime to 0 if the deltaTime is
larger than the attack delay.
    if self.deltaTime >= self.attackInterval then
        self:FireMissile()
        self.deltaTime = 0
    end
end
end
end

@ExecSpace("Client")
method void FindProjectileManager()
    -- Searches for a MissileManager entity that exists in the InGame map.
    local currentMap = self.Entity.CurrentMapName
    local poolEntity =
_EntityService.GetEntityByPath("/maps/"..currentMap.."/ProjectileManager")
    -- Resets the missileManager to poolEntity if the poolEntity was found.
    if invalid(poolEntity) then
        self.projectileManager = poolEntity.ProjectileManager
        self.isAlive = true
    else
        -- Generates an error log if the ProjectileManager couldn't be found in
InGame.
        log("Error: ProjectileManager couldn't be found in the InGame map.")
    end
end

@ExecSpace("Client")
method void FireMissile()
    -- Halts code execution if there is no projectileManager.
    if self.projectileManager == nil then return end

    -- Determines whether the projectileManager is nil and whether the default
projectile has been registered.
    if self.projectileManager == nil or self.defaultMissile == nil or
self.defaultMissile == "" then
        -- Leaves an error log if either one doesn't meet the conditions.
        log("Failed to reset player MissileData!")
    end

    -- Loads usable defaultMissile entities from ProjectileManager.
    local missileEntity = self.projectileManager:GetMissile(self.defaultMissile)

```

```

-- Executes code when the entity received from projectileManager is usable.
if invalid(missileEntity) then
-- Loads the MissileComponent from the received entity.
local missileComp = missileEntity.MissileComponent
    if missileComp then
        -- Determines the missile's launch direction.
        local fireDir = 1
        -- Since the default resource is facing left, if the entity's FlipX is
inactive, it will be set to fly to the left.
        if self.Entity.SpriteRendererComponent.FlipX == false then
            fireDir = -1
        end
        -- Loads the current monster's position.
        local monsterPos = self.Entity.TransformComponent.Position
        -- Delivers the monster's current position to the projectile.
        local firePos = Vector3(monsterPos.x, monsterPos.y, 0)
        -- Commands the projectile to be fired.
        missileComp:FireMissile(Vector2(fireDir, 0), false, firePos)
    end
end
end

@ExecSpace("Client")
method void SendCalcDropItem()
-- Delivers to ItemHelper, so the item drop state can be calculated when a monster
dies.
_ItemHelper:CalcDropItemData(self.itemDropRate, self.mustDropItemValue,
self.Entity.TransformComponent.Position)
end

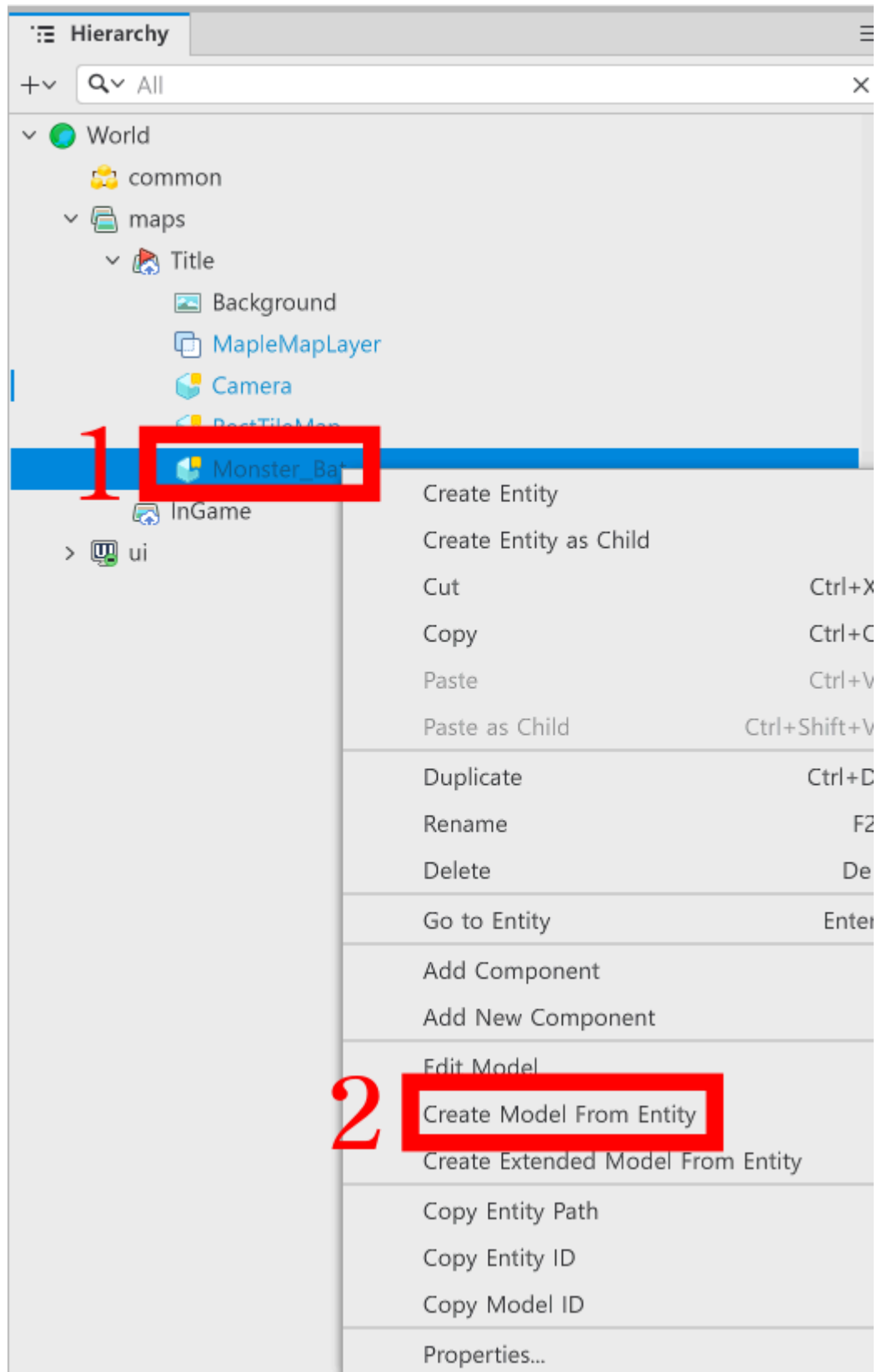
@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleHitEvent(HitEvent event)
-- Code isn't executed when the monster is in the dead state.
if not self.isAlive then
    return
end
-- Loads the collided missile entity.
local attackerEntity = event.AttackerEntity
-- Loads the damage delivered by the missile.
local totalDamage = event.TotalDamage
-- Registers the effect output position to display the hit effect.
local effectSpawnPos = Vector3(attackerEntity.TransformComponent.Position.x,
attackerEntity.TransformComponent.Position.y, 0)
-- Proceeds with damage calculation.
self.currentHP = self.currentHP - totalDamage
-- Displays the hit effect if a hit is registered.
_EffectService:PlayEffect(event.Extra, self.Entity, effectSpawnPos, 0,
Vector3(0.5, 0.5, 0.5))
-- Plays the hit taken sound. A different sound can be played by changing the
first value in the parameter.
_SoundService:PlaySound("5873f227d7604a63997546a92824c135", 1.0)
-- Proceeds with death processing if HP is reduced to 0.

```

```
if self.currentHP <= 0 then
    -- Converts survival state to false.
    self.isAlive = false
    -- Executes if an item is dropped, since a monster died.
    self:SendCalcDropItem()
    -- Displays the effect that plays on death. A different effect can be
    displayed upon death by changing the first value in the parameter.
    _EffectService:PlayEffect("f67f25d5f8e14d2c9962be0fa42808f7", self.Entity,
    effectSpawnPos, 0, Vector3(0.5, 0.5, 0.5))
    -- Checks the condition fulfilment state by delivering the monster defeat
    status to Gamelogic.
    _GameLogic:CountKillScore()
    -- Destroys itself to eliminate an entity.
    _EntityService:Destroy(self.Entity)
end
end

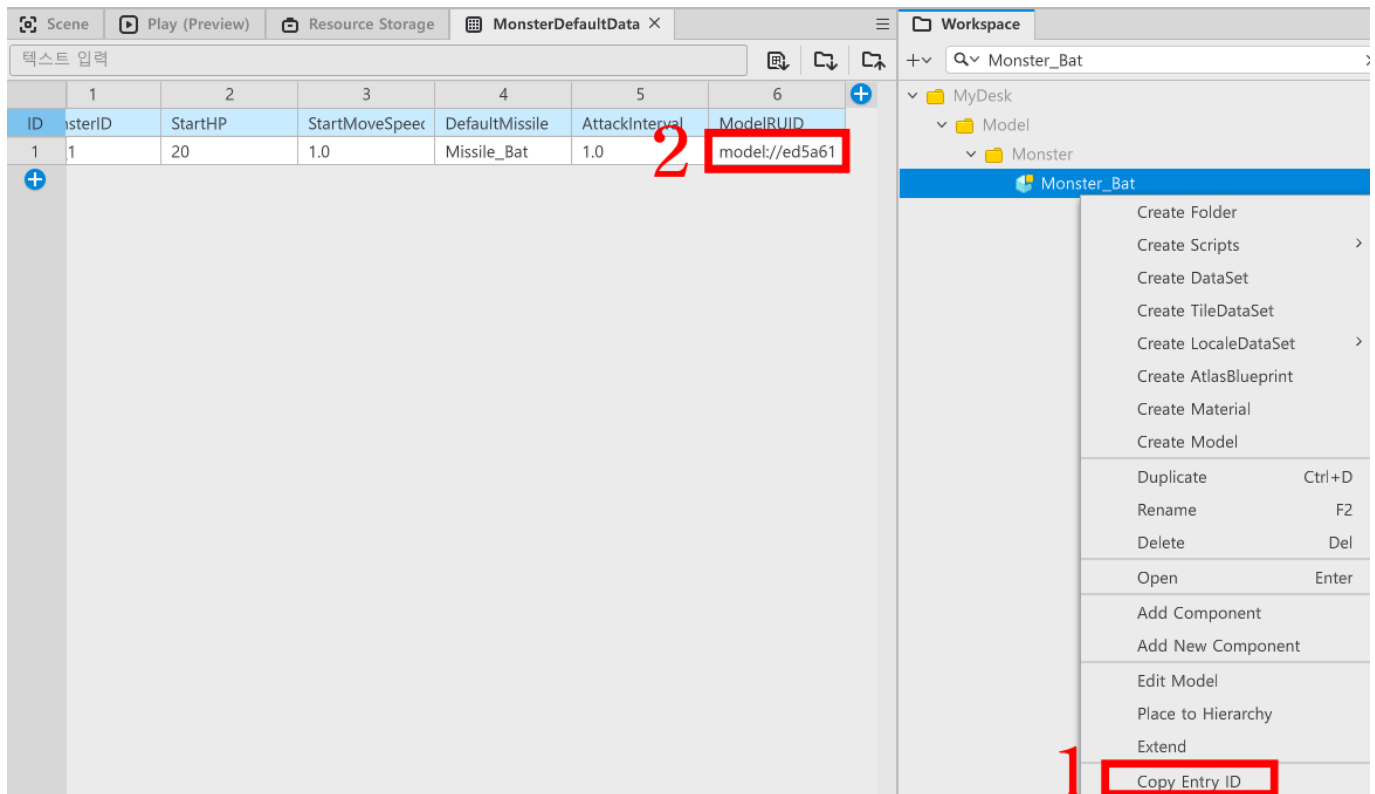
end
```

## Modelization of Created Entities



- Turn a created entity into a Model and add to WorkSpace.
- Right-click the entity that will become a Model and click **Create Model From Entity** to create a model.

## Add Model ID to a DataSet



- Right-click the Model that was created in WorkSpace.
- Click Copy Entry ID to copy the ID value of the target model.
- Adds the target value to the MonsterDefaultData's ModelRUID.

### What is the Role of ModelRUID?

- The ModelRUID's role is to find the Model in WorkSpace and then copy that Model.
- Used to create a MonsterSpawnManager for the advanced training topic of implementing items.
- Learn how to copy Models with the ModelRUID value to summon various Monsters or copy other Models as needed.

### Minimum Implementation Standards

- A Model that composes the Monster must be created.
- The created Monster Model must chase the player.

### Usage and Extension Direction

- Create Monster Models with various forms and values to provide a variety of Monsters.
- Implement Monster movement without the use of AIChase.
- Create monster movement patterns by implementing movement methods for each Monster.

---

## [💡 Basic Training] Creating Basic Attacks

---



## Video Timeline

- You can follow along by using the timeline under the training video.
- Creating Basic Attacks: 01:42:04

## Learning Goals

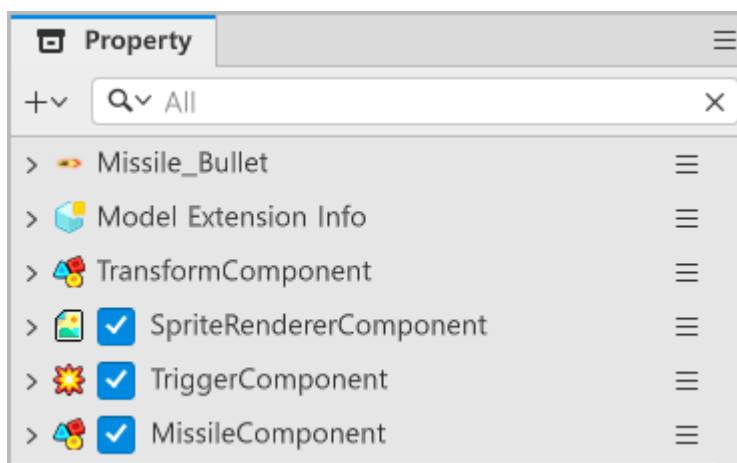
- Use the TriggerComponent to learn about the Collision Group, which sets collision groups.
- Understand the Matrix that determines the collision relationship between Collision Groups and how to set them.

## Practical Exercises to Do After Watching the Video

- Learn about the Matrix for the TriggerComponent's Collision Group.
- Create your own Collision Group and proceed with the practical application of creating relationships using the Matrix.

## Composing Projectile Entities

- Create an empty entity in Hierarchy and compose the Components like the following.



- **SpriteRendererComponent:** A Component used to output the projectile's image. Set and use an image of your choice for the projectile.
- **TriggerComponent:** A Component used to set the projectile's collisions.
- **MissileComponent:** A Component used to control the projectile's data and actions.

## Writing a MissileComponent Script

- Write the following script.
- This script is written based on Plain Text.

```
@Component
script MissileComponent extends Component

property number moveSpeed = 0 -- Property that saves the projectile's speed.

property number atkValue = 0 -- Property that saves the projectile's attack power.
```

```
property number lifeTime = 0 -- Property that saves the projectile's duration.

property boolean isShotPlayer = false -- Property that classifies the projectile's
launching entity.

property string hitEffectRUID = "" -- Property that saves the effect that plays
when the projectile lands.

property Vector2 fireDirection = Vector2(0,0) -- Property that saves the
projectile's firing direction.

property number deltaTimer = 0 -- Property that calculates when the projectile was
created.

property string missileID = "" -- Property that saves the current projectile's ID.

property any poolManager = nil

@ExecSpace("ClientOnly")
method void InitData(table MissileData)
-- Generates an error log and exits the code if the received MissileData is an
unusable value.
if MissileData == nil or next(MissileData) == nil then
    log("MissileData is empty.")
    return
end

-- Saves the ID of the projectile from the received data.
self.missileID = MissileData.MissileID
-- Saves the attack power of the projectile from the received data.
self.atkValue = MissileData.AtkValue
-- Saves the speed of the projectile from the received data.
self.moveSpeed = MissileData.MoveSpeed
-- Saves the duration of the projectile from the received data.
self.lifeTime = MissileData.LifeTime
-- Saves the effect ID that will play when the projectile from the received data
collides with a target.
self.hitEffectRUID = MissileData.HitEffectRUID
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- Immediately exits the code if the entity is unusable.
if self.Entity.Enable == false then
    return
end
-- Immediately exits if the game's play state isn't in-play.
if not _GameLogic.IsStillPlay then
    return
end

-- Fetches the projectile's current position.
local transform = self.Entity.TransformComponent
```

```

-- Calculates the speed and the distance that must be traveled over the duration.
local moveVec = Vector3(self.fireDirection.x, self.fireDirection.y, 0) *
self.moveSpeed * delta
-- Moves the projectile's current position up to the distance it must travel.
transform.Position = transform.Position + moveVec

-- Adds time equal to delta to the deltaTime.
self.deltaTime = self.deltaTime + delta
-- Returns the projectile if the deltaTime is larger than lifeTime.
if self.deltaTime >= self.lifeTime then
    self:ReturnSelf()
end
end

@ExecSpace("ClientOnly")
method void ReturnSelf()
    -- Returns the projectile to ProjectileManager.
    if self.poolManager and isValid(self.poolManager) then
        self.poolManager:ReturnMissile(self.Entity)
    else
        -- Destroys the projectile if the poolManager is nil.
        _EntityService:Destroy(self.Entity)
    end
end

@ExecSpace("Client")
method void FireMissile(Vector2 FireDirection, boolean IsShotPlayer, Vector3
FireStartPos)
    -- Resets the deltaTime to 0.
    self.deltaTime = 0
    -- Resets the projectile's launch redirection.
    self.fireDirection = FireDirection
    -- Resets the projectile's launch direction. If the player is the launching
    entity, it's saved as true.
    self.isShotPlayer = IsShotPlayer
    -- Changes the projectile's current position to the launch position.
    self.Entity.TransformComponent.Position = FireStartPos
    -- Converts the projectile's use state to true.
    self.Entity.Enable = true
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
    -- Fetches the entity that has a detected collision.
    local trigger = event.TriggerBodyEntity
    -- Resets hitSuccess to false to save the collidable status.
    local hitSuccess = false
    -- Proceeds with processing if the player-fired missile collides with the monster.
    if isValid(trigger.MonsterState) and self.isShotPlayer then
        -- Requests a damage calculation if the entity collided has a HitComponent and
        is still alive.
        if trigger.HitComponent and trigger.MonsterState.isAlive then
            trigger.HitComponent:OnHit(self.Entity, self.atkValue, false,

```

```

self.hitEffectRUID, 1)
    hitSuccess = true
end

-- Proceeds with processing if the player is hit by a missile fired by a monster.
elseif invalid(trigger.PlayerState) and self.isShotPlayer == false then
    -- Determines whether the player has a HitComponent, whether it's alive, and
    whether it's currently invulnerable.
    if trigger.HitComponent and trigger.PlayerState.isAlive and not
trigger.PlayerState.isInvincibility then
        -- Calls OnHit and requests damage calculation if damage can be taken.
        trigger.HitComponent:OnHit(self.Entity, self.atkValue, false,
self.hitEffectRUID, 1)
        hitSuccess = true
    end
end

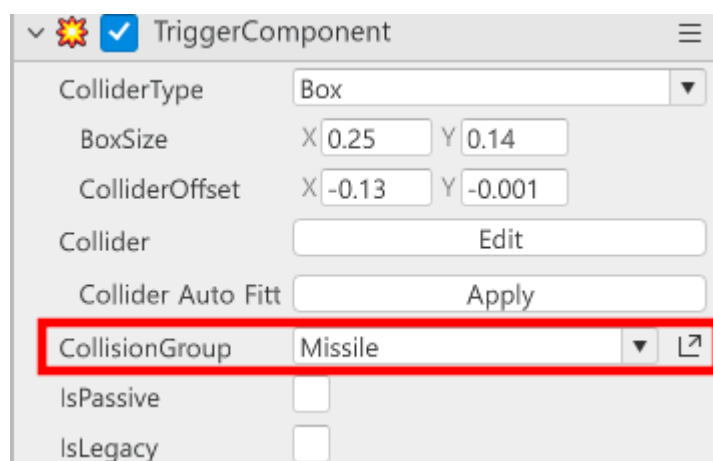
-- Deactivates the entity again if the damage calculation took place.
if hitSuccess then
    self:ReturnSelf()
end
end

end

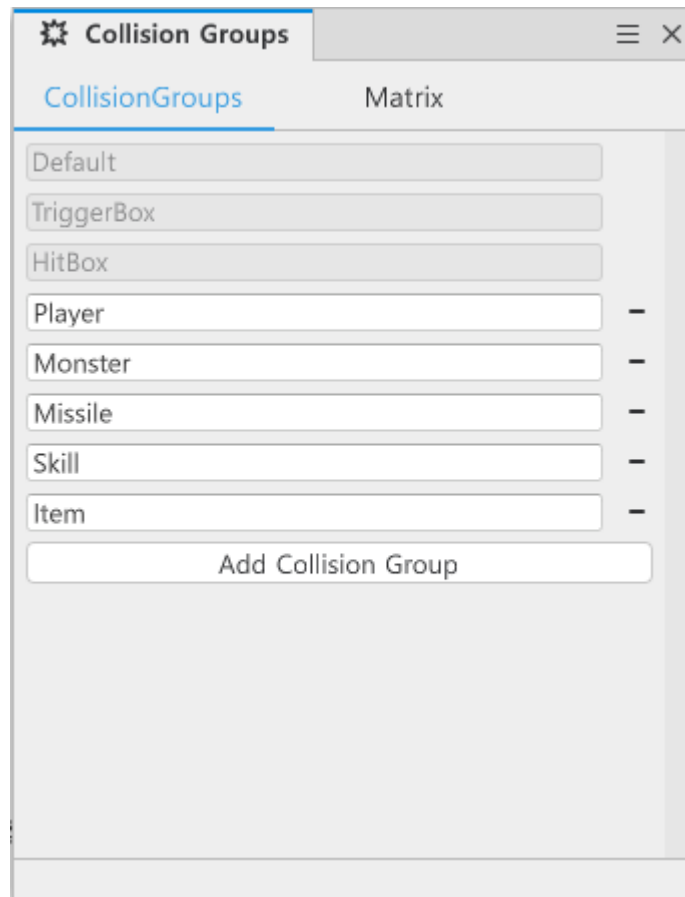
```

## Setting a TriggerComponent

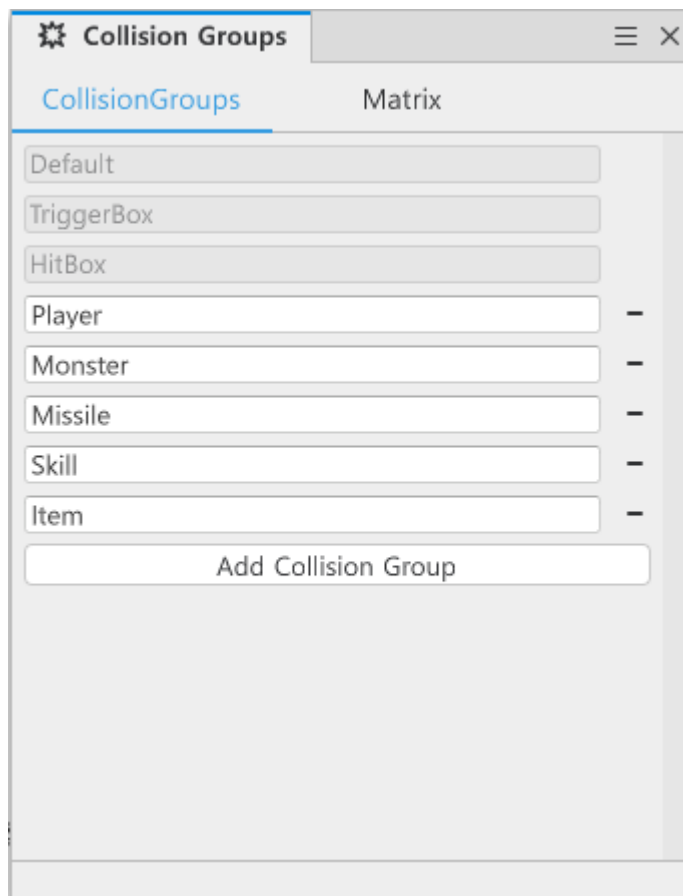
- Change the components of a TriggerComponent to change the collision relationship between Collision Groups.



- Find the TriggerComponent of the created entity and click the Open Panel button on the right of CollisionGroup.



- Click Add Collision Group from the Collision Groups window and add CollisionGroup.
  - **Player:** A Collision Group used to designate the player.
  - **Monster:** A Collision Group used to designate enemies.
  - **Missile:** A Collision Group used to designate projectiles.
  - **Skill:** A Collision Group used to designate skills.
  - **Item:** A Collision Group used to give benefits to the player.
- Complete the settings above and move to Matrix.



- Set the Matrix like the above image.
- This Matrix sets the following collision relationship.
  - Player checks the collisions with Monster, Missile, and Item.
  - Monster checks the collisions with Player, Missile, and Skill.
  - Missile checks the collisions with Player, Monster, and Skill.
  - Skill checks the collisions with Monster and Missile.
  - Item checks the collisions with Player.

## Why a Matrix Should Be Set Up

- Collision calculations are one of the more frequent elements that are checked.
- Adding unnecessary CollisionGroups and checking the collisions of all CollisionGroups will increase the amount of unnecessary calculations.
- To optimize performance, you need re-remove unnecessary collision checks and only check those required for higher efficiency.

## Minimum Implementation Standards

Create a projectile Model to have at least 3 types of usable projectiles.

- The projectile data must be input normally through the projectile's InitData.
- When firing a projectile with a value that was input with InitData using FireMissile, the projectile must be fired normally.
- The projectile must disappear after colliding with the Monster.

## Usage and Extension Direction

- Create a pattern by adding various movements to a projectile.
- Create a new character using various projectiles.
- Create patterns by differentiating the projectiles fired by each enemy.

---

## [💡 Basic Training] Creating an Object Pool

---

### Video Timeline

- You can follow along by using the timeline under the training video.
- Creating an Object Pool: 02:02:03

### Learning Goals

- Learn about the principle behind an object pool.
- Manage and use projectiles using the object pool.
- Learn further optimization methods by managing memory usage through the object pool.

### Practical Exercises to Do After Watching the Video

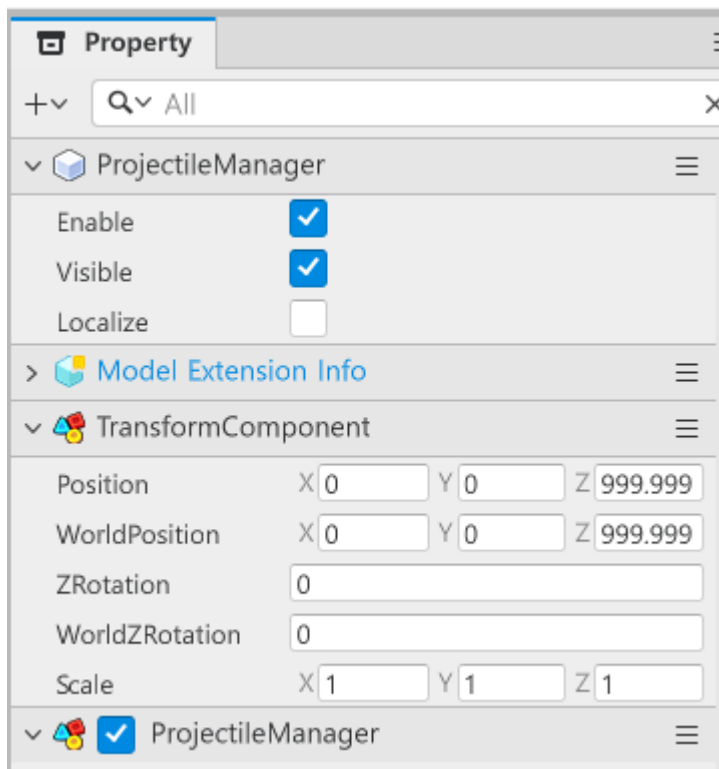
- See for yourself the how the principles of an object pool functions and the results of the execution.
- Think about which elements can be utilized with object pools using this method.
- Design a structure that can manage those elements using the object pool method.

### What is an Object Pool?

- Many entities exist when executing the game.
- Of these entities, some units like monsters may die and disappear.
- If there are a high number of units, creating and eliminating all of those units may overload the computer's calculations.
- However, if the activation is removed instead of creating and eliminating entities, it can lower the load on your computer.
- This is because it only needs to be set as On or Off like a switch.
- The object pool can make usable entities in advance and deliver them to minimize the need to create and eliminate unnecessary entities.

### Creating an Object Pool

- Entities must be created in the Map to create an object pool.
- The ProjectileManager is used as an object pool that manages projectiles in the InGame map for the sample world.
- The object pool is made up of the following Components.



- **TransformComponent:** A Component that is provided by default. However the Position value must be set to **0** so that it can be created in the correct position.
- **ProjectileManager:** A Component that was added to create an object pool. This Component must be created directly through New Component.

## Setting a ProjectileManager Component

- Write the following code.
- The target code is based on Plain Text.

```
@Component
script ProjectileManager extends Component

property table projectilePool = {} -- A table that contains the projectile entity.

@ExecSpace("ClientOnly")
method Entity GetMissile(string MissileKey)
-- Attempts a reset if there are no elements that use MissileKey as a Key value.
if self.projectilePool[MissileKey] == nil then
    self.projectilePool[MissileKey] = {}
end

-- Registers elements that use MissileKey from projectilePool as a Key to the
Pool.
local pool = self.projectilePool[MissileKey]
-- Registers the entity to nil to register a usable entity.
local entity = nil

-- Starts a navigation if there are no elements from the pool.
if #pool > 0 then
    -- Navigates elements in the pool.
```



```

    for i = 1, #pool do
        -- Returns the target entity if the index i entity exists and if the
        current use state is false.
        if invalid(pool[i]) and not pool[i].Enable then
            entity = pool[i]
            break
        end
    end
end

-- Attempts creation, since if the entity is still nil, the usable entity doesn't
exist.
if entity == nil then
    -- Requests projectile data from the DataTableLogic based on the MissieKey
    value.
    local missileData = _DataTableLogic:GetMissileDataByMissileID(MissileKey)
    -- Creates a projectile through SpawnService.
    entity = _SpawnService:SpawnByModelId(missileData.ModelRUID,
    missileData.MissileID, Vector3.zero, self.Entity)
    -- Registers projectile data to the created projectile.
    entity.MissileComponent:InitData(missileData)
    -- Registers itself to the projectile's poolManager.
    entity.MissileComponent.poolManager = self
    -- Changes the projectile usage state to true.
    entity.Enable = true
    -- Registers the entity created to projectilePool.
    table.insert(self.projectilePool[MissileKey], entity)
else
    -- Changes the found entity's usage state to true, since there is a usable
    entity.
    entity.Enable = true
end

-- Returns the found or created entity.
return entity
end

@ExecSpace("ClientOnly")
method void ReturnMissile(Entity missile)
    -- Immediately exits the code if the received entity is unusable.
    if not invalid(missile) then return end
    -- Returns the received entity's usable state as false.
    missile.Enable = false
end

@ExecSpace("Client")
method void ResetPool()
    -- Searches for a child entity based on its own entity.
    local childList = self.Entity.Children
    -- Returns if there is no child entity.
    if childList == nil then return end
    -- Iterates the found child entity list and changes the entity's Enable to false.
    for i, child in pairs(childList) do
        if invalid(child) then
            child.Enable = false
        end
    end
end

```

```

        end
    end
end

@ExecSpace("Client")
method Entity GetProjectileSkill(string SkillKey)
    -- Attempts a reset if there is no Value with SkillKey from projectilePool.
    if self.projectilePool[SkillKey] == nil then
        self.projectilePool[SkillKey] = {}
    end

    -- Registers elements that have SkillKey as a key to the pool.
    local pool = self.projectilePool[SkillKey]
    -- Registers the entity to nil for returns.
    local entity = nil

    -- Starts a navigation if there are no elements from the pool.
    if #pool > 0 then
        -- Navigates elements in the pool.
        for i = 1, #pool do
            -- Returns the target entity if the index i entity exists and if the
            current use state is false.
            if isValid(pool[i]) and not pool[i].Enable then
                entity = pool[i]
                break
            end
        end
    end

    -- Newly creates an entity if there is no usable skill projectile entity in the
    pool.
    if entity == nil then
        -- Requests data from DataTableLogic based on SkillKey.
        local skillData = _DataTableLogic:GetSkillDataBySkillID(SkillKey)
        -- Requests a projectile skill model be created through SpawnService.
        entity = _SpawnService:SpawnByModelId(skillData.ModelRUID, skillData.SkillID,
        Vector3.zero, self.Entity)
        -- Requests projectile skill data entry.
        entity.ProjectileSkillComponent:InitData(skillData)
        -- Registers the entity created to projectilePool.
        table.insert(self.projectilePool[SkillKey], entity)
    end

    -- Changes the usage state of a entity that will be returned to true.
    entity.Enable = true
    -- Returns a found or created entity.
    return entity
end

end

```

- The previously created PlayerState and Monster State refer to the ProjectileManager.

- The PlayerState is searched with ConnectMissilePool, and the MonsterState is searched with FindProjectileManager.
- Through this, the Player and Monster can fire projectiles with the object pool.

## Minimum Implementation Standards

- Implements the function so that projectile entities aren't repeatedly created and destroyed by using the object pool.
- Neither the player nor Monster makes projectile creation requests.
- All projectiles are implemented in the form of creating or returning usable entities from the object pool.

## Usage and Extension Direction

- Edit the code so that entities are **created in advance** and used as one of the principles of an object pool.
- Check if the projectiles are usable as intended based on the edited code.
- Implement the function so that the object pool only created **necessary entities** in advance.

---

## [📖 Advanced Training] Implementing Skills

---

### Video Timeline

- You can follow along by using the timeline under the training video.
- Creating Skills: 02:11:47

### Learning Goals

- Learn about TriggerEvent types.
- Understand the trigger conditions for each event and create the functions required.
- Understand what the risks are when using the Stay TriggerEvent and which situations require the use of that Event.

### Practical Exercises to Do After Watching the Video

- Create a skill Model.
- Change a projectile-type skill provided in the video into a different type.
- Adjust SkillData to adjust and change the skill's functionality.
- Create one more projectile-type skill and change the player character's skill.

### Creating a DataSet

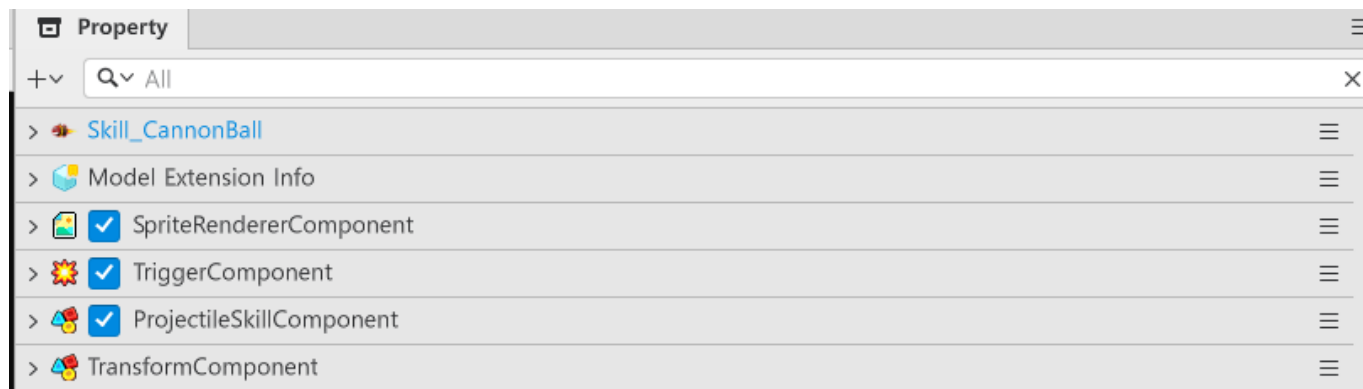
- Skills also need a DataSet to manage their data.
- The sample world has the following created DataSet.

|        |                 |                  |                      |             |           |          |             |             |                |           |
|--------|-----------------|------------------|----------------------|-------------|-----------|----------|-------------|-------------|----------------|-----------|
| Scene  | Play (Preview)  | Resource Storage | MonsterDefaultData X | SkillData X |           |          |             |             |                |           |
| 텍스트 입력 |                 |                  |                      |             |           |          |             |             |                |           |
|        | 1               | 2                | 3                    | 4           | 5         | 6        | 7           | 8           | 9              | 10        |
| ID     | SkillID         | SkillType        | IsPierce             | AtkValue    | MoveSpeed | LifeTime | MaxHitCount | HitInterval | ModelRUID      | HitEffect |
| 1      | Skill_CannonBal | Projectile       | true                 | 20          | 1.0       | 5.0      | 5           | 0.2         | model://a309b2 | ee451f0e  |

| Name          | Description                                                                                                                                                                                             |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SkillID       | The ID value needed to classify skills. The player's skills can be designated based on this value.                                                                                                      |
| SkillType     | The value needed to classify skill types. The sample world only created the projectile type, but the value was created with the possibility of other skill types being made in the future.              |
| IsPierce      | The value that determines the penetrative status of a projectile skill. This value was created so that the projectile skill disappears even after it collides with 1 enemy, even if the value is false. |
| AtkValue      | The value that determines the skill's attack power.                                                                                                                                                     |
| MoveSpeed     | The value that determines the projectile skill's speed.                                                                                                                                                 |
| LifeTime      | The value that determines the projectile skill's duration.                                                                                                                                              |
| MaxHitCount   | The value that determines the number of hits for multi-hit-type skills.                                                                                                                                 |
| HitInterval   | The value that determines the amount of time until the subsequent hit is triggered for multi-hit-type skills.                                                                                           |
| ModelRUID     | The value used to register a skill Model's Entry ID.                                                                                                                                                    |
| HitEffectRUID | The value used to register the RUID of the effect that is displayed when hit by the skill.                                                                                                              |

## Composing Skill Entities

- Entities must be composed to create a skill Model.
- Compose the following Components for the skill Entity.



- **SpriteRendererComponent:** A Component used to output the skill's image.
- **TriggerComponent:** A Component used to check the skill's collision state. The CollisionGroup of the target Component must be set to Skill.
- **ProjectileSkillComponent:** A Component that controls the projectile skill's data and actions. Must be created through Add Component.

## Composing Scripts

- The **ProjectileSkillComponent** must be composed as follows.
- The following code is written based on Plain Text.

```
@Component
script ProjectileSkillComponent extends Component

property table collideTable = {} -- A table that manages entities that are
colliding with skill projectiles.

property number atkValue = 0 -- A Property that saves the skill projectile's
attack power.

property number hitInterval = 0 -- A Property that adjusts the projectile skill's
multi-hit interval.

property number maxHitCount = 0 -- A Property that adjusts the projectile skill's
max hit interval.

property boolean isPierce = false -- A Property that determines the projectile
skill's penetration state.

property number lifeTime = 0 -- A Property that determines a projectile skill's
duration.

property number moveSpeed = 0 -- A Property that determines the projectile skill's
speed.

property string hitEffectRUID = "" -- A Property that saves the effect ID that
will be displayed when hit by a projectile skill.

property number deltaTimer = 0 -- A Property needed for calculating the projectile
skill's duration.
```

```
property Vector2 fireDirection = Vector2(0,0) -- A Property that saves which
direction the projectile should travel in.
```

```
@ExecSpace("Client")
method void InitData(table SkillData)
-- Determines whether the received skill data is valid.
if not isValid(SkillData) or next(SkillData) == nil then
    log("The received SkillData is nil!")
    return
end
-- Resets skill data.
-- Resets the deltaTime that checks the projectile skill's lifespan.
self.deltaTime = 0
-- Resets the table that contains the collided monsters.
self.collideTable = {}
-- Resets the skill's attack power value.
self.atkValue = SkillData.AtkValue
-- Resets the skill's attack delay value.
self.hitInterval = SkillData.HitInterval
-- Resets the skill's max attack count.
self.maxHitCount = SkillData.MaxHitCount
-- Resets the skill's duration.
self.lifeTime = SkillData.LifeTime
-- Resets the skill's speed.
self.moveSpeed = SkillData.MoveSpeed
-- Determines whether the skill can penetrate the target.
self.isPierce = SkillData.IsPierce
-- Resets the RUID of the effect that will be displayed when hit by the skill.
self.hitEffectRUID = SkillData.HitEffectRUID
-- Deactivates the projectile skill's usage first.
self.Entity.Enable = false
end
```

```
@ExecSpace("Client")
method table GetOrCreateCollideEntity(Entity CollideEntity)
-- Executes the following code if the CollideEntity is usable.
if isValid(CollideEntity) then
    -- Loads the CodeEntity's ID.
    local entityID = CollideEntity.Id
    -- Newly registers to collideTable if there are no entities that have ID
    values in the collideTable.
    if not isValid(self.collideTable[entityID]) or not
next(self.collideTable[entityID]) then
        self.collideTable[entityID] = {
            Target = CollideEntity,
            HitCount = 0,
            LastHitTime = nil
        }
    end
    -- Returns entities that have ID from CollideEntity.
    return self.collideTable[entityID]
end
end
```

```

@ExecSpace("Client")
method void ApplyDamage(table CheckData)
-- Changes the target ID's Table Value to nil if there are no entities due to
being already dead.
if not isvalid(CheckData.Target) then
    self.collideTable[CheckData.Target.Id] = nil
    return
end

-- Proceeds with damage calculation to the component after checking the target's
MonsterState component status.
if isvalid(CheckData.Target.MonsterState) then
    CheckData.Target.HitComponent.OnHit(self.Entity, self.atkValue, false,
self.hitEffectRUID, 1)
end

-- Registers the last hit time.
CheckData.LastHitTime = _UtilLogic.ElapsedSeconds
-- Increases the damage count that the target entity can take by 1.
CheckData.HitCount = CheckData.HitCount + 1
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
- Halts the following code if the game's Play state is false.
if not _GameLogic.IsStillPlay then
    return
end
-- Directs the projectile skill to move.
self:Move(delta)
-- Requests the calculation of the projectile's duration.
self:RunLifeTimer(delta)

end

@ExecSpace("Client")
method void RunLifeTimer(number DeltaTime)
-- Increases the projectile skill's deltaTime by DeltaTime.
self.deltaTime = self.deltaTime + DeltaTime
-- Deactivates usage if the deltaTime goes over lifeTime.
if self.deltaTime >= self.lifeTime then
    self.Entity.Enable = false
end

end

@ExecSpace("Client")
method void Move(number DeltaTime)
-- Loads the entity's current position.
local transform = self.Entity.TransformComponent
-- Calculates the projectile skill's position by adding the speed and DeltaTime.
local moveVec = Vector3(self.fireDirection.x, self.fireDirection.y, 0) *
self.moveSpeed * DeltaTime

```

```

-- Changes the projectile skill's position.
transform.Position = transform.Position + moveVec
end

@ExecSpace("Client")
method void FireSkill(Vector2 FireDirection, Vector3 FireStartPos)
-- Resets the deltaTime value that's needed to calculate a projectile skill's
duration to 0.
self.deltaTime = 0
-- Resets the projectile skill's launch direction.
self.fireDirection = FireDirection
-- Resets the projectile skill's launch position.
local transform = self.Entity.TransformComponent
if transform then
    transform.Position = FireStartPos
end
-- Converts the projectile's usage to true.
self.Entity.Enable = true
end

@EventSender("Self")
handler HandleTriggerStayEvent(TriggerStayEvent event)
-- Determines whether the colliding Entity exists normally.
if invalid(event.TriggerBodyEntity) then
    -- Deletes the enemy projectile if the colliding Entity is a projectile.
    if invalid(event.TriggerBodyEntity.MissileComponent) and not
event.TriggerBodyEntity.MissileComponent.isShotPlayer then
        event.TriggerBodyEntity.MissileComponent:ReturnSelf()
        return
    end
    -- Proceeds with damage calculation as long as the hit count is satisfied when
the colliding entity is an enemy.
    if invalid(event.TriggerBodyEntity.MonsterState) then
        -- Returns an entity that is the same as the colliding entity in the
collideTable.
        local collideTable =
self:GetOrCreateCollideEntity(event.TriggerBodyEntity)
        -- Halts code execution if the returned entity is unusable.
        if not invalid(collideTable) then
            return
        end
        -- Halts code execution if the HitCount goes over the max amount.
        if collideTable.HitCount >= self.maxHitCount then
            return
        end
        -- Loads the current time.
        local currentTime = _UtilLogic.ElapsedSeconds
        -- Immediately applies the damage if damage calculation has never taken
place.
        if collideTable.LastHitTime == nil then
            self:ApplyDamage(collideTable)
            -- Attempts damage calculation if the time when the damage was taken is
larger than the multi-hit time.
            elseif currentTime - collideTable.LastHitTime >= self.hitInterval then

```



```

        self:ApplyDamage(collideTable)
    end
end
end
end

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- Generates an error log if the event is abnormal.
if not isvalid(event) then
    log("The event is an abnormal event!")
    return
end
-- Deactivates the collided entity if the collided entity has a MissileComponent
and if the launching entity isn't the player.
if isvalid(event.TriggerBodyEntity.MissileComponent) and not
event.TriggerBodyEntity.MissileComponent.isShotPlayer then
    event.TriggerBodyEntity.MissileComponent:ReturnSelf()
    return
end
-- Executes the following code if the collided entity has a MonsterState.
if isvalid(event.TriggerBodyEntity.MonsterState) then
    -- Determines whether there is an entity that is the same as the collided
entity in the collideTable.
    local findList = self:GetOrCreateCollideEntity(event.TriggerBodyEntity)

    -- Halts code execution if received damage count of the collided entity goes
over the max value.
    if findList.HitCount >= self.maxHitCount then return end

    -- Immediately requests damage processing from the collided entity.
    self:ApplyDamage(findList)
end
end

@EventSender("Self")
handler HandleTriggerLeaveEvent(TriggerLeaveEvent event)
-- Halts code execution if an entity that is outside the collision range is in an
unusable state.
if not isvalid(event) then
    return
end
-- Determines whether an entity that is outside the collision range is in the
collideTable.
local record = self.collideTable[event.TriggerBodyEntity.Id]
-- For entities that are in collideTable, resets the last hit time and immediately
processes damage when it re-enters collision range.
if record then
    record.LastHitTime = nil
end
end
end
end

```

- If this work is all done, turn the skill entity into a Model and register the Model's EntryID to the SkillData's ModelRUID.

## Call Timing for the HandleTrigger Event

- **EnterEvent**: Called the moment when 2 or more entities with TriggerComponents have overlapping collision areas.
- **StayEvent**: Called for each frame the collision areas of 2 or more entities with TriggerComponents remain overlapped.
- **LeaveEvent**: Called the moment when 2 or more entities with TriggerComponents with overlapping collision areas separate.

## Minimum Implementation Standards

- The player can use a skill entity.
- A skill entity can remove an enemy's projectile.
- The projectile skill attacks multiple enemies and can attack enemies based on a set number of times.

## Usage and Extension Direction

- You can create skill types other than projectile types.
- You can create a projectile skill that fires with varied movement, not just one that travels in a straight line.

---

## [📖 Advanced Training] Implementing Items

---

### Video Timeline

- You can learn from the timeline under the training video.
- Implementing Items: 02:32:10

### Learning Goals

- Learn how to use Logic that will control certain functions.
- If you want to change the function of an item in the future, learn how to design with extensions in mind, so only the Logic can be changed.

### Practical Exercises to Do After Watching the Video

- Change the effect stat of an item and confirm whether the stat change has been applied.
- Create a Model that provides an effect different from the created item.

### Creating a DataSet

- You need 2 types of DataSets when creating a function related to item drops.
- There are the ItemData and ItemDropData sets.
  - ItemData**: The DataSet that determines an item's effect or its efficacy.
  - ItemDropData**: The DataSet that determines an item's chance of dropping.

## ItemDropData DataSet Composition

|    | 1                | 2        | + |
|----|------------------|----------|---|
| ID | ItemID           | DropRate |   |
| 1  | Item_LifeRecover | 20       |   |
| 2  | Item_SkillCount  | 20       |   |
| 3  | Item_MoveSpeed   | 20       |   |
| 4  | Item_AttackInte  | 20       |   |
| 5  | Item_ChangeMi    | 20       |   |
| +  |                  |          |   |

| Name     | Description                                                               |
|----------|---------------------------------------------------------------------------|
| ItemID   | The ID value that identifies an item. It uses the ID value from ItemData. |
| DropRate | The value that determines an item's drop chance.                          |

## ItemDropData DataSet Composition

|    | 1                | 2         | 3                | 4           | 5               | 6              | 7              | + |
|----|------------------|-----------|------------------|-------------|-----------------|----------------|----------------|---|
| ID | ItemID           | ValueType | TargetPlayerStat | NumberValue | StringValue     | SpriteRUID     | GainEffect     |   |
| 1  | Item_LifeRecover | 1         | currentHP        | 1           |                 | a2cfdb8831124f | 32317a7b70484  |   |
| 2  | Item_SkillCount  | 1         | canUseSkillCour  | 1           |                 | 04640b1d108d4  | 196d4ef7652f44 |   |
| 3  | Item_MoveSpeed   | 1         | moveSpeed        | 0.05        |                 | e223bdc2a6cd4  | 6802f1a489f94b |   |
| 4  | Item_AttackInte  | 1         | attackInterval   | -0.05       |                 | fafd61650bf146 | 3db73bd6398e4  |   |
| 5  | Item_ChangeMi    | 2         | currentMissile   |             | Missile_Missile | fd8586b0c4c94c | 0915b72bf4374  |   |
| +  |                  |           |                  |             |                 |                |                |   |

| Name                  | Description                                                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| ItemID                | The ID value that identified an item. Uses this value as an ItemID from the ItemDropData.                                                         |
| ValueType             | An element that identifies whether an item's effect value is a number or text type value.                                                         |
| TargetPlayerStateName | The value that determines which Property from the PlayerState should be affected. It must have the same Property name as seen in the PlayerState. |
| NumberValue           | For items that have number type values, the effect is applied based on that value.                                                                |

| Name        | Description                                                                                                                                                                              |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| StringValue | For items that have text type values, the effect is applied based on that value. It was created with the goal of implementing an item that changes the basic attack in the sample world. |
| SpriteRUID  | The value created to change the item's image. The item's image is changed based on this value.                                                                                           |
| GainEffect  | The value used to set the item's acquisition effect upon obtaining the item.                                                                                                             |

## Creating an ItemSpawnManager

- The Component that's in charge of actual item creation.
- This element is created in the form of an entity, and it is created in InGame in the sample world.
- Create an entity and Component and register the following code to the ItemSpawnManager.
- The code is written based on Plain Text.

```
@Component
script ItemSpawnManager extends Component

property string itemModelID = "model://0b6340f2-19d9-4a27-b18a-70feb94ce2ae" --
The Model ID for use as an item within Workspace.

property table itemObjectPool = {} -- Table for searching and re-using usable
entities.

@ExecSpace("Client")
method void CreateItem(string ItemKey, Vector3 ItemSpawnPosition)
-- Determines whether the itemObjectPool is nil.
if self.itemObjectPool[ItemKey] == nil then
-- Resets to {} if the itemObjectPool is nil.
self.itemObjectPool[ItemKey] = {}
end

-- Registers ItemKey entities received from within the pool to the pool.
local pool = self.itemObjectPool[ItemKey]
-- Sets returnEntity to nil to register usable entities.
local returnEntity = nil

-- Starts the navigation if there is at least 1 pool element.
if #pool > 0 then
-- Starts searches equal to the number of pool elements.
for i = 1, #pool do
-- Returns the entity at index i if the index i entity is in a usable
state.
if invalid(pool[i]) and not pool[i].Enable then
returnEntity = pool[i]
break
end
end
```

```

    end
end

-- Searches for values based on the ItemKey value from DataTableLogic.
local itemData = _DataTableLogic:GetItemDataByItemID(ItemKey)
-- Determines that there are no usable entities if the returnEntity has been nil
so far.
if returnEntity == nil then
    -- Creates an item entity that was created with Model on the map through
    SpawnService.
    returnEntity = _SpawnService:SpawnByModelId(self.itemModelID, itemData.ItemID,
    Vector3.zero, self.Entity)
    -- Registers itself to the ItemComponent's poolManager, so the created item
    can be returned again.
    returnEntity.ItemComponent.poolManager = self
    -- Changes the entity's Enable to true just in case the item is in an unusable
    state.
    returnEntity.Enable = true
    -- Adds the element created in itemObjectPool ItemKey value to Key and entity
    to Value.
    table.insert(self.itemObjectPool[ItemKey], returnEntity)
else
    -- Changes Enable to true if there is a returnEntity, so it can be used.
    returnEntity.Enable = true
end
-- Resets the created item entity's data.
returnEntity.ItemComponent:InitData(itemData, ItemSpawnPosition)
end

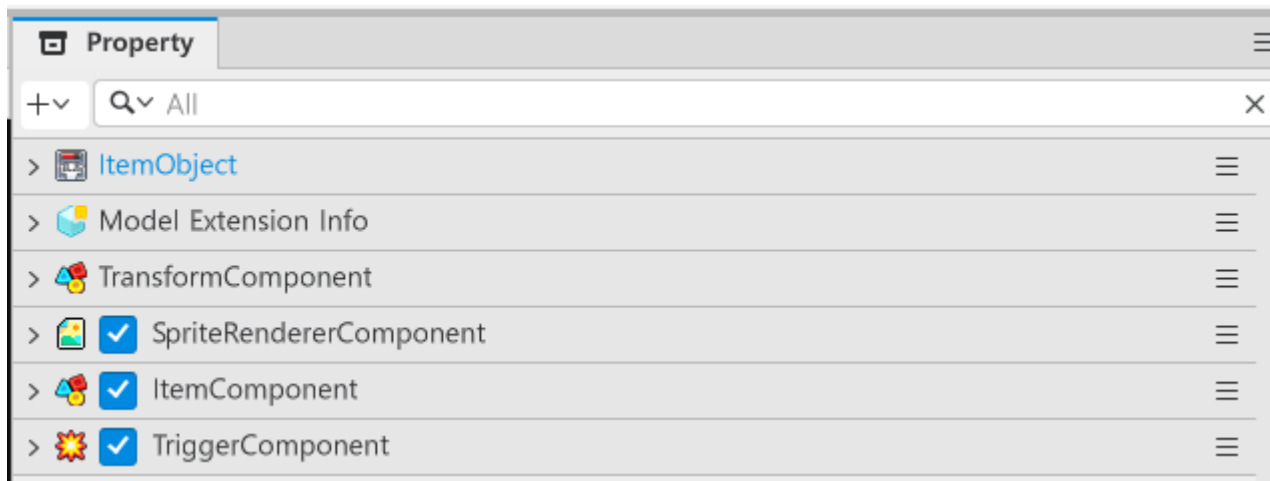
@ExecSpace("Client")
method void ReturnItemEntity(Entity ItemEntity)
-- Exits code through return if the received itemEntity is an unusable entity.
if not isValid(ItemEntity) then return end
-- Deactivates Enable and converts to standby mode, so it can be used if the
ItemEntity can be used.
ItemEntity.Enable = false
end

end

```

## Creating Item Entities

- Create an entity that composes an item and register that entity as Model. The EntryID for the Model used here is used as the itemModelID for the ItemSpawnManager.



- **SpriteRendererComponent:** The Component used to output the item's image. Change the item's icon by applying the SpriteRUID value from ItemData as the Component's ImageRUID.
- **TriggerComponent:** The Component used to detect item collisions. Add this Component and change the Collision Group to Item.
- **ItemComponent:** The Component that contains the item's movement and effect. This Component must be created through Add Component or New Component.

## Writing an ItemComponent Script

- Write the following code to ItemComponent.
- The code is written based on Plain Text.

```
@Component
script ItemComponent extends Component

property SpriteRendererComponent imageSpriteRenderer = nil

property number itemID = 0 -- Property that saves the ItemData's ID value.

property number valueType = 0 -- Property that classifies the ItemData's
application type.

property string targetPlayerStateName = "" -- Property that holds the name of the
Property that needs to be applied in the PlayerState component.

property number numberValue = 0 -- Property that holds the number values when
number format values must be applied.

property string stringValue = "" -- Property that holds text values when text
format values must be applied.

property number moveSpeed = 3.0 -- Property that saves the item's movement speed
in the map.

property number lifeTime = 3.0 -- Property that holds the amount of time the item
can stay on the map.
```

```

property string gainEffect = "" -- Property that holds the effect ID that's
generated upon obtaining the item.

property any poolManager = nil

property number deltaTime = 0 -- Property that calculates an item's duration.

@ExecSpace("Client")
method void InitData(table ItemData, Vector3 SpawnPosition)
-- Starts a reset if the Property that manages the item's image sprite is nil.
if not invalid(self.imageSpriteRenderer) then
-- Searches for a SpriteRendererComponent from an entity that has the
component attached to it.
local findImageSpriteRenderer = self.Entity.SpriteRendererComponent
if not invalid(findImageSpriteRenderer) then
-- Generates a log that says the search failed if the
SpriteRendererComponent wasn't found.
log("SpriteRendererComponent doesn't exist in the item model!")
return
end
-- If the SpriteRenderer is found, the found component is registered to
Property.
self.imageSpriteRenderer = findImageSpriteRenderer
end

-- Returns an error log when the received ItemData is nil and does not attempt a
reset.
if not invalid(ItemData) or not next(ItemData) then
log("The ItemData received from Init is nil!")
return
end

-- Resets the deltaTime to 0 and calculates the duration.
self.deltaTime = 0
-- Registers the received ItemData data to the Component's Property.
-- Registers the item icon image RUID.
self.imageSpriteRenderer.SpriteRUID = ItemData.SpriteRUID
-- Registers the Value that determines the item's influence.
self.valueType = ItemData.ValueType
-- Registers values with names such as the PlayerState's Property.
self.targetPlayerStateName = ItemData.TargetPlayerStateName
-- Registers the increased amount for items that have changes in number-type
values.
self.numberValue = ItemData.NumberValue
-- Registers the String value for items that have changes in the String type Key
values, such as default missiles.
self.stringValue = ItemData.StringValue
-- Registers the ID value of the effect that will be generated upon obtaining an
item.
self.gainEffect = ItemData.GainEffect
-- Receives the location where the monster died and spawns at that location.
self.Entity.TransformComponent.Position = SpawnPosition
-- If the entity is in an inactive state, activates it.

```

```

if not self.Entity.Enable then
    self.Entity.Enable = true
end
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- If the entity is in an inactive state, the script below isn't executed through
the Return immediately.
if not self.Entity.Enable then
    return
end

-- If the game's play state isn't in-play, the script below isn't executed through
the Return immediately.
if not _GameLogic.IsStillPlay then
    return
end

-- Deactivates an item if the deltaTime goes over the item's survival duration.
if self.deltaTime > self.lifeTime then
    self:ReturnSelf()
    return
else
    -- Increases the deltaTime equal to the amount elapsed if the deltaTime
hasn't gone over the lifeTime.
    self.deltaTime = self.deltaTime + delta
end

-- Calls the item's movement function every frame.
self:ItemMove(delta)
end

@ExecSpace("Client")
method void ItemMove(number DeltaTime)
-- Loads the TransformComponent that manages the item's position.
local transform = self.Entity.TransformComponent

-- Calculates the distance that the item must travel.
local moveVec = Vector3(-1, 0, 0) * self.moveSpeed * DeltaTime

-- Moves the item the distance it must travel.
transform.Position = transform.Position + moveVec
end

@ExecSpace("Client")
method void ReturnSelf()
-- Changes the item's Enable to false when the LifeTime is over.
if self.poolManager and invalid(self.poolManager) then
    self.poolManager:ReturnItemEntity(self.Entity)
else
    -- Destroys the item entity if there is no poolManager.
    _EntityService:Destroy(self.Entity)
end
end
end

```



```

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- Processes item use if the collided entity has a PlayerState.
if invalid(event.TriggerBodyEntity.PlayerState) then
    -- Executes the AppliedItem method of the collided player entity.
    event.TriggerBodyEntity.PlayerState:AppliedItem(self.valueType,
self.targetPlayerStateName, self.stringValue, self.numberValue, self.gainEffect)
    -- Deactivates the target entity.
    self:ReturnSelf()
end
end

end

```

## Creating an ItemHelper

- The ItemHelper is a Logic that requests item creation when a monster dies.
- When the ItemHelper receives a request, and if an item needs to be dropped, then an item creation request is sent to the ItemSpawnManager.
- The Logic was created with the name ItemHelper in the sample world.
- The code is written based on Plain Text.

```

@Logic
script ItemHelper extends Logic

property ItemSpawnManager itemSpawnManager = nil

method void CalcDropItemData(number ItemDropRate, string MustDropItemKey, Vector3
SpawnPosition)
-- Returns nil without executing the probability calculation, since the item drop
rate is 0.
if not invalid(ItemDropRate) or ItemDropRate <= 0.0 then
    return
end

-- Determines the item drop probability in any other case.
if ItemDropRate >= 1.0 then
    -- No additional calculations are made, since the item must be dropped.
else
    -- Executes random based on the ItemDropRate value.
    local randomValue = _UtilLogic:RandomDouble()
    if ItemDropRate < randomValue then
        -- Since the random value is higher than the Item's drop probability, it
considers the drop to be a failure and returns nil.
        return
    end
end

-- The target item is always returned if a MustDropItemKey is set.
if not _UtilLogic:IsNilorEmptyString(MustDropItemKey) then

```

```

-- Requests data from DataTableLogic based on MustDropItemKey.
local findData = _DataTableLogic:GetItemDataByItemID(MustDropItemKey)
-- Creates an item if there is data that has a MustDropItemKey value.
if invalid(findData) and next(findData) then
    self:SendSpawnItem(findData.ItemID, SpawnPosition)
    return
else
    -- Leaves an error log saying that the item couldn't be found if there are
no such items.
    log("Couldn't find an item that is MustDropItemKey!")
    return
end
end

-- Executes random, so a random item is provided if MustDropItemKey is blank.
local getDropData = _DataTableLogic:GetItemDropData()
local totalValue = 0
-- Attempts to calculate the probability after adding all of the probabilities in
the item drop table.
for _, data in pairs(getDropData) do
    totalValue = totalValue + data.DropRate
end

-- Returns an item that has greater timing than the randomized value by
determining the value between 0 and totalValue and then adding DropRate from 1 as
the winner.
local randomValue = _UtilLogic:RandomIntegerRange(0, totalValue)
local resultValue = 0
for _, result in pairs(getDropData) do
    -- Adds DropRate to resultValue.
    resultValue = resultValue + result.DropRate
    -- If the resultValue value is higher than the determined randomValue, it is
processed as the winner.
    if resultValue >= randomValue then
        -- Searches the ItemData from DataTableLogic with ItemID.
        local findData = _DataTableLogic:GetItemDataByItemID(result.ItemID)
        -- Returns the data that was searched if the data is normally loaded.
        if invalid(findData) and next(findData) then
            self:SendSpawnItem(findData.ItemID, SpawnPosition)
            return
        else
            -- Leaves a log and returns nil if the data that was found is nil or is
data that can't be used.
            log("Couldn't find a value with "..result.ItemID.." as the name from
ItemData!")
            return
        end
    end
end
end
end
end

```

```

method void SendSpawnItem(string ItemID, Vector3 SpawnPosition)
-- Generates an error log if no ItemSpawnManager is found InGame.
if not invalid(self.itemSpawnManager) then
    log("The ItemHelper's itemSpawnManager is nil!")

```

```

        return
    end
    -- Directs the InGame's ItemSpawnManager to create an item.
    self.itemSpawnManager:CreateItem(ItemID, SpawnPosition)
end

method void InitData()
    -- Searches for the InGame ItemSpawnManager if the InGame ItemSpawnManager
    couldn't be found.
    if not isvalid(self.itemSpawnManager) then
        local findEntity =
        _EntityService:GetEntityByPath("/maps/InGame/ItemSpawnManager")
        -- Generates an error log if the ItemSpawnManager couldn't be found.
        if not isvalid(findEntity.ItemSpawnManager) then
            log("Either the ItemSpawnManager entity or the ItemSpawnManager component
            doesn't exist in the InGame map!")
            return
        end
        -- Registers the found ItemSpawnManager to property.
        self.itemSpawnManager = findEntity.ItemSpawnManager
    end
end

method void ReleaseData()
    -- Accesses the ItemSpawnManager from the Title screen and disables it, so it
    can't be used.
    self.itemSpawnManager = nil
end

end

```

## Minimum Implementation Standards

- The ItemHelper needs to be able to output random items as intended.
- The ItemComponent needs to be able to receive and apply DataSet values from ItemData.

## Usage and Extension Direction

- Create items that provide effects different from other created items.
- Implement the projectile enhancement function and add the item acquisition effect to implement a basic attack enhancement system.

---

## [📖 Advanced Training] Creating a MonsterSpawn

---

## Video Timeline

- This content is discussed together with 'Implementing Items'.
- Implementing Items: 02:32:10

## Learning Goals

- Learn how to use TimeService to execute specific actions after a set duration.
- Use GameLogic to implement a function that can check game over and stage clear statuses.

## Practical Exercises to Do After Watching the Video

- Change the MonsterSpawnData value to adjust when the Monster is output.
- Create an additional Monster and then edit MonsterSpawnData, so that Monster can spawn.
- Edit ItemDropRate and MustDropItem to change the chance of an item dropping when each Monster is defeated.

## Creating a DataSet

- Create a DataSet to spawn a Monster.
- The sample world created one with the name **MonsterSpawnData**.

|    | 1       | 2         | 3         | 4         | 5            | 6            | 7             |  |
|----|---------|-----------|-----------|-----------|--------------|--------------|---------------|--|
| ID | StageID | MonsterID | SpawnPosX | SpawnPosY | EntranceTime | ItemDropRate | MustDropItem  |  |
| 1  | 1       | Bat_1     | 8.0       | 2.0       | 3.0          | 0.0          |               |  |
| 2  | 1       | Bat_1     | 8.0       | -2.0      | 5.0          | 0.8          |               |  |
| 3  | 1       | Bat_1     | 6.0       | 0.0       | 8.0          | 1.0          | Item_MoveSpee |  |
| 4  | 1       | Bat_1     | 6.0       | -5.0      | 8.0          |              |               |  |
| +  |         |           |           |           |              |              |               |  |

| Name                    | Description                                                                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| StageID                 | The ID value used to identify the Monster type that needs to be spawned by the MonsterSpawnManager based on the current stage number.           |
| MonsterID               | The ID value used to search for which Monster should spawn. This value is used to search the MonsterID from MonsterDefaultData.                 |
| SpawnPosX,<br>SpawnPosY | The value that determines where on the map the Monster spawns using the X and Y values of each Position.                                        |
| EntranceTime            | The value that determines how many seconds after the game starts when the target Monster should spawn.                                          |
| ItemDropRate            | The value used to designate the item drop chance when a Monster dies.                                                                           |
| MustDropItem            | The value used when you want to have fixed item drops when a Monster needs to drop an item. This value uses the ItemID value found in ItemData. |

## Creating a MonsterSpawnManager

- An entity needs to be added to InGame to create a MonsterSpawnManager.
- Also, the MonsterSpawnManager Component must be composed.
- The following is the code that composes the MonsterSpawnManager Component.
- This code was written based on Plain Text.

```
@Component
script MonsterSpawnManager extends Component

property number stageID = 0 -- Property that categorizes stages.

property number needKillCount = 0 -- Property that calculates and saves the number
of monsters that can be defeated when the stage starts.

property table timerTable = {} -- Table that saves the time when the monsters
appear.

@ExecSpace("Client")
method void StartGame()
-- Resets the number of monsters that need to be defeated to clear the stage to 0
first.
self.needKillCount = 0
-- Resets the timerTable that contains the stage's monster appearance times.
self.timerTable = {}
-- Sets the stageID to 1, so the game can run as intended if stageID is 0.
if self.stageID == 0 then
    self.stageID = 1
    -- Notifies through a log that the stageID is currently registered as 0.
    log("StageID hasn't been registered to the SpawnManager!")
    return
end
-- Fetches the monster spawn data from DataTableLogic according to stageID.
local getData = _DataTableLogic:GetMonsterSpawnDataTableByStageID(self.stageID)
-- Generates an error through the log if the getData is unusable.
if not isValid(getData) or next(getData) == nil then
    log("Couldn't load the table value from the "self.stageID.." data table!")
    return
end
-- Counts the number of getData found and processes the getData values one at a
time.
for i = 1, #getData do
    -- Registeres the data at index i from getData to data.
    local data = getData[i]
    -- Attempts to spawn monsters through the data values if data is usable.
    if data then
        -- Increases the number of monsters that must be defeated to clear the
stage by 1, since there is a usable monster.
        self.needKillCount = self.needKillCount + 1
        -- Sets the timer for monster spawns.
        self.timerTable[i] = _TimerService:SetTimerOnce(function()
            -- Requests the data of monsters that have to be spawned from
```

```

DataTableLogic.
    local monsterData =
_DataTableLogic:GetMonsterDefaultDataByMonsterID(data.MonsterID)

    -- Creates monsters that need to be created.
    local monster = _SpawnService:SpawnByModelId(
        -- Searches for the Model through the monster's ID and creates the
target Model.
        monsterData.ModelRUID,
        - Changes the entity's name to MonsterID.
        monsterData.MonsterID,
        -- Sets the creation position of the monster that you are trying
to create.
        Vector3(data.SpawnPosX, data.SpawnPosY, 0),
        -- Sets the entity that will be created as its child in the
Hierarchy.
        self.Entity
    )
    -- Resets the value if a monster was created normally.
    if monster then
        -- Resets the monster's position to the MONsterSpawnData again one
more time.
        monster.TransformComponent.Position = Vector3(data.SpawnPosX,
data.SpawnPosY, 0)
        -- Resets the monster data according to MonsterDefaultData.
        monster.MonsterState:InitData(monsterData, data.ItemDropRate,
data.MustDropItem)
    end
end,
    -- Registers the time, so the creation can happen based on the spawn time
registered in MonsterSpawnData.
    data.EntranceTime)
end
end
-- Delivers the number of monsters that must be defeated to clear the stage to
Gamelogic if all monster creation reservations are complete.
_GameLogic:InitClearKillCount(self.needKillCount)
end

@ExecSpace("Client")
method void ClearAllChild()
    -- Searches for all of the child entities and registers them to the chidList.
    local childList = self.Entity.Children
    -- Immediately ends the code if the childList is nil.
    if childList == nil then return end
    -- Navigates and processes the amount of time equal to the number of childList
since chidList holds 1 or more.
    for i, child in pairs(childList) do
        -- Removes the child element if the entity at index i is usable.
        if isvalid(child) then
            _EntityService:Destroy(child)
        end
    end
end
-- Starts a navigation of the timerTable elements if there are elements in

```

```
timerTable.  
if self.timerTable ~= nil and next(self.timerTable) ~= nil then  
    -- Removes the scheduled timer at index i.  
    for i = 1, #self.timerTable do  
        _TimerService:ClearTimer(self.timerTable[i])  
    end  
end  
end  
end  
  
end
```

## Minimum Implementation Standards

- Change MonsterSpawnData to adjust the portions related to Monster spawns.
- You must be able to add Monsters to spawn a variety of Monsters.
- Adjust the item drop chances for each monster through ItemDropRate and MustDropItem.

## Usage and Extension Direction

- Manage stage progression through number in GameLogic and, when InGame begins, change the StageID value of MonsterSpawnManager.
- Implement a function that changes the Monster that spawns based on the MonsterSpawnData's StageID value.

---

## [💡 Basic Training] Composing UI

---

### Video Timeline

- You can learn from the timeline under the training video.
- Implementing UI: 03:00:20

### Learning Goals

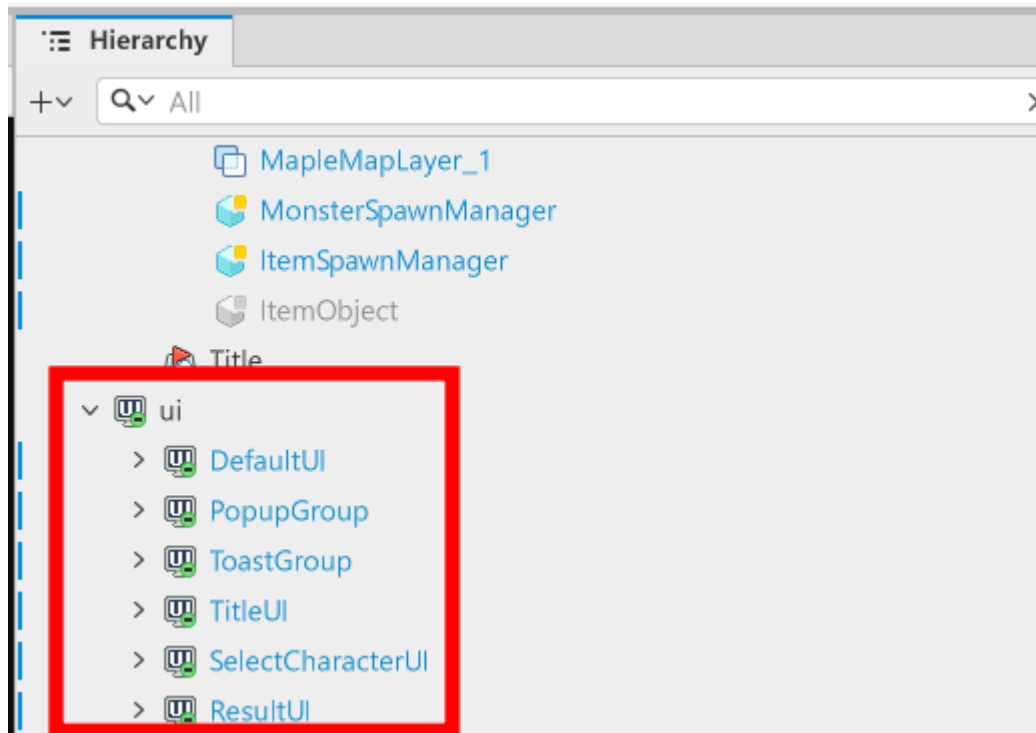
- Learn how to compose a UI in MapleStory Worlds.
- Learn about the localization function and how to apply localization.
- Understand the role of UIManageLogic and learn how to use it.
- Reference the sample world's code to create a UI that fits the game you want to create and apply functions as needed.

### Practical Exercises to Do After Watching the Video

- Directly implement functions implemented in the sample world.

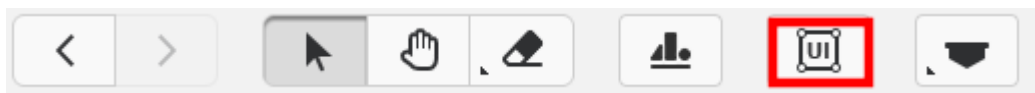
- Change the UI in the implementation stage to create UI that makes the world more unique.
- Create a new UIGroup and manage it to add UI to deliver the necessary information.

## Understanding UI Groups



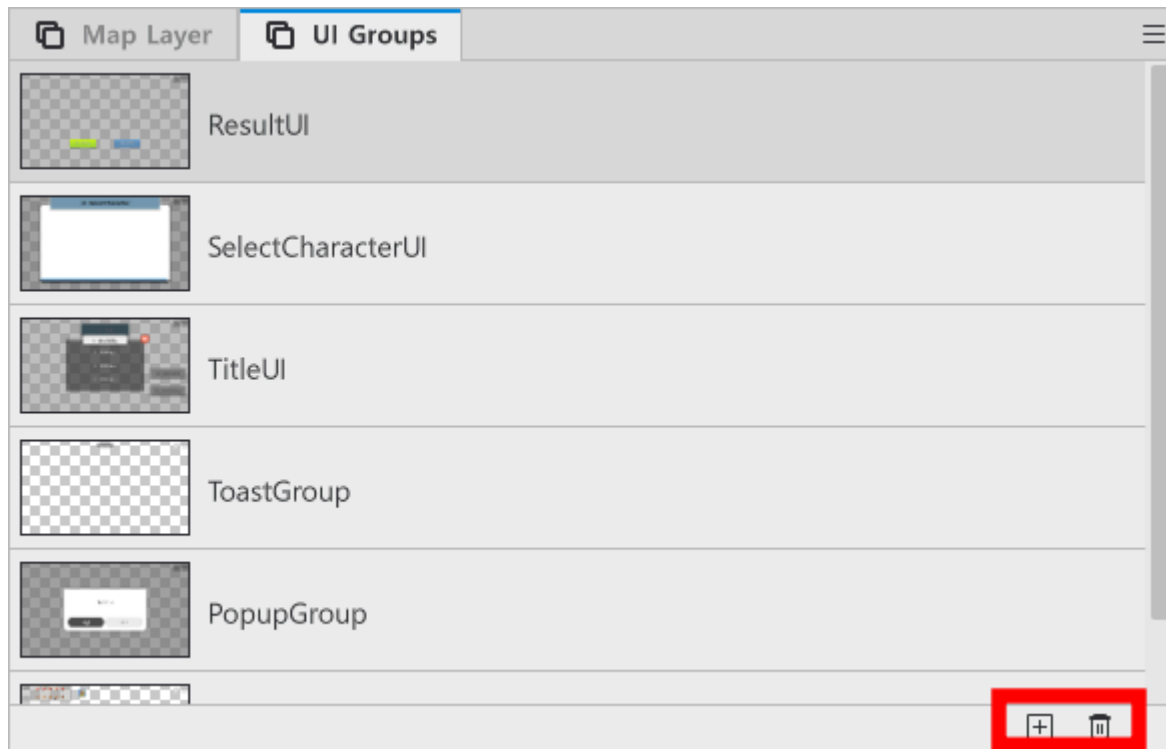
- Below the Hierarchy window are entities that are composed with the name ui.
- Those entities are a ui group that outputs the ui.
- Edit the ui group to change or create the ui that needs to be output.

## Creating a UI Group



- Click the UI button on the top of the editor to change to UI editing mode.

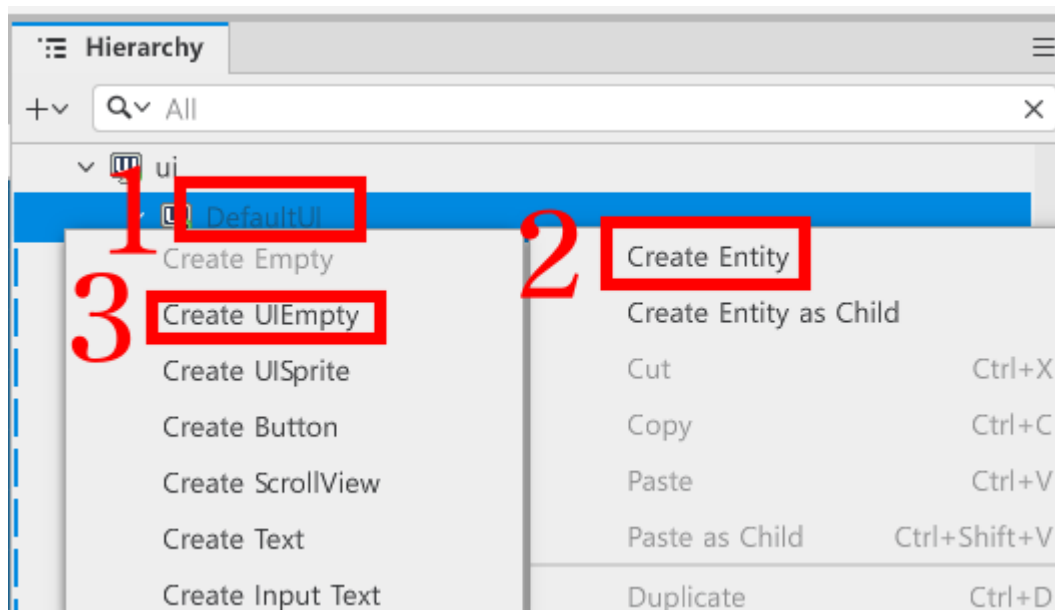




- Click on the + button from UI Groups on the bottom of the editor window to add new UI Groups.
- If you have UI Groups that needs to be deleted, click the UI Group to delete and click on the recycling bin icon to delete it.

## Creating Empty UI Entities

- You can create entities in the UI like you did with existing entities.
- The difference is when creating an empty entity, you must use **Create UIEmpty** instead of Create Entity.



- Right-click the ui group you want to add an entity to from the Hierarchy window.
- Click Create Entity.
- Click Create UIEmpty to create an empty UI entity.

## Applying a Component to a UI Entity

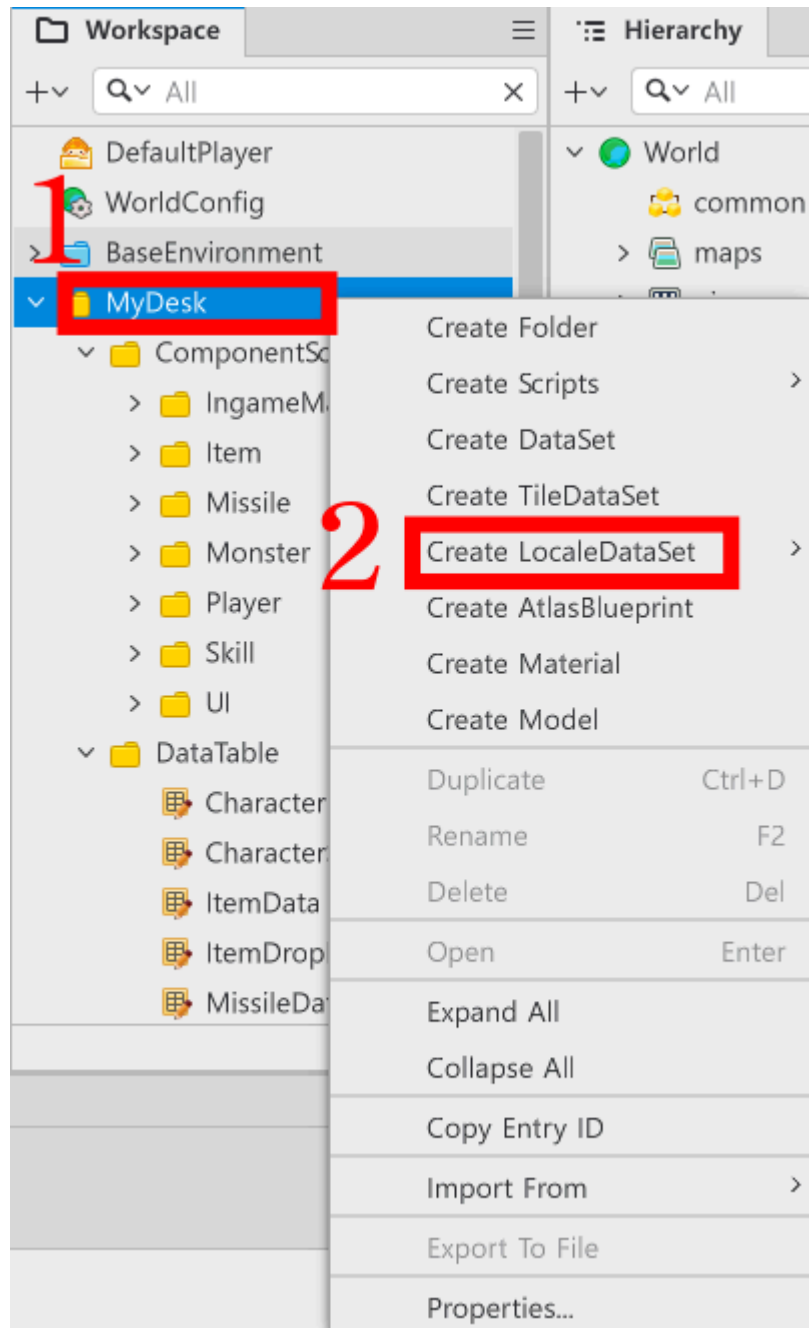
- Like normal entities, you can add Components to UI entities to execute additional functions.
- Adding Components is done the same way as the Add Component function for regular entities.
- This lesson will talk about the often used Components.
  - **SpriteGUIRendererComponent**: A Component used when outputting an image for the UI. It's recommended to use this Component rather than SpriteRendererComponent when outputting images in the UI.
  - **TextGUIRendererComponent**: A Component used when outputting text on the UI.
  - **ButtonComponent**: A Component used when the UI needs a button function.

## Applying a Controller to a UIGroup

- The Controller is used to create a Component and manage a target UI.
- Apply a Controller Component to each UIGroup and compose functions, so each Component serves a UI-related function.
- Register each Controller to UIManageLogic and call UIManageLogic to request UI-related functions to a different UI.
- The following UIControllers were created in the sample world.
  - **DefaultUIController**: A Component used to manage the DefaultUI. It usually manages player HP and skill use count updates, and it also takes the role of registering the player's basic attacks and skill use function to each corresponding button.
  - **TitleUIController**: A Component used to compose the title screen. This Component was created to process the Start Game button and How to Play UI output functions.
  - **ResultUI**: When the GameLogic delivers the game's results to UIManagLogic, the UIManageLogic requests that target Component which results UI should be output.

## Creating a Localization DataSet

- The localization function is a requirement when providing the service in different countries.
- You can create the relevant functions by using the Locale DataSet in MapleStory Worlds.



- Right-click MyDesk from Workspace.
- Click Create LocaleDataSet.
- Click Default to create a Locale DataSet.
- The sample world is created with a StringTable.

|    | 1               | 2             | 3         | 4         | 5                |
|----|-----------------|---------------|-----------|-----------|------------------|
| ID | Key             | Source        | Note      | ko        | en               |
| 1  | UI_Title        | Shooting Game | 타이틀에 사용   | 슈팅 게임 프레  | Shooting Game    |
| 2  | UI_StartGame    | Start Game    | 타이틀 화면의   | 게임 시작     | Start Game       |
| 3  | UI_Pirate       | Pirate        | UI표기용 직업명 | 해적        | Pirate           |
| 4  | UI_SelectCharac | Select        | 캐릭터 선택 창  | 캐릭터 선택    | Select Character |
| 5  | UI_Retry        | Retry         | 결과창에서 다시  | 다시하기      | Retry            |
| 6  | UI_Exit         | Exit          | 결과창의 나가기  | 나가기       | Exit             |
| 7  | UI_HowToPlay    | How to play   | 게임 방법 버튼  | 플레이 방법    | How to play      |
| 8  | UI_InfoPlay1    | Move          | 게임 방법 이동  | 이동 : WSAD | Move: WSAD       |
| 9  | UI_InfoPlay2    | Attack        | 게임 방법 공격  | 공격 : J    | Attack : J       |
| 10 | UI_InfoPlay3    | Skill         | 게임 방법 스킬  | 필살기 : K   | Skill : K        |

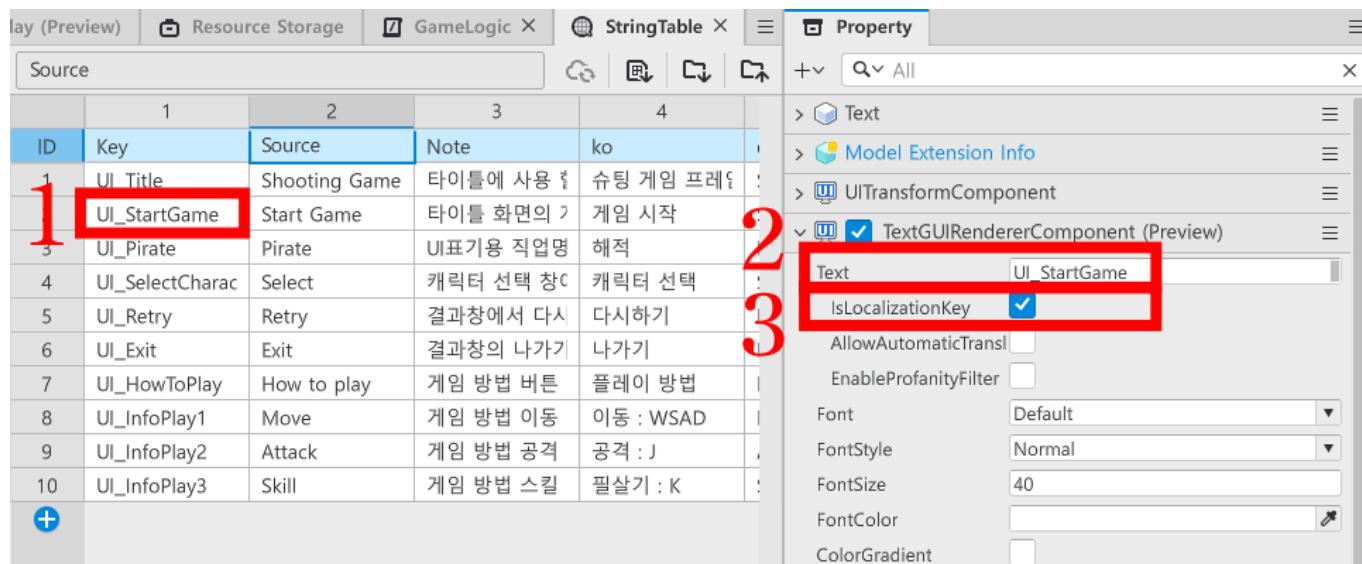
| Name   | Description                                                                                                                                                                                |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Key    | The ID value used to search the text that needs to be localized when MapleStory Worlds proceeds with localizing.                                                                           |
| Source | The value that holds the source text of the translated text. When using the automatic translation function provided by MapleStory Worlds, the translation value is found using this value. |
| Note   | The space that has information about the reasoning and use of the target text.                                                                                                             |
| ko     | The value that holds the text that will be output in Korean.                                                                                                                               |
| en     | The value that holds the text that will be output in English.                                                                                                                              |

- You can also add languages that are officially supported in MapleStory Worlds when creating a world.

## Applying Localization

- There are largely 2 ways to apply the localization function.
- You can use the TextGUIRendererComponent value or use scripts.

### How to Utilize the TextGUIRendererComponent



- Enter the localized value you want to apply to the created Locale DataSet.
- Enter the Locale DataSet's key to TextGUIRendererComponent's Text value.
- Change the TextGUIRendererComponent's isLocalizationKey value to the checked state.
- You can ensure that the localization function will be applied in this state.

## How to Utilize the Script

- The sample world created a LocalizeLogic for expedient use.
- The following is the code for the created LocalizeLogic.
- This code was written based on Plain Text.

```
@Logic
script LocalizeLogic extends Logic

@ExecSpace("ClientOnly")
method string GetLocalizeStringByKey(string StringKey)
-- Searches and returns the value in StringTable based on the received StringKey
value.
return _LocalizationService.GetText(StringKey)
end

end
```

- When delivering the Locale DataSet Key value as a StringKey parameter, text that matches the language of the country that the game is running in is output.
  - Example: When the country the game is played in is an English-speaking language like America, the en value is output.

## Utilizing UIManageLogic

- If all of the UI settings are complete, then each Controller is registered to UIManageLogic, and the function must be set-up.
- The following is the UIManageLogic code.
- The following code is written based on a Plain Text environment.

```
@Logic
script UIManageLogic extends Logic

property TitleUIController titleUIController = nil

property SelectCharacterUIController selectCharacterUIController = nil

property ResultUIController resultUIController = nil

property DefaultUIController defaultUIController = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- Searches for and registers TitleUIController when the game starts.
self:InitTitleUIController()
-- Searches for and registers SelectCharacterUIController when the game starts.
self:InitSelectCharacterUIController()
-- Searches for and registers ResultUIController when the game starts.
self:InitResultUIController()
-- Searches for and registers DefaultUIController when the game starts.
self:InitDefaultUIController()
end

@ExecSpace("ClientOnly")
method void InitTitleUIController()
-- If there is no registered TitleUIController, registers the target component.
if not isvalid(self.titleUIController) then
    local titleUIEntity = _EntityService:GetEntityByPath("/ui/TitleUI")
    -- If titleUIGroup is successfully found, registers the target entity's
    component.
    if isvalid(titleUIEntity) then
        local component = titleUIEntity.TitleUIController
        -- Determines whether the component has been registered normally and then
        assigns it.
        if isvalid(component) then
            self.titleUIController = component
        end
    end
end
end

@ExecSpace("ClientOnly")
method void InitSelectCharacterUIController()
-- If there is no registered SelectCharacterUIController, registers the target
component.
if not isvalid(self.selectCharacterUIController) then
    local selectCharacterUIEntity =
```

```

_EntityService:GetEntityByPath("/ui/SelectCharacterUI")
    -- If SelectCharacterUIController is successfully found, registers the target
entity's component.
    if invalid(selectCharacterUIEntity) then
        local component = selectCharacterUIEntity.SelectCharacterUIController
        -- Determines whether the component has been registered normally and then
assigns it.
        if invalid(component) then
            self.selectCharacterUIController = component
        end
    end
end
end

@ExecSpace("Client")
method void ShowTitle()
    -- Shows the title screen.
    self.titleUIController:Show()
    -- Hides all other UI.
    self.selectCharacterUIController:Hide()
    self.resultUIController:Hide()
    self.defaultUIController:Hide()
end

@ExecSpace("Client")
method void CallSelectCharacterUI()
    -- Shows the character selection screen.
    self.selectCharacterUIController:Show()
    -- Attempts to reset the character list in the character selection screen.
    self.selectCharacterUIController:InitCharacterSelectFrame()
end

@ExecSpace("Client")
method void ShowInGame()
    -- Resets the defaultUI of the game screen UI.
    self.defaultUIController:Show()
    -- Requests a reset of the auto-attack UI.
    self.defaultUIController:UpdateAutoUI()
    -- Hides other UI.
    self.titleUIController:Hide()
    self.selectCharacterUIController:Hide()
    self.resultUIController:Hide()
end

@ExecSpace("Client")
method void InitResultUIController()
    -- If there is no registered ResultUIController, registers the target component.
    if not invalid(self.resultUIController) then
        local resultUIEntity = _EntityService:GetEntityByPath("/ui/ResultUI")
        -- If ResultUIController is successfully found, registers the target entity's
component.
        if invalid(resultUIEntity) then
            local component = resultUIEntity.ResultUIController
            -- Determines whether the component has been registered normally and then

```

```
assigns it.
    if invalid(component) then
        self.resultUIController = component
    end
end
end
end

@ExecSpace("Client")
method void ShowResultUI(boolean IsClear)
-- Displays the results UI.
self.resultUIController:Show(IsClear)
if IsClear then
    -- Play Congratulations Sound Effect
    _SoundService:PlaySound("5d57b617cfc242cca347925e209c8e05", 1.0)
else
    -- Play Failure Sound Effect
    _SoundService:PlaySound("52dccfb8889b4181b9181780dc7e9040", 1.0)
end
end

@ExecSpace("Client")
method void InitDefaultUIController()
--If there is no registered defaultUIController, registers the target component.
if not invalid(self.defaultUIController) then
    local titleUIEntity = _EntityService:GetEntityByPath("/ui/DefaultUI")
    --If DefaultUIGroup is successfully found, registers the target entity's
    component.
    if invalid(titleUIEntity) then
        local component = titleUIEntity.DefaultUIController
        -- Determines whether the component has been registered normally and then
        assigns it.
        if invalid(component) then
            self.defaultUIController = component
        end
    end
end
end
end

@ExecSpace("Client")
method void UpdateLifeCountUI(number PlayerLife)
-- Updates the DefaultUI's HP UI in accordance to the PlayerLife number.
self.defaultUIController:UpdateLifeUI(PlayerLife)
end

@ExecSpace("Client")
method void UpdateSkillCountUI(number LeftSkillCount)
-- Updates the DefaultUI's skill UI in accordance with the LeftSkillCount number.
self.defaultUIController:UpdateSkillUI(LeftSkillCount)
end

end
```



- The UI Group's Controllers have been registered, and the method has been created based on the UI updates needed.
- When there is a UI update request in GameLogic or PlayerState, the Controller in charge is called and executes the relevant function.

## Minimum Implementation Standards

- Compose the UIManageLogic, so the Controller that manages the UI and the Controller's functions can be used conveniently from the outside.
- An interaction must be triggered when a button-like element is clicked in the UI.
- The UI must deliver the information needed to progress in the game.

## Usage and Extension Direction

- Create an Event, so when the player gets hit, the update is triggered without making a update request to UIManageLogic.
  - This method becomes the groundwork for synchronizations when making future communication between the Server and Client.
- Delivers visual information to users when a new UI is created or by adding effects as needed.

## Closing Remarks

- The composition of UI can vary widely based on the type and direction you want to take the game.
- The sample world has a UI created to show the overall flow, and you don't need to follow it to the letter.
- If anything, please use your own unique ideas and implement them to create a fantastic game never seen before.